
ERP Reporting & Consolidation Framework

1. Core Stock & Movement Reports

The ERP must generate detailed reports at the **department**, **sector**, and **enterprise-wide** levels, with full flexibility for organizations that may have:

- Different **numbers of sectors/departments** within the same tier.
- Different **numbers of tiers/levels** in their organizational hierarchy.

Reports must include:

- **Opening Stock** (by units, weight, volume, and value).
- **Purchases** (local, imports, with landed cost allocation).
- **Interdepartment Transfers:**
 - **Horizontal Transfers** (same tier/section): *recorded individually but excluded from consolidation.*
 - Example: Feed or medicines moved between poultry houses.
 - **Vertical Transfers** (different tiers/sections): *included in consolidation.*
 - Example: Live birds moved from chicken houses to the abattoir.
- **Production Outputs** (finished goods, by-products, wastage).
- **Sales & Invoices** (gross sales, returns, discounts, net revenue).
- **Closing Stocks** (valuation by FIFO, LIFO, weighted average, FEFO, or scenario costing).

2. Extended Reports

- **Yield Reports** (outputs vs. inputs, efficiency tracking).
- **Abnormal Loss & Gain Reports** (damages, spoilage, system corrections).
- **Surplus/Shortage Reports** (stock take variances).
- **Profit & Loss Statements** (per department, sector, and enterprise).
- **Balance Sheet & Cash Flow Statements** (with consolidation rules).
- **Asset Records/Reports** (fixed assets, consumables, disposals, revaluations).
- **Financial Ratios** (liquidity, solvency, profitability, efficiency).

3. Special Inventory Features

- **Consignment In & Out** (third-party stock handling).
 - **Returns** (purchase returns, sales returns).
 - **Expiry & FEFO Compliance** (expiry tracking with first-expiry-first-out enforcement).
 - **Landed Cost Allocator** (import duties, shipping, insurance allocation).
 - **Serial & Lot Tracking** (traceability, regulatory compliance).
 - **Quality Holds** (stock quarantined for inspection/testing).
 - **Audit Pack** (auto-generated audit-ready report set).
 - **Scenario Costing** (simulate alternative costing methods or transfer prices).
 - **Activity-Based Overhead Allocation** (overheads distributed by usage drivers).
-

4. Cross-Industry Usability

All reporting and consolidation logic must be **generic enough** to apply across:

- Agriculture (inputs → outputs → processing → retail).
 - Manufacturing (raw materials → WIP → finished goods).
 - Services (resource allocation, billing, utilization).
 - Distribution/Wholesale/Retail (multi-tier warehousing and shops).
 - Public sector institutions (multi-branch consolidation, regulatory reporting).
-

5. Consolidation Rules

To ensure **accuracy and avoid duplication**:

1. **Department-Level Reports** → Detailed and stand-alone.
 2. **Sector-Level Consolidation** → Aggregate multiple departments.
 - **Include** vertical transfers (as value is realized at next tier).
 - **Exclude** horizontal transfers (no net value creation across same tier).
 3. **Enterprise-Wide Consolidation** → Combine all sectors.
 - Apply same inclusion/exclusion rules.
 - Ensure eliminations for intercompany/sector purchases and transfers.
-

6. Additional Factors

- **Multi-Currency Support** (local and foreign exchange, revaluations).
- **Regulatory Compliance** (Smart Invoice, ZRA, VAT, PAYE integration).
- **Audit Trail & Logs** (who posted, approved, adjusted).
- **Configurable Reports** (daily, weekly, monthly, quarterly, yearly).
- **Drill-Down Capability** (from enterprise → sector → department → transaction).
- **Flexible Sector Definitions** (user-defined tiers, dynamic scaling).

- **AI Forecasting Hooks** (predictive yield, cost, sales).
-

✓ This framework now captures the consolidation rule:

- For **same-tier consolidation**, **vertical transfers are included** while **horizontal transfers are excluded**.
 - For **cross-tier consolidation**, both interdepartment vertical transfers and all other valid transactions are included.
-

Here's a **full, exhaustive rewrite** that captures every single requirement, with structure, principles, and all the technical and business logic, so nothing gets lost. I'll treat this as the **ERP Reporting & Consolidation Master Specification**.

ERP Reporting & Consolidation Master Specification

1. Design Goals & Core Principles

The reporting and consolidation framework for the ERP must be designed with the following guiding principles:

1. **Accuracy & Integrity**
 - No double counting, duplication, or misclassification.
 - Reports must reconcile back to the **Stock Ledger** and **General Ledger**.
 - Consolidation rules must eliminate duplicate intra-entity movements.
 2. **Flexibility & Extensibility**
 - Supports variable **organization structures** with different numbers of tiers and departments.
 - Works across **multiple industries** (agriculture, manufacturing, services, trade, public sector).
 3. **Granularity & Traceability**
 - Reports available at **item batch/lot/serial**, department, sector, and enterprise level.
 - Traceability from consolidated balances down to **transaction-level movements**.
 4. **Multi-Dimensional Reporting**
 - Units of Measure (UoM): pieces, weight, volume, and monetary value.
 - Configurable costing methods: FIFO, LIFO, Weighted Average, Standard, Scenario Costing.
 5. **Integration & Automation**
 - Full integration with **GL postings, tax modules, audit trails**, and **Smart Invoice/ZRA compliance**.
 - Automated **validations** (stock cannot go negative, all transfers must net to zero).
-

2. Organizational Model (Variable Tiers)

- **Tier Structure:** Each organization can define its own hierarchy.
 - Enterprise → Sectors → Departments → Sub-Departments → Batches/Operations
 - **Variable Number of Tiers:** Some organizations may have only 2 levels (Enterprise → Department), others may have 5+.
 - **Same Tier Consolidation Rule:**
 - **Vertical transfers** (cross-level): **included**.
 - **Horizontal transfers** (same level): **excluded** from consolidation (to avoid duplication).
-

3. Item Batches & Units of Measure

- **Batch & Lot Tracking:** Each stock item is linked to batch/lot/serial numbers.
 - **FEFO Compliance:** First-Expiry-First-Out enforced for perishable items.
 - **UoM Handling:**
 - Base UoM (e.g., kg, liter, unit).
 - Secondary UoM for reporting (e.g., cartons, pallets).
 - Conversion factors maintained in **Item Master**.
-

4. Stock Ledger & Movement Taxonomy

Stock Ledger

- Master record of all **movements**, reconciled to GL.
- Each entry must have:
 - transaction_id
 - item_id
 - batch/lot_no
 - movement_type (purchase, production, transfer, sale, adjustment, etc.)
 - uom_qty, weight, volume, monetary_value
 - source_dept, dest_dept
 - reference_doc (PO, SO, invoice, GRN, etc.)

Movement Taxonomy

1. **Opening Balance**
2. **Purchases (Local/Imports)**
 - Landed cost allocation.
3. **Interdepartment Transfers**
 - **Horizontal (same tier):** recorded but excluded from consolidation.
 - **Vertical (cross tier):** included.

4. **Production Outputs** (finished, by-products, scrap).
 5. **Sales/Invoices** (with returns).
 6. **Closing Stock** (valuation by costing method).
 7. **Adjustments** (abnormal loss/gain, stock take).
 8. **Consignments** (inbound/outbound).
 9. **Expiry & Quality Holds**.
-

5. Costing & Transfer Pricing

- **Costing Methods Supported:**
 - FIFO, LIFO, Weighted Average, Standard, FEFO, Scenario Costing.
 - **Transfer Pricing:**
 - Vertical transfers valued at **transfer price** (not just cost).
 - Rules configurable: at cost, cost + markup, or market price.
 - **Activity-Based Overhead Allocation (ABC):**
 - Overheads distributed by drivers (machine hours, labor hours, batch size).
-

6. Core Tables (Minimal Schema)

```
-- Item Master
items (
  item_id PK,
  item_name,
  base_uom,
  alt_uom,
  conversion_factor,
  category,
  expiry_days
);

-- Stock Ledger
stock_ledger (
  txn_id PK,
  item_id FK,
  batch_no,
  movement_type,
  qty,
  weight,
  volume,
  value,
  source_dept,
  dest_dept,
  transfer_type, -- HORIZONTAL / VERTICAL
  reference_doc,
  gl_posted BOOLEAN,
  created_at
```

```

);

-- Departments & Tiers
departments (
    dept_id PK,
    dept_name,
    sector_id,
    tier_level
);

-- Consolidation Rules
consolidation_rules (
    rule_id PK,
    movement_type,
    include_in_consolidation BOOLEAN
);

```

7. Reports & Reconciliations

Each report must **tie back** to either Stock Ledger totals or GL balances:

- **Stock Report** → reconciles with Stock Ledger closing balance.
 - **Purchase Report** → reconciles with AP & GRN totals.
 - **Sales Report** → reconciles with AR & Invoice Register.
 - **Yield Report** → reconciles with production inputs vs. outputs.
 - **Abnormal Loss/Gain Report** → reconciles with adjustments in GL.
 - **P&L, Balance Sheet, Cash Flow** → reconciles with GL (trial balance).
 - **Asset Report** → reconciles with Asset Register and GL Fixed Assets.
-

8. Consolidation Rules (Detailed)

- **Eliminate duplicates:**
 - Same-tier horizontal transfers excluded in consolidated reports.
 - Vertical transfers included but eliminated at receiving tier to avoid double counting.
 - **Enterprise Consolidation:**
 - Sum sector results.
 - Remove inter-sector duplication (like intercompany eliminations).
 - **Audit Pack Validation:**
 - Trial balance must equal 0.
 - Consolidated stock must equal sum of department stock.
-

9. Report Pack (Specs + SQL Skeletons)

Example: Stock Report

```
SELECT
    item_id,
    SUM(CASE WHEN movement_type='OPENING' THEN qty ELSE 0 END) AS opening,
    SUM(CASE WHEN movement_type='PURCHASE' THEN qty ELSE 0 END) AS purchases,
    SUM(CASE WHEN movement_type='TRANSFER' AND transfer_type='VERTICAL' THEN
qty ELSE 0 END) AS vertical_in,
    SUM(CASE WHEN movement_type='TRANSFER' AND transfer_type='VERTICAL' THEN -
qty ELSE 0 END) AS vertical_out,
    SUM(CASE WHEN movement_type='SALE' THEN -qty ELSE 0 END) AS sales,
    SUM(CASE WHEN movement_type='CLOSING' THEN qty ELSE 0 END) AS closing
FROM stock_ledger
WHERE dept_id = :dept_id
GROUP BY item_id;
```

10. GL Integrations

- Every stock movement generates **automatic GL postings**.
 - Purchase → Debit Inventory, Credit AP.
 - Sale → Debit AR, Credit Revenue; Credit Inventory, Debit COGS.
 - Transfer → Debit Receiving Dept, Credit Sending Dept.
 - Adjustment → Debit/credit Stock Adjustment Account.
-

11. Validation Controls

- No negative stocks allowed.
 - Transfers must balance (sending = receiving).
 - All GL postings must balance (DR = CR).
 - Expired items blocked from sales/production.
 - All reports must reconcile to Stock Ledger and GL.
-

12. Cross-Industry Extensibility

- Agriculture → feed, medicine, live animals, slaughter outputs.
 - Manufacturing → raw material, WIP, finished goods.
 - Services → consumables, billing units, staff utilization.
 - Retail/Wholesale → warehouse transfers, sales returns, promotions.
 - Public Sector → CDF allocations, government stock movements.
-

13. Example Calculations

- **Yield** = $(\text{Output Qty} \div \text{Input Qty}) \times 100$
 - **Abnormal Loss** = Expected Yield – Actual Yield
 - **Transfer Valuation** = Qty × Transfer Price
 - **Closing Stock** = Opening + Purchases + Transfers In – Transfers Out – Sales – Losses
-

14. UI/UX Design

- Reports accessed via **Report Center Dashboard**.
 - Filters: by sector, department, item, date range, batch/lot.
 - Drill-down: Enterprise → Sector → Department → Transaction.
 - Export: PDF, Excel, JSON (Smart Invoice).
 - Schedules: Daily, Weekly, Monthly, Yearly auto-generation.
-

15. Developer Tasks Checklist

- ☐ Build Stock Ledger schema & API.
 - ☐ Implement Movement Taxonomy.
 - ☐ Apply Consolidation Rules (horizontal exclusion, vertical inclusion).
 - ☐ Automate GL Postings.
 - ☐ Build Reports (SQL + API endpoints).
 - ☐ Add Validation Controls.
 - ☐ Design UI Report Center.
 - ☐ Enable Exports (PDF, Excel, JSON).
 - ☐ Implement Audit Pack auto-generation.
-

16. Final Notes

- Must ensure **scalability** (multi-company, multi-branch).
- Schema must remain **minimal yet extensible**.
- Reports must **always reconcile** back to Stock Ledger and GL.
- Must be **regulator-ready** (ZRA, Smart Invoice, IFRS/IPSAS).
- Built to be **cross-industry, future-proof, and auditable**.

Here's a clean, exhaustive breakdown of **all report packs** before we touch schema or API. Each pack includes: purpose, scope & levels, filters, core fields (units/weight/volume/value), calculations, consolidation logic (with your same-tier rule), reconciliations, GL links, validations, output & frequency, and a tight SQL skeleton you can adapt later.

Master List of Report Packs (Breakdown First)

Levels everywhere: **Department** → **Sector** → **Enterprise**.

Consolidation rule: for **same-tier consolidation**, **include vertical transfers** but **exclude horizontal transfers**. Vertical = cross-level/section; Horizontal = within same level/section.

1) Opening Stock Report

Purpose: Establish period starting balances by item/batch.

Scope & Levels: By item, batch/lot/serial, location/department; rollups to sector & enterprise.

Filters: Date (as-of), org level, item(s), category, batch/lot, currency.

Core Fields: qty_units, qty_weight, qty_volume, value.

Calcs: Pull last closing balance prior to period start.

Consolidation:

- Dept: no eliminations.
- Sector/Enterprise: exclude same-tier horizontals (n/a for opening), vertical not applicable; just sum opening per node.

Reconciles To: Prior period **Closing Stock Report** and **GL Inventory** opening balances.

GL Links: Opening = prior closing; no new postings.

Validations: No negative opening unless flagged.

Output/Frequency: As-of daily; PDF/Excel.

SQL Skeleton:

```
-- Opening = balance as of :period_start - 1
SELECT item_id, batch_no,
       SUM(qty_units) AS opening_units,
       SUM(qty_weight) AS opening_weight,
       SUM(qty_volume) AS opening_volume,
       SUM(value) AS opening_value
FROM stock_balances
WHERE as_of = (SELECT MAX(as_of) FROM stock_balances WHERE as_of <
:period_start)
```

```
AND dept_id IN (:dept_scope)
GROUP BY item_id, batch_no;
```

2) Purchases & Landed Cost Report

Purpose: Show purchases (local/import), landed cost components and final inventory value impact.

Scope: Supplier, PO/GRN, item/batch, department.

Filters: Date range, supplier, currency, incoterms, landed cost types.

Fields: qty (U/W/V), base cost, duty, freight, insurance, other LC, total landed value, unit landed cost.

Calcs:

- $\text{landed_unit_cost} = (\text{base} + \text{all_LC}) / \text{qty_units}$
- Allocate LC by weight/volume/value as configured.

Consolidation: Sum by node; transfers not relevant here.

Reconciles To: AP GRNs, GL (Inventory Dr, AP Cr), Landed Cost clearing.

GL Links: Dr Inventory, Cr AP; LC Dr Inventory, Cr Accrual/Payables.

Validations: GRN must match PO, tax codes present.

Output: By PO/GRN & by item.

SQL Skeleton:

```
SELECT grn_no, supplier_id, item_id, batch_no,
       SUM(qty_units) qty, SUM(base_cost) base,
       SUM(duty+freight+insurance+other) lc,
       SUM(base_cost + duty + freight + insurance + other) total_value,
       (SUM(base_cost + duty + freight + insurance +
other)/NULLIF(SUM(qty_units),0)) unit_landed_cost
FROM purchase_receipts_v
WHERE receipt_date BETWEEN :from AND :to
      AND dept_id IN (:dept_scope)
GROUP BY grn_no, supplier_id, item_id, batch_no;
```

3) Interdepartment Transfers Report (Horizontal & Vertical)

Purpose: Visibility & control of internal stock movements.

Scope: Source dept → Dest dept; item/batch.

Filters: Date range, source/dest dept, movement class (HORIZONTAL/VERTICAL).

Fields: qty (U/W/V), transfer price/value, cost basis, approvals.

Calcs: Net by direction and class.

Consolidation:

- **Same-tier consolidation:** Include vertical, exclude horizontal.

- Enterprise: eliminate inter-node duplicates by netting Source vs Dest.
Reconciles To: Stock Ledger zero-sum per item across org; **GL** internal transfer accounts.
GL Links: Dr Receiving Dept Inv, Cr Sending Dept Inv (or internal clearing).
Validations: Must net to zero qty/value globally; cannot transfer negative.
SQL Skeleton:

```
SELECT movement_class, source_dept, dest_dept, item_id, batch_no,
       SUM(qty_units) qty_units, SUM(value) value
FROM stock_ledger
WHERE movement_type='TRANSFER'
      AND txn_date BETWEEN :from AND :to
      AND dept_id IN (:dept_scope)
GROUP BY movement_class, source_dept, dest_dept, item_id, batch_no;
```

4) Production Outputs & Consumption (BoM) Report

Purpose: Show WIP consumption vs outputs (FG, by-products, scrap).
Scope: Work orders/batches, items, operations/centers.
Filters: Date range, plant/line, WO status.
Fields: inputs (U/W/V/value), outputs (U/W/V/value), scrap, rework.
Calcs: Standard vs actual consumption, variances; cost roll-up.
Consolidation: Vertical transfers from production to next tier **included**; horizontals excluded at same-tier consolidation.
Reconciles To: Yield Report, COGM/COGS, **GL** WIP/FG.
GL Links: Dr WIP, Cr RM; on completion Dr FG, Cr WIP.
Validations: No output without issued inputs (unless allowed).
SQL Skeleton:

```
-- Inputs
SELECT wo_no, item_id, SUM(qty_units) in_units, SUM(value) in_value
FROM stock_ledger
WHERE movement_type='ISSUE_TO_WO' AND txn_date BETWEEN :from AND :to
GROUP BY wo_no, item_id;

-- Outputs
SELECT wo_no, item_id, SUM(qty_units) out_units, SUM(value) out_value
FROM stock_ledger
WHERE movement_type='RECEIPT_FROM_WO' AND txn_date BETWEEN :from AND :to
GROUP BY wo_no, item_id;
```

5) Yield & Efficiency Report

Purpose: Measure conversion effectiveness by product/WO/department.
Scope: Inputs vs outputs by batch/WO.
Filters: Date, product family, process line.

Fields: input units/weight/volume/value; outputs; standard yield; actual yield; variance.

Calcs:

- $\text{actual_yield\%} = \text{outputs_units} / \text{inputs_units} * 100$
- $\text{variance\%} = \text{actual} - \text{standard}$

Consolidation: Same rules; yields computed at the node, roll up by weighted averages.

Reconciles To: Production, Costing, P&L variances.

GL Links: Production variance to P&L.

SQL Skeleton:

```
SELECT wo_no,  
       SUM(in_units) in_units,  
       SUM(out_units) out_units,  
       (SUM(out_units)/NULLIF(SUM(in_units),0))*100 actual_yield_pct  
FROM prod_io_summary_v  
WHERE period BETWEEN :from AND :to AND dept_id IN (:dept_scope)  
GROUP BY wo_no;
```

6) Sales & Invoices Report (with Returns)

Purpose: Revenue, quantity, pricing, returns.

Scope: Customer, invoice/credit note, item/batch, channel.

Filters: Date, customer, channel, currency, tax codes.

Fields: qty (U/W/V), gross, discounts, tax, net, returns.

Calcs: Net sales = invoices – returns; margins via COGS link.

Consolidation: No transfer eliminations; sales are external.

Reconciles To: AR subledger, **GL** Revenue/AR/VAT.

GL Links: Dr AR, Cr Revenue; returns reverse.

Validations: Tax mapping; Smart Invoice payloads present.

SQL Skeleton:

```
SELECT doc_no, customer_id, item_id, batch_no,  
       SUM(qty_units) qty, SUM(net_amount) net_amount, SUM(tax_amount) tax  
FROM ar_invoices_v  
WHERE doc_date BETWEEN :from AND :to  
       AND dept_id IN (:dept_scope)  
GROUP BY doc_no, customer_id, item_id, batch_no;
```

7) Closing Stock Report (Multi-UoM & Valuation)

Purpose: Period end inventory by item/batch with valuation.

Scope: Item/batch/location; valuation per method (FIFO/WAvg/etc.).

Filters: As-of date, method, currency.

Fields: qty (U/W/V), unit cost, total value, ageing buckets.

Calcs: Cost via method engine; ageing days.

Consolidation: Exclude same-tier horizontals; include vertical where applicable.

Reconciles To: GL Inventory control; Opening + Moves – Sales = Closing.

GL Links: Inventory control balance.

SQL Skeleton:

```
SELECT item_id, batch_no, SUM(qty_units) qty,
       valuation_method, SUM(valued_amount) value
FROM inventory_valuation_v
WHERE as_of=:as_of AND dept_id IN (:dept_scope)
GROUP BY item_id, batch_no, valuation_method;
```

8) Abnormal Loss & Gain Report

Purpose: Identify shrinkage, write-offs, unexpected recoveries.

Scope: By item/batch/department.

Filters: Date, reason codes, approver.

Fields: qty (U/W/V), value, reason, doc refs.

Calcs: Loss = negative adjustments; Gain = positive adjustments outside normal ops.

Consolidation: Rollups; no transfer eliminations (adjustments are local).

Reconciles To: Stock adjustments GL, Audit pack.

GL Links: Dr/Cred Stock Adj vs P&L.

SQL Skeleton:

```
SELECT reason_code, item_id, SUM(qty_units) qty, SUM(value) value
FROM stock_ledger
WHERE movement_type='ADJUSTMENT'
      AND abnormal_flag=TRUE
      AND txn_date BETWEEN :from AND :to
GROUP BY reason_code, item_id;
```

9) Surplus/Shortage (Stocktake Variance) Report

Purpose: Count variances between physical and system.

Scope: Cycle count or full count events.

Filters: Count event, area, item/batch.

Fields: book qty/value vs physical qty/value; variance qty/value.

Calcs: Variance = physical – book.

Consolidation: Local per node; roll up summaries.

Reconciles To: Adjustment postings; GL stock adjustments.

GL Links: Dr/Cr inventory vs variance account.

SQL Skeleton:

```
SELECT event_id, item_id, batch_no,
       book_qty, physical_qty,
       (physical_qty - book_qty) variance_qty,
       variance_value
FROM stocktake_variances_v
```

```
WHERE event_id=:event_id AND dept_id IN (:dept_scope);
```

10) Consignment In/Out Report

Purpose: Track third-party stock you hold (in) and your stock held by others (out).

Scope: Consignor/consignee, item/batch, location.

Filters: Date, partner, consignment status.

Fields: qty (U/W/V), value (memo for consignment in if off-balance), on-hand vs issued/returned.

Consolidation: Kept separate from owned inventory in enterprise consolidation; clear labeling.

Reconciles To: Consignment statements; AR/AP where ownership transfers.

GL Links: Off-balance for consignment-in until ownership flips; on transfer, normal postings.

SQL Skeleton:

```
SELECT consignment_type, partner_id, item_id, batch_no,
       SUM(qty_units) qty, SUM(value) value
FROM consignment_ledger
WHERE txn_date BETWEEN :from AND :to AND dept_id IN (:dept_scope)
GROUP BY consignment_type, partner_id, item_id, batch_no;
```

11) Returns Report (Purchase & Sales)

Purpose: Visibility on returns to supplier and from customer.

Scope: Supplier/customer, item/batch, reason codes.

Filters: Date, reason, quality status.

Fields: qty (U/W/V), value, tax impacts, replacement/credit.

Consolidation: External movement; standard rollups.

Reconciles To: AP/AR credit notes; **GL** reversals.

SQL Skeleton:

```
SELECT direction, partner_id, item_id, batch_no,
       SUM(qty_units) qty, SUM(net_amount) net
FROM returns_v
WHERE doc_date BETWEEN :from AND :to
      AND dept_id IN (:dept_scope)
GROUP BY direction, partner_id, item_id, batch_no;
```

12) Expiry & FEFO Compliance Report

Purpose: Proactive expiry management and FEFO rule adherence.

Scope: Perishable items by batch/expiry date.

Filters: Expiry window (e.g., next 30/60/90 days), item groups.

Fields: qty by age bucket, expiry date, FEFO pick status, blocked/hold flags.

Consolidation: Rollups; no transfer eliminations.

Reconciles To: Quality & hold queues; sales/issue picks.

SQL Skeleton:

```
SELECT item_id, batch_no, expiry_date,
       SUM(qty_units) qty,
       CASE WHEN expiry_date <= :as_of THEN 'EXPIRED'
            WHEN expiry_date <= :as_of + INTERVAL '30 day' THEN 'D30'
            WHEN expiry_date <= :as_of + INTERVAL '60 day' THEN 'D60'
            ELSE 'D90+' END bucket
FROM inventory_batches_v
WHERE dept_id IN (:dept_scope)
GROUP BY item_id, batch_no, expiry_date;
```

13) Serial/Lot Traceability Report

Purpose: Full provenance from purchase to sale (and reverse).

Scope: Single serial/lot or range.

Filters: serial/lot/batch, item.

Fields: all touchpoints (PO→GRN→WIP→FG→SO/Invoice), dates, locations.

Consolidation: N/A—trace path end-to-end.

Reconciles To: Compliance and recall procedures.

SQL Skeleton:

```
-- Forward & backward trace
SELECT * FROM trace_chain_v
WHERE item_id=:item_id AND (serial_no=:serial OR batch_no=:batch);
```

14) Quality Holds & Release Report

Purpose: Items under QA quarantine and their disposition.

Scope: Hold reason, batch, duration, release outcomes.

Filters: Date, reason, QC status.

Fields: qty on hold, passed, failed, rework.

Consolidation: Local to node; roll up counts.

Reconciles To: QA logs; post-release movements.

SQL Skeleton:

```
SELECT reason_code, item_id, batch_no,
       SUM(qty_units) hold_qty, SUM(released_qty) released_qty
FROM quality_hold_v
WHERE hold_date BETWEEN :from AND :to AND dept_id IN (:dept_scope)
GROUP BY reason_code, item_id, batch_no;
```

15) Activity-Based Overhead Allocation Report (ABC)

Purpose: Transparent overhead distribution to products/departments.

Scope: Overhead pools, cost drivers, rates, allocations.

Filters: Period, pool, driver.

Fields: driver_qty, rate, allocated_value by product/WO/dept.

Calcs: $\text{allocated} = \text{driver_qty} \times \text{rate}$

Consolidation: Rollups; avoid double-allocating across same tier.

Reconciles To: P&L overhead totals.

GL Links: Dr Inventory/WIP/COGS (per policy), Cr Overhead pool.

SQL Skeleton:

```
SELECT pool_id, driver_id, target_key,
       SUM(driver_qty) driver_qty,
       rate, SUM(driver_qty*rate) allocated_value
FROM abc_allocations_v
WHERE period=:period AND dept_id IN (:dept_scope)
GROUP BY pool_id, driver_id, target_key, rate;
```

16) Scenario Costing Report

Purpose: Compare cost methods (e.g., FIFO vs WAvg vs Standard) and transfer pricing rules.

Scope: Item/department/period.

Filters: Costing scenarios, date range.

Fields: qty on hand, unit costs per scenario, value per scenario, deltas.

Consolidation: Use same consolidation rule under each scenario.

Reconciles To: Selected official method; P&L sensitivity.

SQL Skeleton:

```
SELECT item_id, scenario_code,
       SUM(qty_units) qty, SUM(valued_amount) value,
       AVG(unit_cost) unit_cost
FROM scenario_valuation_v
WHERE as_of=:as_of AND dept_id IN (:dept_scope)
GROUP BY item_id, scenario_code;
```

17) COGS & Margin Report

Purpose: Profitability by product/customer/channel.

Scope: Sales joined to inventory cost (method).

Filters: Date, product, customer, channel.

Fields: sales qty/value, COGS, gross margin %, returns impact.

Calcs: $\text{GM\%} = (\text{Sales} - \text{COGS}) / \text{Sales} * 100$

Consolidation: Standard; no transfer eliminations.

Reconciles To: P&L revenue & COGS; Inventory changes.

GL Links: Revenue/COGS postings.

SQL Skeleton:

```

SELECT item_id, customer_id,
       SUM(sales_amount) sales, SUM(cogs_amount) cogs,
       (SUM(sales_amount)-SUM(cogs_amount)) margin,
       ((SUM(sales_amount)-SUM(cogs_amount))/NULLIF(SUM(sales_amount),0))*100
margin_pct
FROM sales_cogs_v
WHERE doc_date BETWEEN :from AND :to AND dept_id IN (:dept_scope)
GROUP BY item_id, customer_id;

```

18) Financial Statements (P&L, Balance Sheet, Cash Flow)

Purpose: Statutory & management financials aligned with inventory/ops.

Scope: Dept/Sector/Enterprise; multi-currency.

Filters: Periods, consolidation scope, eliminations toggle.

Fields: Standard FS lines; notes cross-link.

Consolidation:

- Eliminate internal sales/transfers (horizontal excluded at same tier; vertical included but eliminated at receiving when needed).

Reconciles To: Trial Balance totals, inventory valuations, AR/AP.

GL Links: Derived directly from GL (with inventory subledger tie-outs).

SQL Skeleton (pivoted TB):

```

SELECT fs_line, SUM(amount) amount
FROM gl_to_fs_map_v
WHERE period BETWEEN :from AND :to
   AND entity_scope IN (:scope)
GROUP BY fs_line;

```

19) Asset Register & Movements Report

Purpose: Fixed asset additions, disposals, depreciation, revaluations.

Scope: Asset class, location, department.

Filters: Period, class, status.

Fields: cost, accum depn, net book value, movements.

Consolidation: Standard rollups.

Reconciles To: GL FA control; depreciation expense.

GL Links: Dr Depn Exp, Cr Accum Depn; disposals.

SQL Skeleton:

```

SELECT asset_id, class_id, location_id,
       opening_cost, additions, disposals, closing_cost,
       opening_accum, depn_charge, closing_accum,
       (closing_cost - closing_accum) nbv
FROM asset_movements_v
WHERE period=:period AND dept_id IN (:dept_scope);

```

20) Cash Flow (Direct/Indirect) Report

Purpose: Liquidity view from GL and subledgers.

Scope: Entity scope; method selectable.

Filters: Period, method.

Fields: operating, investing, financing cash flows.

Consolidation: Inter-entity cash flows eliminated.

Reconciles To: Bank reconciliations.

SQL Skeleton (indirect outline):

```
SELECT section, SUM(amount) amount
FROM cashflow_indirect_v
WHERE period BETWEEN :from AND :to AND scope IN (:scope)
GROUP BY section;
```

21) Financial Ratios Dashboard

Purpose: KPI lens across liquidity, solvency, profitability, efficiency.

Scope: Entity levels; sector benchmarking.

Filters: Period, sector.

Fields/Calcs (examples):

- Current Ratio, Quick Ratio, Debt/Equity, Gross & Net Margin, ROA/ROE, Inventory Turns, DSO/DPO/DIH, Cash Conversion Cycle.

Consolidation: Derived from consolidated FS.

Reconciles To: FS lines.

SQL Skeleton:

```
-- Example: Inventory Turns
SELECT (COALESCE(cogs,0)/NULLIF(avg_inventory,0)) AS inv_turns
FROM kpi_basis_v
WHERE period BETWEEN :from AND :to AND scope IN (:scope);
```

22) Procurement Performance & Pricing Variance Report

Purpose: Supplier OTD, price variance vs standard/last.

Scope: Supplier/item/site.

Filters: Period, supplier class.

Fields: lead time, OTD%, price var, quality rejects.

Consolidation: Rollups.

Reconciles To: Purchase & Quality reports.

SQL Skeleton:

```
SELECT supplier_id, item_id,
       AVG(lead_time_days) lead_time,
```

```

        AVG(otd_flag::int)*100 otd_pct,
        AVG(price - standard_price) price_var
FROM procurement_kpi_v
WHERE period BETWEEN :from AND :to AND dept_id IN (:dept_scope)
GROUP BY supplier_id, item_id;

```

23) Operations Throughput & Bottleneck Report

Purpose: Capacity, run rates, downtime, OEE-style insights.

Scope: Line/cell, shift, product.

Filters: Date, line.

Fields: planned vs actual output, downtime minutes, throughput.

Consolidation: Rollups; transfers apply per earlier rule.

Reconciles To: Production outputs and WIP.

SQL Skeleton:

```

SELECT line_id, shift_id,
       SUM(planned_qty) planned, SUM(actual_qty) actual,
       SUM(downtime_min) downtime
FROM ops_throughput_v
WHERE date BETWEEN :from AND :to AND dept_id IN (:dept_scope)
GROUP BY line_id, shift_id;

```

24) Audit Pack (Auto-Generated)

Purpose: One-click audit binder.

Contents (minimum):

- Opening & Closing Stock, Purchases (with LC), Transfers (H/V, eliminations), Production I/O, Yield, Sales & Returns, Expiry/FEFO, Consignment, Adjustments (Abnormal), Stocktake Variances, Valuation Method memo, COGS tie-out, GL tie-outs, AR/AP ageing, Asset roll-forward, Cash/bank reconciliations, Policy notes, Change logs.

Consolidation: Include elimination schedules (horizontal excluded, vertical elimination notes).

Reconciles To: Trial balance & FS.

Output: PDF binder + CSV/Excel annexures; JSON for Smart Invoice as applicable.

SQL Skeleton: *Composed of all above views with a report orchestrator (stored proc / service).*

25) Master Data Exceptions Report

Purpose: Data hygiene impacting reports.

Scope: Missing UoM conversions, absent tax codes, inactive items with stock, negative stock,

orphaned batches, unposted GL.

Consolidation: N/A; exception list.

Reconciles To: Validation controls resolution.

SQL Skeleton:

```
SELECT issue_type, entity_key, details
FROM data_quality_exceptions_v
WHERE detected_date BETWEEN :from AND :to;
```

☞ Reconciliation Matrix (What Each Must Tie To)

- Opening ↔ Prior Closing; GL opening balances.
 - Purchases & LC ↔ AP subledger and GL Inventory + LC accounts.
 - Transfers ↔ Zero-sum globally; GL internal transfer accounts.
 - Production I/O ↔ WIP/FG movements; Yield Report.
 - Yield ↔ Production outputs/inputs; P&L variances.
 - Sales/Returns ↔ AR subledger; GL Revenue/Tax.
 - Closing ↔ GL Inventory control; movement roll-forward.
 - Abnormal & Stocktake ↔ GL stock adjustments; Audit pack.
 - Consignment ↔ Statements; ownership flip GL when triggered.
 - COGS & Margin ↔ Sales + Valuation; P&L.
 - FS (P&L/BS/CF) ↔ Trial Balance; eliminations schedule.
 - Assets ↔ FA control & depreciation postings.
 - Ratios ↔ FS lines.
 - Audit Pack ↔ All of the above.
-

✓ Validation Controls (applies across packs)

- No negative stock (configurable exceptions with approval).
 - Transfers must balance globally (qty & value).
 - Mandatory batch/serial for controlled items; expiry for perishables.
 - FEFO enforcement on issue/sale (or documented override).
 - All transactions GL-posted (or flagged).
 - Tax & Smart Invoice payloads validated where applicable.
 - Exchange rates locked per doc date for FX items.
-

Cross-Industry Extensibility

- **Agriculture:** feed/medicine issues, live-bird vertical transfers, carcass/by-products, FEFO cold chain.
 - **Manufacturing:** RM→WIP→FG, scrap/rework, line throughput.
 - **Retail/Wholesale:** multi-warehouse, promotions, returns, serials.
 - **Services/Public:** consumables, allocation to jobs/programs, ABC.
 - **Mining/Processing:** grade/assay lots, moisture (weight vs dry).
-

Minimal Views You'll Eventually Need (for later SQL work)

- `stock_ledger` (golden source), `inventory_valuation_v`, `purchase_receipts_v`, `ar_invoices_v`, `returns_v`, `prod_io_summary_v`, `trace_chain_v`, `quality_hold_v`, `abc_allocations_v`, `scenario_valuation_v`, `sales_cogs_v`, `gl_to_fs_map_v`, `asset_movements_v`, `cashflow_indirect_v`, `procurement_kpi_v`, `ops_throughput_v`, `stocktake_variances_v`, `consignment_ledger`, `data_quality_exceptions_v`, `stock_balances`.
-

UI/UX (How Users Run These)

- **Report Center** with saved views & schedules.
 - Filters always include: level scope, date/period, item/batch, category, partner, currency, method (valuation/scenario).
 - Drill-down from enterprise → sector → department → transaction (with document preview and GL posting).
 - Export: PDF, Excel, CSV, JSON; bulk “Audit Pack” export.
 - “Eliminations toggle” to preview before/after consolidation rules.
 - “Scenario toggle” (FIFO/WAvg/Standard; transfer pricing policy).
-

Developer Tasking (for later—just listing)

- Implement source views above, tie validations, build orchestrated report pack generator, add elimination engine (horizontal exclusion at same tier; vertical inclusion + elimination where needed), GL tie-out checks, export services, scheduler, and access controls.
-



Final Notes

- The above breakdown is intentionally **report-first** to lock scope and reconciliation.
- Once you approve, I'll produce the **SQL starter** (schemas + materialized views + example queries) and then the **FastAPI starter**.
- Throughout, we'll keep your consolidation rule **front-and-center: same-tier consolidation includes vertical but excludes horizontal transfers**.

Here's the single-page matrix of every **Report Pack**. It keeps your same-tier rule explicit (✓ include vertical, ✗ exclude horizontal) and shows what each report reconciles to and hits in GL.

Abbreviations: U/W/V = Units / Weight / Volume; Inv = Inventory; TB = Trial Balance; AP/AR = Payables/Receivables; WO = Work Order; ABC = Activity-Based Costing; FEFO = First-Expiry-First-Out.

#	Report Pack	Purpose	Key Filters	Core Fields (U/W/V/ Value)	Same-Tier Consolidation Rule	Reconciles To	GL Links	Output / Frequency
1	Opening Stock	Establish starting balances	As-of date, scope, item/batch	Opening qty & value	N/A (carried from prior close)	Prior Closing Stock; GL Inv opening	None (carried fwd)	As-of, PDF/Excel
2	Purchases & Landed Cost	Purchases + duties/freight allocation	Date, supplier, GRN/PO, currency, LC types	Received qty, base cost, LC, landed unit cost	N/A	AP GRNs; GL Inv↑ / AP↑; LC clearing	Dr Inv, Cr AP; Dr Inv, Cr Accruals	Periodic, by PO/GRN
3	Interdept Transfer s	Track internal moves	Date, source/dest dept, H/V	Qty, transfer price, value	✓ Vertical included; ✗ Horizontal excluded	Net zero across org; Transfer schedules	Dr Inv (recv) / Cr Inv (send) or clearing	Daily/periodic
4	Production I/O	Inputs consumed & outputs received	Date, WO, line, item	Inputs & outputs (U/W/V/ Value), scrap	Vertical to next tier ✓; horizontal ✗ at same-tier	Yield Report; WIP/FG movement	Dr WIP/FG, Cr RM/WIP	By WO/period
5	Yield & Efficiency	Conversion performance	Date, product, WO	Inputs vs outputs, standard vs actual, variance	Node calc; roll-up weighted	Production & P&L variances	Variance postings (policy)	Periodic
6	Sales & Invoices (+Returns)	Revenue & quantities	Date, customer, channel, currency	Qty, gross, discounts, tax, net	External; no eliminations	AR; GL Revenue/Tax	Dr AR, Cr Revenue; returns reversals	Daily/periodic

#	Report Pack	Purpose	Key Filters	Core Fields (U/W/V/Value)	Same-Tier Consolidation Rule	Reconciles To	GL Links	Output / Frequency
7	Closing Stock (Valuation)	Period end inventory & valuation	As-of, method (FIFO/WA vg/etc.)	On-hand qty, unit cost, value, ageing	✓ Vertical; ✗ Horizontal	GL Inv control; roll-forward	Inv control balance	As-of
8	Abnormal Loss/Gain	Shrinkage/unexpected recoveries	Date, reason, approver	Qty, value, reason	Local; roll-up	Stock adj GL; Audit pack	Dr/Cr Stock Adj ↔ P&L	Periodic
9	Stocktake Variances	Physical vs book	Count event, area	Book vs physical; variance	Local; roll-up	Adjustment postings; Inv control	Dr/Cr Inv ↔ Variance	Per event
10	Consignment In/Out	Third-party stock tracking	Date, partner, status	Qty, value (memo for cons-in)	Separate from owned; clear tagging	Statements; ownership flips	Off-balance until flip; then normal	Periodic
11	Returns (AP/AR)	Supplier & customer returns	Date, partner, reason	Qty, value, tax	External; standard	AP/AR CNs; GL reversals	Reverse original flows	Periodic
12	Expiry & FEFO	Proactive expiry control	As-of, window (D30/D60/D90), item	Qty by expiry bucket, holds	N/A eliminations	QA queues; pick lists	Blocks per policy (no direct GL)	Rolling/as-of
13	Serial/Lot Traceability	End-to-end provenance	Serial/lot, item	All touchpoints chain	Not a consolidation report	Compliance/recall trail	Reference only	On demand
14	Quality Holds & Releases	QA quarantine & disposition	Date, reason, status	Hold qty, released/failed qty	N/A eliminations	QA logs; post-release moves	Posting happens on disposition	Periodic
15	ABC Allocation	Transparent overhead spreads	Period, pool, driver	Driver qty, rate, allocated value	Avoid double-alloc at same tier	P&L overhead totals	Dr Inv/WIP/COGS, Cr Overhead Pool	Period close

#	Report Pack	Purpose	Key Filters	Core Fields (U/W/V/Value)	Same-Tier Consolidation Rule	Reconciles To	GL Links	Output / Frequency
16	Scenario Costing	Compare methods/pricing	As-of, scenario(s)	Qty, unit cost/value per scenario	Apply rule per scenario	Official method & sensitivity	Only chosen method posts	As-of
17	COGS & Margin	Product/customer/channel margin	Date, product, customer	Sales, COGS, margin, %	External; standard	P&L Revenue & COGS	Revenue/COGS postings	Periodic
18	P&L	Profit & loss	Period, scope, FX	Revenue, COGS, Opex, other	Eliminate internals (✗ horiz; ✓ vert but eliminate where required)	TB totals	From GL	Period close
19	Balance Sheet	Financial position	As-of, scope	Assets, liabilities, equity	As above	TB & subledgers (AR/AP/Inv/FA)	From GL	As-of
20	Cash Flow (Dir/Indir)	Liquidity view	Period, method	Operating, investing, financing	Inter-entity cash eliminated	Bank recs	From GL & subledgers	Period close
21	Financial Ratios	KPIs (liquidity, profitability, efficiency)	Period, scope	Derived metrics	Based on consolidated FS	FS lines	No direct postings	Periodic
22	Procurement Perf/Price Var	Supplier OTD, price variances	Period, supplier	Lead time, OTD%, price var, rejects	Standard	Purchases & QA	Variances per policy	Periodic
23	Ops Throughput & Bottlenecks	Capacity & OEE-style	Date, line/shift	Planned vs actual, downtime	Standard; transfers per rule	Production outputs/WIP	None directly	Daily/periodic

#	Report Pack	Purpose	Key Filters	Core Fields (U/W/V/Value)	Same-Tier Consolidation Rule	Reconciles To	GL Links	Output / Frequency
2 4	Audit Pack (Auto)	One-click audit binder	Period, scope	All packs + eliminations schedules	Shows ✓/✗ applications	TB & FS tie-outs	Mixed per report	Period close
2 5	Master Data Exceptions	Data hygiene blockers	Period	Missing UoM/tax, negative stock, unposted GL	N/A	Validation queue	None; drives fixes	Continuous

Where each report must reconcile (at a glance)

- **Opening ↔ Prior Closing; Closing ↔ GL Inv control** and movement roll-forward
 - **Purchases/Landed ↔ AP GRN and GL Inv/AP/LC**
 - **Transfers ↔ Zero-sum** org-wide; **Internal transfer GL**
 - **Production I/O & Yield ↔ WIP/FG and P&L variances**
 - **Sales/Returns ↔ AR and GL Revenue/Tax**
 - **Abnormal/Stocktake ↔ GL Stock Adjustments**
 - **Consignment ↔ Partner statements; GL only on ownership change**
 - **COGS & Margin ↔ Sales + Valuation; P&L**
 - **FS (P&L/BS/CF) ↔ TB (+ eliminations schedule)**
 - **Assets** (in Asset Register pack) ↔ **FA control & depn** (if you want, we can add a dedicated “Asset Register” report row back into the matrix alongside #19 Balance Sheet)
-

Here’s a **PostgreSQL SQL starter** that sets you up with:

- Minimal but extensible **schema**
- **Movement taxonomy** & automatic **horizontal/vertical** classification
- **Org model with variable tiers**
- **Batch/lot & FEFO** foundations
- **Core GL posting hooks**
- **Views/materialized views** that implement your **consolidation rule** (same-tier: include vertical, exclude horizontal)
- **Starter queries** for each report pack (compact, ready to adapt)

Dialect: **PostgreSQL 14+**.

Naming: snake_case, singular table names.

Conventions: timestamps in UTC; monetary values in minor units (or DECIMAL with currency_code).

Security (row-level) and FastAPI endpoints can layer on top later.

0) Bootstrap & Enums

```
-- Enable helpful extensions
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Simple enums (extend as needed)
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'movement_type') THEN
        CREATE TYPE movement_type AS ENUM (
            'OPENING', 'PURCHASE', 'PURCHASE_RETURN',
            'TRANSFER', 'ISSUE_TO_WO', 'RECEIPT_FROM_WO',
            'SALE', 'SALES_RETURN', 'ADJUSTMENT',
            'CONSIGNMENT_IN', 'CONSIGNMENT_OUT', 'COUNT_VARIANCE'
        );
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'transfer_class') THEN
        CREATE TYPE transfer_class AS ENUM ('HORIZONTAL', 'VERTICAL', 'NA');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typname = 'quality_status') THEN
        CREATE TYPE quality_status AS ENUM ('FREE', 'HOLD', 'BLOCKED', 'EXPIRED');
    END IF;
END $$;
```

1) Organization Model (variable tiers)

```
-- Enterprises / companies (multi-tenant optional)
CREATE TABLE enterprise (
    enterprise_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name TEXT NOT NULL,
    base_currency TEXT NOT NULL DEFAULT 'ZMW',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Organizational units: sectors, departments, sub-departments, sites, etc.
CREATE TABLE org_unit (
    org_unit_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),
    name TEXT NOT NULL,
```

```

    code TEXT UNIQUE,
    tier_level INT NOT NULL CHECK (tier_level >= 1),
    parent_id UUID REFERENCES org_unit(org_unit_id),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ON org_unit (enterprise_id, tier_level);
CREATE INDEX ON org_unit (parent_id);

```

2) Master Data: Items, UoM, Batches

```

CREATE TABLE uom (
    uom_code TEXT PRIMARY KEY,                -- e.g., 'EA', 'KG', 'L'
    description TEXT
);

CREATE TABLE item (
    item_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),
    name TEXT NOT NULL,
    sku TEXT UNIQUE,
    base_uom TEXT NOT NULL REFERENCES uom(uom_code),
    alt_uom TEXT REFERENCES uom(uom_code),
    alt_to_base_factor NUMERIC(20,6),          -- alt qty * factor = base qty
    category TEXT,
    expiry_days INT,                          -- for FEFO
    serialized BOOLEAN NOT NULL DEFAULT FALSE, -- true if each unit has serial
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Optional item cost policy per org
CREATE TABLE item_cost_policy (
    item_id UUID PRIMARY KEY REFERENCES item(item_id),
    valuation_method TEXT NOT NULL DEFAULT 'WAVG', --
    'WAVG', 'FIFO', 'LIFO', 'STD'
    standard_cost NUMERIC(20,6)                -- used if STD
);

-- Batch/Lot/Serial registry
CREATE TABLE batch (
    batch_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),
    item_id UUID NOT NULL REFERENCES item(item_id),
    batch_no TEXT,                            -- supplier lot, internal lot, or serial
    serial_no TEXT,                           -- if serialized, store here
    mfg_date DATE,
    expiry_date DATE,
    UNIQUE (item_id, batch_no, serial_no)
);

CREATE INDEX ON batch (item_id, expiry_date);

```

3) Partners & Docs (minimal)

```
CREATE TABLE partner (  
  partner_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),  
  name TEXT NOT NULL,  
  type TEXT NOT NULL,          --  
  'SUPPLIER', 'CUSTOMER', '3PL', 'CONSIGNOR', 'CONSIGNEE'  
  tax_id TEXT,  
  currency TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
-- Simple document headers (expand later)  
CREATE TABLE doc_header (  
  doc_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),  
  doc_no TEXT NOT NULL,  
  doc_type TEXT NOT NULL,      --  
  'PO', 'GRN', 'SO', 'INV', 'CN', 'STOCK_COUNT', 'INT_TRF', 'WO'  
  doc_date DATE NOT NULL,  
  partner_id UUID REFERENCES partner(partner_id),  
  currency TEXT,  
  created_by TEXT,  
  UNIQUE (enterprise_id, doc_no, doc_type)  
);
```

4) Stock Ledger (single source of truth)

```
-- All inventory-affecting movements live here  
CREATE TABLE stock_ledger (  
  ledger_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),  
  txn_ts TIMESTAMPTZ NOT NULL DEFAULT now(),  
  txn_date DATE NOT NULL DEFAULT CURRENT_DATE,  
  org_unit_id UUID NOT NULL REFERENCES org_unit(org_unit_id),  
  item_id UUID NOT NULL REFERENCES item(item_id),  
  batch_id UUID REFERENCES batch(batch_id),  
  movement movement_type NOT NULL,  
  qty_units NUMERIC(20,6) NOT NULL DEFAULT 0,          -- in base_uom units  
  qty_weight NUMERIC(20,6) NOT NULL DEFAULT 0,         -- optional dimension  
  qty_volume NUMERIC(20,6) NOT NULL DEFAULT 0,         -- optional dimension  
  value NUMERIC(20,6) NOT NULL DEFAULT 0,             -- monetary value impact  
  (+/-)  
  -- transfers:  
  source_org_unit_id UUID REFERENCES org_unit(org_unit_id), -- for transfers,  
  issues, receipts  
  dest_org_unit_id UUID REFERENCES org_unit(org_unit_id),  
  transfer_cls transfer_class NOT NULL DEFAULT 'NA',  
  -- quality/expiry:  
  quality quality_status NOT NULL DEFAULT 'FREE',  
  -- references:
```

```

    doc_id UUID REFERENCES doc_header(doc_id),
    reference_no TEXT,
    gl_posted BOOLEAN NOT NULL DEFAULT FALSE,
    meta JSONB NOT NULL DEFAULT '{}'::jsonb
);

CREATE INDEX ON stock_ledger (enterprise_id, txn_date);
CREATE INDEX ON stock_ledger (org_unit_id, txn_date);
CREATE INDEX ON stock_ledger (item_id, txn_date);
CREATE INDEX ON stock_ledger (movement);
CREATE INDEX ON stock_ledger (source_org_unit_id, dest_org_unit_id);

```

Auto-classify transfer HORIZONTAL vs VERTICAL

```

-- Helper view to compute tier of source/dest and set transfer class
CREATE OR REPLACE VIEW v_stock_ledger_classified AS
SELECT
    sl.*,
    src.tier_level AS src_tier,
    dst.tier_level AS dst_tier,
    CASE
        WHEN sl.movement <> 'TRANSFER' OR sl.source_org_unit_id IS NULL OR
sl.dest_org_unit_id IS NULL
        THEN 'NA'::transfer_class
        WHEN src.tier_level = dst.tier_level
        THEN 'HORIZONTAL'::transfer_class
        ELSE 'VERTICAL'::transfer_class
    END AS calc_transfer_cls
FROM stock_ledger sl
LEFT JOIN org_unit src ON src.org_unit_id = sl.source_org_unit_id
LEFT JOIN org_unit dst ON dst.org_unit_id = sl.dest_org_unit_id;

-- Keep transfer_cls synchronized (optional trigger)
CREATE OR REPLACE FUNCTION trg_set_transfer_class() RETURNS trigger AS $$
BEGIN
    IF NEW.movement = 'TRANSFER' AND NEW.source_org_unit_id IS NOT NULL AND
NEW.dest_org_unit_id IS NOT NULL THEN
        PERFORM 1;
        -- look up tiers
        DECLARE src_t INT; DECLARE dst_t INT;
        SELECT tier_level INTO src_t FROM org_unit WHERE org_unit_id =
NEW.source_org_unit_id;
        SELECT tier_level INTO dst_t FROM org_unit WHERE org_unit_id =
NEW.dest_org_unit_id;
        IF src_t IS NOT NULL AND dst_t IS NOT NULL THEN
            NEW.transfer_cls := CASE WHEN src_t = dst_t THEN 'HORIZONTAL' ELSE
'VERTICAL' END;
        ELSE
            NEW.transfer_cls := 'NA';
        END IF;
    ELSE
        NEW.transfer_cls := COALESCE(NEW.transfer_cls, 'NA');
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;

```

```
DROP TRIGGER IF EXISTS set_transfer_class ON stock_ledger;
CREATE TRIGGER set_transfer_class
BEFORE INSERT OR UPDATE ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION trg_set_transfer_class();
```

5) GL Posting Hook (minimal)

```
CREATE TABLE gl_posting (
  gl_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),
  ledger_id UUID NOT NULL REFERENCES stock_ledger(ledger_id),
  posting_ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  account_dr TEXT NOT NULL,
  account_cr TEXT NOT NULL,
  amount NUMERIC(20,6) NOT NULL,
  currency TEXT NOT NULL DEFAULT 'ZMW'
);

CREATE INDEX ON gl_posting (enterprise_id);
CREATE INDEX ON gl_posting (ledger_id);
```

Your service layer (FastAPI) will map movement → GL policy (e.g., Purchase: Dr Inventory / Cr AP; Transfer: Dr Receiving Inv / Cr Sending Inv; Sale: COGS/Revenue, etc.) and insert rows here. Keep **DR=CR** invariant at all times.

6) Balances & Valuation (starter)

Rolling balances (by org, item, batch)

```
-- Daily snapshot (can be materialized or computed on the fly)
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_daily_item_balances AS
SELECT
  enterprise_id,
  org_unit_id,
  item_id,
  batch_id,
  txn_date AS as_of,
  SUM(qty_units) AS units_delta,
  SUM(qty_weight) AS weight_delta,
  SUM(qty_volume) AS volume_delta,
  SUM(value) AS value_delta
FROM stock_ledger
GROUP BY enterprise_id, org_unit_id, item_id, batch_id, txn_date
WITH NO DATA;

-- Index for fast range scans
CREATE INDEX IF NOT EXISTS ON mv_daily_item_balances (enterprise_id,
org_unit_id, item_id, batch_id, as_of);
```

```
-- A convenient balance-as-of view (uses window sum over the MV)
CREATE OR REPLACE VIEW v_item_balances_asof AS
SELECT
    enterprise_id, org_unit_id, item_id, batch_id, as_of,
    SUM(units_delta) OVER w AS units_balance,
    SUM(weight_delta) OVER w AS weight_balance,
    SUM(volume_delta) OVER w AS volume_balance,
    SUM(value_delta) OVER w AS value_balance
FROM mv_daily_item_balances
WINDOW w AS (PARTITION BY enterprise_id, org_unit_id, item_id, batch_id ORDER
BY as_of ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW);
```

Weighted-average valuation (illustrative)

```
-- Per org+item, compute moving WAVG unit cost per day
CREATE OR REPLACE VIEW v_item_cost_wavg AS
WITH daily AS (
    SELECT enterprise_id, org_unit_id, item_id, as_of,
           SUM(CASE WHEN value_delta > 0 THEN value_delta ELSE 0 END) AS
inflow_value,
           SUM(CASE WHEN value_delta > 0 THEN units_delta ELSE 0 END) AS
inflow_units
    FROM mv_daily_item_balances
    GROUP BY 1,2,3,4
)
SELECT d.enterprise_id, d.org_unit_id, d.item_id, d.as_of,
       CASE WHEN SUM(inflow_units) OVER w = 0 THEN NULL
            ELSE (SUM(inflow_value) OVER w) / NULLIF(SUM(inflow_units) OVER
w,0) END AS wavg_unit_cost
FROM daily d
WINDOW w AS (PARTITION BY d.enterprise_id, d.org_unit_id, d.item_id ORDER BY
d.as_of ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW);
```

(For FIFO/LIFO, maintain layer tables; this starter gives you WAVG out-of-the-box.)

7) Consolidation helpers (apply your rule centrally)

```
-- Parameterized "scope" helper via a temp table (populate per request)
-- CREATE TEMP TABLE tmp_scope (enterprise_id UUID, org_unit_id UUID);

-- Same-tier consolidation: include vertical, exclude horizontal
CREATE OR REPLACE VIEW v_consolidated_movements AS
SELECT
    sl.enterprise_id,
    sl.item_id,
    sl.batch_id,
    sl.movement,
```

```

-- choose a reporting org key (e.g., sector or enterprise) via outer
grouping
CASE
  WHEN sl.movement = 'TRANSFER' THEN
    CASE
      WHEN sl.transfer_cls = 'HORIZONTAL' THEN 0          -- EXCLUDE
      WHEN sl.transfer_cls = 'VERTICAL' THEN sl.value      -- INCLUDE
      ELSE sl.value
    END
  ELSE sl.value
END AS value_cons,
CASE
  WHEN sl.movement = 'TRANSFER' THEN
    CASE
      WHEN sl.transfer_cls = 'HORIZONTAL' THEN 0
      WHEN sl.transfer_cls = 'VERTICAL' THEN sl.qty_units
      ELSE sl.qty_units
    END
  ELSE sl.qty_units
END AS units_cons,
sl.txn_date,
sl.org_unit_id,
sl.source_org_unit_id,
sl.dest_org_unit_id,
sl.transfer_cls
FROM v_stock_ledger_classified sl;

```

When **rolling up to the same tier** (e.g., consolidating departments within a sector), **aggregate from v_consolidated_movements** to guarantee “include vertical, exclude horizontal.”

For **enterprise-wide**, also use v_consolidated_movements then group by enterprise or by parent sector tree as needed.

8) Starters for each Report Pack

Each query assumes you'll bind `:enterprise_id`, `:from`, `:to`, and either *one org level* or a list via temp table `tmp_scope(org_unit_id)`. Replace with your favorite scoping pattern.

8.1 Opening Stock

```

-- Opening = balance as of :from - 1 (units & value)
WITH last_day AS (
  SELECT (:from::date - INTERVAL '1 day')::date AS as_of
)
SELECT b.enterprise_id, b.org_unit_id, b.item_id, b.batch_id,
       b.units_balance AS opening_units,
       b.value_balance AS opening_value
FROM v_item_balances_asof b
JOIN last_day d ON b.as_of = d.as_of
WHERE b.enterprise_id = :enterprise_id
      AND b.org_unit_id IN (SELECT org_unit_id FROM tmp_scope);

```

8.2 Purchases & Landed Cost

```
SELECT sl.org_unit_id, sl.item_id, sl.batch_id,
       SUM(sl.qty_units) AS qty_units,
       SUM(CASE WHEN sl.movement='PURCHASE' THEN sl.value ELSE 0 END)
AS base_value,
       SUM((sl.meta->>'duty')::numeric
          + (sl.meta->>'freight')::numeric
          + (sl.meta->>'insurance')::numeric
          + (sl.meta->>'other_lc')::numeric)
AS landed_components,
       SUM(sl.value
          + COALESCE((sl.meta->>'duty')::numeric,0)
          + COALESCE((sl.meta->>'freight')::numeric,0)
          + COALESCE((sl.meta->>'insurance')::numeric,0)
          + COALESCE((sl.meta->>'other_lc')::numeric,0))
AS total_landed_value
FROM stock_ledger sl
WHERE sl.enterprise_id = :enterprise_id
      AND sl.org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND sl.movement IN ('PURCHASE')
      AND sl.txn_date BETWEEN :from AND :to
GROUP BY sl.org_unit_id, sl.item_id, sl.batch_id;
```

8.3 Interdepartment Transfers (rule applied)

```
SELECT
  source_org_unit_id, dest_org_unit_id, item_id, batch_id, transfer_cls,
  SUM(CASE WHEN transfer_cls='HORIZONTAL' THEN 0 ELSE qty_units END) AS
qty_cons,
  SUM(CASE WHEN transfer_cls='HORIZONTAL' THEN 0 ELSE value END) AS
val_cons
FROM v_stock_ledger_classified
WHERE enterprise_id = :enterprise_id
      AND movement = 'TRANSFER'
      AND txn_date BETWEEN :from AND :to
      AND (source_org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
          OR dest_org_unit_id IN (SELECT org_unit_id FROM tmp_scope))
GROUP BY source_org_unit_id, dest_org_unit_id, item_id, batch_id,
transfer_cls;
```

8.4 Production Inputs & Outputs

```
-- Inputs to WO
WITH inputs AS (
  SELECT item_id, batch_id, SUM(qty_units) in_units, SUM(value) in_val
  FROM stock_ledger
  WHERE enterprise_id=:enterprise_id
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND movement='ISSUE_TO_WO'
        AND txn_date BETWEEN :from AND :to
  GROUP BY item_id, batch_id
),
outputs AS (
```

```

SELECT item_id, batch_id, SUM(qty_units) out_units, SUM(value) out_val
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement='RECEIPT_FROM_WO'
      AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id
)
SELECT COALESCE(i.item_id,o.item_id) item_id,
       COALESCE(i.batch_id,o.batch_id) batch_id,
       COALESCE(in_units,0) in_units, COALESCE(in_val,0) in_val,
       COALESCE(out_units,0) out_units, COALESCE(out_val,0) out_val
FROM inputs i
FULL JOIN outputs o USING (item_id, batch_id);

```

8.5 Yield

```

WITH io AS (
  SELECT wo_no,                                -- assume wo_no stored in meta->>'wo_no'
         SUM(CASE WHEN movement='ISSUE_TO_WO'    THEN qty_units ELSE 0 END)
in_units,
         SUM(CASE WHEN movement='RECEIPT_FROM_WO' THEN qty_units ELSE 0 END)
out_units
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND txn_date BETWEEN :from AND :to
GROUP BY wo_no
)
SELECT wo_no, in_units, out_units,
       CASE WHEN in_units=0 THEN NULL ELSE (out_units/in_units)*100 END AS
actual_yield_pct
FROM io;

```

8.6 Sales & Invoices (with returns)

```

SELECT item_id, batch_id,
       SUM(CASE WHEN movement='SALE'            THEN qty_units ELSE 0 END) AS
qty_sold,
       SUM(CASE WHEN movement='SALE'            THEN value ELSE 0 END)    AS
sales_value,
       SUM(CASE WHEN movement='SALES_RETURN'    THEN qty_units ELSE 0 END) AS
qty_returned,
       SUM(CASE WHEN movement='SALES_RETURN'    THEN value ELSE 0 END)    AS
returns_value,
       SUM(CASE WHEN movement IN ('SALE','SALES_RETURN') THEN value ELSE 0
END) AS net_revenue
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement IN ('SALE','SALES_RETURN')
      AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id;

```

8.7 Closing Stock (valuation)

```
-- Use balances as of :to and join to WAVG cost
WITH last_bal AS (
  SELECT *
  FROM v_item_balances_asof
  WHERE enterprise_id=:enterprise_id
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND as_of = :to::date
),
cost AS (
  SELECT enterprise_id, org_unit_id, item_id, MAX(as_of) AS as_of,
        MAX(wavg_unit_cost) FILTER (WHERE as_of <= :to::date) AS unit_cost
  FROM v_item_cost_wavg
  WHERE enterprise_id=:enterprise_id
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND as_of <= :to::date
  GROUP BY enterprise_id, org_unit_id, item_id
)
SELECT b.org_unit_id, b.item_id, b.batch_id,
      b.units_balance AS closing_units,
      c.unit_cost,
      (b.units_balance * COALESCE(c.unit_cost,0)) AS closing_value
FROM last_bal b
LEFT JOIN cost c USING (enterprise_id, org_unit_id, item_id);
```

8.8 Abnormal Loss/Gain

```
SELECT item_id, batch_id,
      SUM(qty_units) qty, SUM(value) value,
      meta->>'reason' AS reason
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement='ADJUSTMENT'
      AND (meta->>'abnormal')::boolean = true
      AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id, reason;
```

8.9 Stocktake Variances

```
SELECT meta->>'count_event' AS event_id, item_id, batch_id,
      SUM(CASE WHEN meta->>'subtype'='BOOK' THEN qty_units ELSE 0 END)
AS book_qty,
      SUM(CASE WHEN meta->>'subtype'='PHYSICAL' THEN qty_units ELSE 0 END)
AS physical_qty,
      SUM(qty_units) AS variance_qty,
      SUM(value) AS variance_value
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement='COUNT_VARIANCE'
      AND txn_date BETWEEN :from AND :to
GROUP BY event_id, item_id, batch_id;
```

8.10 Consignment In/Out

```
SELECT movement, item_id, batch_id,
       SUM(qty_units) qty, SUM(value) val,
       meta->>'partner_id' AS partner_id
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement IN ('CONSIGNMENT_IN','CONSIGNMENT_OUT')
      AND txn_date BETWEEN :from AND :to
GROUP BY movement, item_id, batch_id, partner_id;
```

8.11 Expiry & FEFO

```
SELECT b.item_id, b.batch_id, b.expiry_date,
       SUM(v.units_balance) AS qty_on_hand,
       CASE
         WHEN b.expiry_date < :as_of::date THEN 'EXPIRED'
         WHEN b.expiry_date <= :as_of::date + INTERVAL '30 day' THEN 'D30'
         WHEN b.expiry_date <= :as_of::date + INTERVAL '60 day' THEN 'D60'
         ELSE 'D90+'
       END AS bucket
FROM batch b
JOIN v_item_balances_asof v USING (enterprise_id, item_id, batch_id)
WHERE b.enterprise_id = :enterprise_id
      AND v.org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND v.as_of = :as_of::date
GROUP BY b.item_id, b.batch_id, b.expiry_date, bucket;
```

8.12 Serial/Lot Traceability (forward/backward)

```
-- One chain view: join on doc_id and reference fields as needed
SELECT *
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND (batch_id = :batch_id OR meta->>'serial_no' = :serial_no)
ORDER BY txn_ts;
```

8.13 Quality Holds

```
SELECT item_id, batch_id, quality,
       SUM(qty_units) qty
FROM stock_ledger
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id, quality;
```

8.14 ABC Allocation (reporting)

```
-- Assumes an ABC staging/allocation table populated by your cost engine
CREATE TABLE IF NOT EXISTS abc_allocation (
  alloc_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id),
period_start DATE NOT NULL,
period_end DATE NOT NULL,
pool_code TEXT NOT NULL,
driver_code TEXT NOT NULL,
target_key TEXT NOT NULL, -- item_id or wo_no or org_unit_id...
driver_qty NUMERIC(20,6) NOT NULL,
rate NUMERIC(20,6) NOT NULL,
allocated_value NUMERIC(20,6) NOT NULL
);

SELECT pool_code, driver_code, target_key,
       SUM(driver_qty) driver_qty,
       AVG(rate) rate,
       SUM(allocated_value) allocated_value
FROM abc_allocation
WHERE enterprise_id=:enterprise_id
      AND period_start>=:from AND period_end<=:to
GROUP BY pool_code, driver_code, target_key;

```

8.15 Scenario Costing (compare methods)

```

-- Provide scenario snapshots in a table/view populated by your engine
CREATE TABLE IF NOT EXISTS scenario_valuation (
  enterprise_id UUID NOT NULL,
  org_unit_id UUID NOT NULL,
  item_id UUID NOT NULL,
  as_of DATE NOT NULL,
  scenario_code TEXT NOT NULL, --
'WAVG', 'FIFO', 'STD', 'TP_COST', 'TP_MARKUP_10'
  qty_units NUMERIC(20,6) NOT NULL,
  unit_cost NUMERIC(20,6),
  valued_amount NUMERIC(20,6)
);

SELECT item_id, scenario_code,
       SUM(qty_units) qty, AVG(unit_cost) unit_cost, SUM(valued_amount) value
FROM scenario_valuation
WHERE enterprise_id=:enterprise_id
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND as_of=:as_of::date
GROUP BY item_id, scenario_code;

```

8.16 COGS & Margin (starter)

```

-- Sales joined to valuation (WAVG shown for simplicity)
WITH sales AS (
  SELECT item_id,
         SUM(CASE WHEN movement='SALE' THEN value ELSE 0 END) AS sales_amt,
         SUM(CASE WHEN movement='SALE' THEN qty_units ELSE 0 END) AS qty_sold
  FROM stock_ledger
  WHERE enterprise_id=:enterprise_id
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND txn_date BETWEEN :from AND :to
  GROUP BY item_id

```

```

),
cost AS (
  SELECT item_id,
         AVG(unit_cost) AS unit_cost
  FROM v_item_cost_wavg
  WHERE enterprise_id=:enterprise_id
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND as_of <= :to::date
  GROUP BY item_id
)
SELECT s.item_id, s.qty_sold, s.sales_amt,
       (s.qty_sold * COALESCE(c.unit_cost,0)) AS cogs,
       (s.sales_amt - (s.qty_sold * COALESCE(c.unit_cost,0))) AS margin,
       CASE WHEN s.sales_amt=0 THEN NULL
            ELSE ((s.sales_amt - (s.qty_sold *
COALESCE(c.unit_cost,0)))/s.sales_amt)*100 END AS margin_pct
FROM sales s
LEFT JOIN cost c ON c.item_id = s.item_id;

```

(Financial statements, ratios, procurement KPIs, assets & cash flow come from the GL side; you already have the inventory tie-outs here.)

9) Integrity & Validation (DB-side starters)

```

-- Prevent negative issue if you want hard guardrails (soft = warn in
service)
CREATE OR REPLACE FUNCTION check_negative_stock() RETURNS trigger AS $$
DECLARE bal NUMERIC;
BEGIN
  -- Only check on movements that reduce stock
  IF NEW.qty_units < 0 THEN
    -- Compute balance as-of now (simplified quick check)
    SELECT COALESCE(SUM(qty_units),0) INTO bal
    FROM stock_ledger
    WHERE enterprise_id = NEW.enterprise_id
          AND org_unit_id = NEW.org_unit_id
          AND item_id = NEW.item_id
          AND COALESCE(batch_id, '00000000-0000-0000-0000-000000000000') =
COALESCE(NEW.batch_id, '00000000-0000-0000-0000-000000000000');
    IF bal + NEW.qty_units < 0 THEN
      RAISE EXCEPTION 'Negative stock not allowed for item % at org % (bal:
%, delta: %)',
        NEW.item_id, NEW.org_unit_id, bal, NEW.qty_units;
    END IF;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_no_negative_stock ON stock_ledger;
CREATE TRIGGER trg_no_negative_stock
BEFORE INSERT ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION check_negative_stock();

```

10) Indices you'll likely want (performance)

```
-- Hot paths
CREATE INDEX IF NOT EXISTS stock_ledger_idx1 ON stock_ledger (enterprise_id,
txn_date, movement);
CREATE INDEX IF NOT EXISTS stock_ledger_idx2 ON stock_ledger (enterprise_id,
item_id, txn_date);
CREATE INDEX IF NOT EXISTS stock_ledger_idx3 ON stock_ledger (enterprise_id,
org_unit_id, txn_date);
CREATE INDEX IF NOT EXISTS stock_ledger_idx4 ON stock_ledger (transfer_cls)
WHERE movement='TRANSFER';
CREATE INDEX IF NOT EXISTS stock_ledger_idx5 ON stock_ledger USING gin
(meta);
```

11) How to run (quick checklist)

1. Run all DDL above on a clean **PostgreSQL** database.
 2. Populate **enterprise**, **org_unit** (with tier_level), **uom**, **item**, **batch**.
 3. Insert **stock_ledger** movements (purchases, transfers, production, sales...).
 4. `REFRESH MATERIALIZED VIEW mv_daily_item_balances;` (schedule daily/hourly).
 5. Query report starters in **Section 8** with a **tmp_scope** containing the org units you're consolidating.
 6. For **same-tier consolidation**, always aggregate via **v_consolidated_movements** (it already **excludes horizontal** and **includes vertical** as required).
 7. Add service-side GL map → insert **gl_posting** rows per movement policy.
-

Final Notes

- This starter is **minimal but production-oriented**: you can bolt on FIFO/LIFO layers, FX revaluation, Smart Invoice payload tables, audit trails, and row-level security as you proceed.
- The **consolidation rule** is encoded centrally so every report that uses the consolidated views **guarantees: same-tier = include vertical, exclude horizontal**.
- Next steps (when you're ready): I'll wire a **FastAPI** scaffold with endpoints for each report, job to refresh MVs, and GL posting policies.

Here's a **maximum, production-ready PostgreSQL starter** for your ERP reporting stack. It's opinionated for correctness, performance, security, and auditability while keeping your **consolidation rule** baked in (same-tier consolidation: **include vertical, exclude horizontal**). You can paste this into migrations in chunks.

Target: **PostgreSQL 15+** (tested patterns work on 14+).

Focus: multi-tenant isolation, variable tiers, batches/serials, FEFO, movement taxonomy, multi-currency + FX, landed cost, transfer pricing, costing (WAVG now, FIFO layer-ready), Smart-Invoice fields, validations, GL hooks, partitioning, RLS, audits, jobs, MVs, indexes.

0) Bootstrap, extensions, domains, enums

```
-- One-time cluster setup (superuser)
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS btree_gin;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- Optional schedulers (install at DB level)
-- CREATE EXTENSION IF NOT EXISTS pg_cron;
-- CREATE EXTENSION IF NOT EXISTS timescaledb;

-- Domain types for money & quantities
CREATE DOMAIN dom_money AS NUMERIC(20,6) CHECK (VALUE IS NOT NULL);
CREATE DOMAIN dom_qty AS NUMERIC(20,6) CHECK (VALUE IS NOT NULL);
CREATE DOMAIN dom_pct AS NUMERIC(9,4);

-- Enums
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='movement_type') THEN
        CREATE TYPE movement_type AS ENUM (
            'OPENING', 'PURCHASE', 'PURCHASE_RETURN',
            'TRANSFER', 'ISSUE_TO_WO', 'RECEIPT_FROM_WO',
            'SALE', 'SALES_RETURN', 'ADJUSTMENT',
            'CONSIGNMENT_IN', 'CONSIGNMENT_OUT', 'COUNT_VARIANCE'
        );
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='transfer_class') THEN
        CREATE TYPE transfer_class AS ENUM ('HORIZONTAL', 'VERTICAL', 'NA');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='quality_status') THEN
        CREATE TYPE quality_status AS ENUM ('FREE', 'HOLD', 'BLOCKED', 'EXPIRED');
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='valuation_method') THEN
        CREATE TYPE valuation_method AS ENUM ('WAVG', 'FIFO', 'LIFO', 'STD');
```

```

END IF;

IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='doc_type') THEN
    CREATE TYPE doc_type AS ENUM
('PO', 'GRN', 'SO', 'INV', 'CN', 'STOCK_COUNT', 'INT_TRF', 'WO');
END IF;
END $$;

```

1) Security: tenants, roles, RLS template

```

-- Tenants
CREATE TABLE enterprise (
    enterprise_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    code TEXT UNIQUE NOT NULL,
    name TEXT NOT NULL,
    base_currency TEXT NOT NULL DEFAULT 'ZMW',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Minimal role model (app-level)
-- CREATE ROLE app_admin NOINHERIT;
-- CREATE ROLE app_enterprise_admin NOINHERIT;
-- CREATE ROLE app_user NOINHERIT;

-- Every tenant session must set this (via app connection pooler / JWT
claims)
-- SELECT set_config('erp.current_enterprise', '<uuid>', false);

-- RLS helper
CREATE OR REPLACE FUNCTION erp.current_enterprise_id() RETURNS uuid
LANGUAGE sql STABLE AS $$
    SELECT NULLIF(current_setting('erp.current_enterprise', true), '')::uuid
$$;

-- Enable secure schema for functions
CREATE SCHEMA IF NOT EXISTS erp AUTHORIZATION CURRENT_USER;

```

2) Organization model (variable tiers)

```

CREATE TABLE org_unit (
    org_unit_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
CASCADE,
    name TEXT NOT NULL,
    code TEXT,
    tier_level INT NOT NULL CHECK (tier_level >= 1),
    parent_id UUID REFERENCES org_unit(org_unit_id) ON DELETE SET NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (enterprise_id, code)

```

```
);
CREATE INDEX ON org_unit (enterprise_id, tier_level);
CREATE INDEX ON org_unit (enterprise_id, parent_id);

ALTER TABLE org_unit ENABLE ROW LEVEL SECURITY;
CREATE POLICY org_unit_tenant_qls ON org_unit
USING (enterprise_id = erp.current_enterprise_id());
```

3) Master data: UoM, items, cost policy, FX, tax, partners

```
CREATE TABLE uom (
  uom_code TEXT PRIMARY KEY,
  description TEXT
);

CREATE TABLE item (
  item_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
CASCADE,
  name TEXT NOT NULL,
  sku TEXT,
  base_uom TEXT NOT NULL REFERENCES uom(uom_code),
  alt_uom TEXT REFERENCES uom(uom_code),
  alt_to_base_factor dom_qty,          -- alt * factor = base
  category TEXT,
  expiry_days INT,
  serialized BOOLEAN NOT NULL DEFAULT FALSE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (enterprise_id, sku)
);
CREATE INDEX ON item (enterprise_id, category);

ALTER TABLE item ENABLE ROW LEVEL SECURITY;
CREATE POLICY item_tenant_qls ON item
USING (enterprise_id = erp.current_enterprise_id());

CREATE TABLE item_cost_policy (
  item_id UUID PRIMARY KEY REFERENCES item(item_id) ON DELETE CASCADE,
  method valuation_method NOT NULL DEFAULT 'WAVG',
  standard_cost dom_money
);

-- FX & currency
CREATE TABLE currency_rate (
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
CASCADE,
  ccy TEXT NOT NULL,
  rate_to_base NUMERIC(20,8) NOT NULL CHECK (rate_to_base > 0),
  rate_date DATE NOT NULL,
  PRIMARY KEY (enterprise_id, ccy, rate_date)
```

```

);

ALTER TABLE currency_rate ENABLE ROW LEVEL SECURITY;
CREATE POLICY currency_rate_rls ON currency_rate
USING (enterprise_id = erp.current_enterprise_id());

-- Basic tax map (Smart Invoice-capable)
CREATE TABLE tax_code (
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
    CASCADE,
    tax_code TEXT NOT NULL,
    description TEXT,
    rate dom_pct NOT NULL CHECK (rate >= 0),
    is_vat BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (enterprise_id, tax_code)
);

ALTER TABLE tax_code ENABLE ROW LEVEL SECURITY;
CREATE POLICY tax_code_rls ON tax_code
USING (enterprise_id = erp.current_enterprise_id());

-- Partners
CREATE TABLE partner (
    partner_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
    CASCADE,
    name TEXT NOT NULL,
    type TEXT NOT NULL,                    -- SUPPLIER, CUSTOMER, 3PL, etc.
    tax_id TEXT,
    currency TEXT,                        -- partner default currency
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON partner (enterprise_id, type);

ALTER TABLE partner ENABLE ROW LEVEL SECURITY;
CREATE POLICY partner_rls ON partner
USING (enterprise_id = erp.current_enterprise_id());

```

4) Batches, serials, FEFO

```

CREATE TABLE batch (
    batch_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
    CASCADE,
    item_id UUID NOT NULL REFERENCES item(item_id) ON DELETE CASCADE,
    batch_no TEXT,                        -- supplier/internal lot
    serial_no TEXT,                      -- populated if serialized
    mfg_date DATE,
    expiry_date DATE,
    quality quality_status NOT NULL DEFAULT 'FREE',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (enterprise_id, item_id, batch_no, COALESCE(serial_no, '')) -- no dup
    combo
);

```

```

CREATE INDEX ON batch (enterprise_id, item_id, expiry_date);
CREATE INDEX ON batch (enterprise_id, item_id, quality);

ALTER TABLE batch ENABLE ROW LEVEL SECURITY;
CREATE POLICY batch_rls ON batch
USING (enterprise_id = erp.current_enterprise_id());

```

5) Document headers (typed), Smart-Invoice payloads

```

CREATE TABLE doc_header (
  doc_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
  CASCADE,
  doc_no TEXT NOT NULL,
  dtype doc_type NOT NULL,
  doc_date DATE NOT NULL,
  partner_id UUID REFERENCES partner(partner_id),
  currency TEXT,                -- document currency (for FX)
  status TEXT NOT NULL DEFAULT 'OPEN', -- OPEN/POSTED/CLOSED/CANCELLED
  created_by TEXT,
  meta JSONB NOT NULL DEFAULT '{}'::jsonb,
  UNIQUE (enterprise_id, dtype, doc_no)
);
CREATE INDEX ON doc_header (enterprise_id, dtype, doc_date);

ALTER TABLE doc_header ENABLE ROW LEVEL SECURITY;
CREATE POLICY doc_header_rls ON doc_header
USING (enterprise_id = erp.current_enterprise_id());

-- Smart Invoice payload store (optional)
CREATE TABLE smart_invoice_payload (
  doc_id UUID PRIMARY KEY REFERENCES doc_header(doc_id) ON DELETE CASCADE,
  zra_json JSONB NOT NULL,
  status TEXT NOT NULL DEFAULT 'PENDING', --
  PENDING/SUBMITTED/ACCEPTED/REJECTED
  last_response JSONB,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

6) Stock Ledger (partitioned), classification, constraints

Partitioning: by month for scale.

Rule: ledger is the **golden source**; nothing else mutates quantities.

```

CREATE TABLE stock_ledger (

```

```

    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
CASCADE,
    ledger_id UUID NOT NULL DEFAULT uuid_generate_v4(),
    txn_ts TIMESTAMPTZ NOT NULL DEFAULT now(),
    txn_date DATE NOT NULL DEFAULT CURRENT_DATE,
    org_unit_id UUID NOT NULL REFERENCES org_unit(org_unit_id) ON DELETE
RESTRICT,
    item_id UUID NOT NULL REFERENCES item(item_id) ON DELETE RESTRICT,
    batch_id UUID REFERENCES batch(batch_id) ON DELETE RESTRICT,
    movement movement_type NOT NULL,
    qty_units dom_qty NOT NULL DEFAULT 0,
    qty_weight dom_qty NOT NULL DEFAULT 0,
    qty_volume dom_qty NOT NULL DEFAULT 0,
    value dom_money NOT NULL DEFAULT 0,          -- in enterprise base
currency
    source_org_unit_id UUID REFERENCES org_unit(org_unit_id),
    dest_org_unit_id UUID REFERENCES org_unit(org_unit_id),
    transfer_cls transfer_class NOT NULL DEFAULT 'NA',
    quality quality_status NOT NULL DEFAULT 'FREE',
    doc_id UUID REFERENCES doc_header(doc_id),
    reference_no TEXT,
    gl_posted BOOLEAN NOT NULL DEFAULT FALSE,
    meta JSONB NOT NULL DEFAULT '{}'::jsonb,
    PRIMARY KEY (ledger_id, txn_date)            -- allows range partition
pruning
) PARTITION BY RANGE (txn_date);

-- Create rolling partitions (example: 3 years)
DO $$
DECLARE d DATE := date_trunc('month', current_date) - interval '18 months';
DECLARE stop DATE := date_trunc('month', current_date) + interval '18
months';
DECLARE part TEXT;
BEGIN
    WHILE d <= stop LOOP
        part := to_char(d, '"p_stock_ledger_"YYYY_MM');
        EXECUTE format('
            CREATE TABLE IF NOT EXISTS %I PARTITION OF stock_ledger
            FOR VALUES FROM (%L) TO (%L);',
            part, d::date, (d + interval '1 month')::date
        );
        d := d + interval '1 month';
    END LOOP;
END $$;

-- Helpful indexes (create on parent; propagate to children in PG15+ by
default)
CREATE INDEX IF NOT EXISTS stock_ledger_idx1 ON stock_ledger (enterprise_id,
txn_date, movement);
CREATE INDEX IF NOT EXISTS stock_ledger_idx2 ON stock_ledger (enterprise_id,
org_unit_id, txn_date);
CREATE INDEX IF NOT EXISTS stock_ledger_idx3 ON stock_ledger (enterprise_id,
item_id, txn_date);
CREATE INDEX IF NOT EXISTS stock_ledger_idx4 ON stock_ledger (transfer_cls)
WHERE movement='TRANSFER';
CREATE INDEX IF NOT EXISTS stock_ledger_meta_gin ON stock_ledger USING gin
(meta);

```

```

ALTER TABLE stock_ledger ENABLE ROW LEVEL SECURITY;
CREATE POLICY stock_ledger_rls ON stock_ledger
USING (enterprise_id = erp.current_enterprise_id());

-- Compute & enforce transfer classification
CREATE OR REPLACE FUNCTION trg_set_transfer_class() RETURNS trigger AS $$
DECLARE src_t INT; dst_t INT;
BEGIN
    IF NEW.movement = 'TRANSFER' AND NEW.source_org_unit_id IS NOT NULL AND
NEW.dest_org_unit_id IS NOT NULL THEN
        SELECT tier_level INTO src_t FROM org_unit WHERE org_unit_id =
NEW.source_org_unit_id;
        SELECT tier_level INTO dst_t FROM org_unit WHERE org_unit_id =
NEW.dest_org_unit_id;
        NEW.transfer_cls := CASE WHEN src_t IS NOT NULL AND dst_t IS NOT NULL AND
src_t = dst_t
                                THEN 'HORIZONTAL' ELSE 'VERTICAL' END;
    ELSE
        NEW.transfer_cls := 'NA';
    END IF;
    -- sanity: no self-transfer
    IF NEW.movement='TRANSFER' AND NEW.source_org_unit_id =
NEW.dest_org_unit_id THEN
        RAISE EXCEPTION 'Source and destination cannot be the same for
transfers';
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS set_transfer_class ON stock_ledger;
CREATE TRIGGER set_transfer_class
BEFORE INSERT OR UPDATE ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION trg_set_transfer_class();

-- Hard guard against negative outflows (optional strict mode)
CREATE OR REPLACE FUNCTION trg_no_negative_stock() RETURNS trigger AS $$
DECLARE bal dom_qty;
BEGIN
    -- Only check when NEW reduces stock for FREE/HOLD (BLOCKED/EXPIRED flows
are restricted elsewhere)
    IF NEW.qty_units < 0 AND NEW.quality IN ('FREE','HOLD') THEN
        SELECT COALESCE(SUM(qty_units),0) INTO bal
        FROM stock_ledger
        WHERE enterprise_id = NEW.enterprise_id
          AND org_unit_id = NEW.org_unit_id
          AND item_id = NEW.item_id
          AND COALESCE(batch_id,'00000000-0000-0000-0000-000000000000') =
COALESCE(NEW.batch_id,'00000000-0000-0000-0000-000000000000');
        IF bal + NEW.qty_units < 0 THEN
            RAISE EXCEPTION 'Negative stock not allowed for item %, org %, batch %
(bal %, delta %)',
                NEW.item_id, NEW.org_unit_id, NEW.batch_id, bal, NEW.qty_units;
        END IF;
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;

```

```
DROP TRIGGER IF EXISTS no_negative_stock ON stock_ledger;
CREATE TRIGGER no_negative_stock
BEFORE INSERT ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION trg_no_negative_stock();
```

7) Audit, change history, soft deletes (pattern)

```
CREATE TABLE audit_log (
  audit_id BIGSERIAL PRIMARY KEY,
  enterprise_id UUID NOT NULL,
  table_name TEXT NOT NULL,
  pk JSONB NOT NULL,
  action TEXT NOT NULL,          -- INSERT/UPDATE/DELETE
  changed_by TEXT,
  changed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  diff JSONB
);
CREATE INDEX ON audit_log (enterprise_id, table_name, changed_at DESC);

-- Example: capture stock_ledger writes (use pgAudit in prod if preferred)
CREATE OR REPLACE FUNCTION log_stock_ledger() RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO audit_log (enterprise_id, table_name, pk, action, changed_by,
diff)
      VALUES (NEW.enterprise_id, 'stock_ledger',
jsonb_build_object('ledger_id', NEW.ledger_id), 'INSERT',
      current_user, to_jsonb(NEW));
  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO audit_log (enterprise_id, table_name, pk, action, changed_by,
diff)
      VALUES (NEW.enterprise_id, 'stock_ledger',
jsonb_build_object('ledger_id', NEW.ledger_id), 'UPDATE',
      current_user, jsonb_build_object('old', to_jsonb(OLD), 'new',
to_jsonb(NEW)));
  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO audit_log (enterprise_id, table_name, pk, action, changed_by,
diff)
      VALUES (OLD.enterprise_id, 'stock_ledger',
jsonb_build_object('ledger_id', OLD.ledger_id), 'DELETE',
      current_user, to_jsonb(OLD));
  END IF;
  RETURN COALESCE(NEW, OLD);
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS audit_stock_ledger ON stock_ledger;
CREATE TRIGGER audit_stock_ledger
AFTER INSERT OR UPDATE OR DELETE ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION log_stock_ledger();
```

8) Costing: rolling balances, WAVG, FIFO layer stubs

```
-- Daily deltas (MV for speed)
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_item_daily_delta AS
SELECT
    enterprise_id, org_unit_id, item_id, batch_id, txn_date AS as_of,
    SUM(qty_units) AS units_delta,
    SUM(qty_weight) AS weight_delta,
    SUM(qty_volume) AS volume_delta,
    SUM(value) AS value_delta
FROM stock_ledger
GROUP BY 1,2,3,4,5
WITH NO DATA;

CREATE INDEX IF NOT EXISTS mv_item_daily_delta_idx ON mv_item_daily_delta
(enterprise_id, org_unit_id, item_id, batch_id, as_of);

-- Balances-as-of (window over MV)
CREATE OR REPLACE VIEW v_item_balance_asof AS
SELECT
    enterprise_id, org_unit_id, item_id, batch_id, as_of,
    SUM(units_delta) OVER w AS units_balance,
    SUM(weight_delta) OVER w AS weight_balance,
    SUM(volume_delta) OVER w AS volume_balance,
    SUM(value_delta) OVER w AS value_balance
FROM mv_item_daily_delta
WINDOW w AS (PARTITION BY enterprise_id, org_unit_id, item_id, batch_id
              ORDER BY as_of ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
              ROW);

-- WAVG inflows per day, then cumulative unit cost
CREATE OR REPLACE VIEW v_item_cost_wavg AS
WITH inflow AS (
    SELECT enterprise_id, org_unit_id, item_id, as_of,
           SUM(CASE WHEN value_delta > 0 THEN value_delta ELSE 0 END) AS
inflow_value,
           SUM(CASE WHEN units_delta > 0 THEN units_delta ELSE 0 END) AS
inflow_units
    FROM mv_item_daily_delta
    GROUP BY 1,2,3,4
)
SELECT i.enterprise_id, i.org_unit_id, i.item_id, i.as_of,
       CASE WHEN SUM(inflow_units) OVER w = 0 THEN NULL
            ELSE (SUM(inflow_value) OVER w) / NULLIF(SUM(inflow_units) OVER
w,0) END AS wavg_unit_cost
FROM inflow i
WINDOW w AS (PARTITION BY i.enterprise_id, i.org_unit_id, i.item_id
              ORDER BY i.as_of ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
              ROW);

-- FIFO stubs (optional): layer tables you'll maintain via service
CREATE TABLE fifo_layer (
    fifo_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

    enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
    CASCADE,
    org_unit_id UUID NOT NULL REFERENCES org_unit(org_unit_id),
    item_id UUID NOT NULL REFERENCES item(item_id),
    batch_id UUID,
    layer_date DATE NOT NULL,
    qty_remaining dom_qty NOT NULL,
    unit_cost dom_money NOT NULL,
    CONSTRAINT pos_qty CHECK (qty_remaining >= 0)
);
CREATE INDEX ON fifo_layer (enterprise_id, org_unit_id, item_id, layer_date);

```

9) Consolidation helpers (rule baked-in)

```

-- Classified view (compute H/V once)
CREATE OR REPLACE VIEW v_stock_ledger_classified AS
SELECT
    sl.*,
    src.tier_level AS src_tier,
    dst.tier_level AS dst_tier,
    CASE
        WHEN sl.movement <> 'TRANSFER' OR sl.source_org_unit_id IS NULL OR
        sl.dest_org_unit_id IS NULL
        THEN 'NA'::transfer_class
        WHEN src.tier_level = dst.tier_level
        THEN 'HORIZONTAL'::transfer_class
        ELSE 'VERTICAL'::transfer_class
    END AS calc_transfer_cls
FROM stock_ledger sl
LEFT JOIN org_unit src ON src.org_unit_id = sl.source_org_unit_id
LEFT JOIN org_unit dst ON dst.org_unit_id = sl.dest_org_unit_id
WHERE sl.enterprise_id = erp.current_enterprise_id();

-- Scope temp table pattern (populate per request: list of org_unit_ids at
same tier)
-- CREATE TEMP TABLE tmp_scope (org_unit_id uuid PRIMARY KEY);

-- Same-tier consolidation: include vertical, exclude horizontal
CREATE OR REPLACE VIEW v_consolidated_movements AS
SELECT
    sl.enterprise_id,
    sl.item_id,
    sl.batch_id,
    sl.movement,
    sl.txn_date,
    -- quantities
    CASE WHEN sl.movement='TRANSFER' AND sl.calc_transfer_cls='HORIZONTAL' THEN
0
        ELSE sl.qty_units END AS units_cons,
    CASE WHEN sl.movement='TRANSFER' AND sl.calc_transfer_cls='HORIZONTAL' THEN
0
        ELSE sl.value END AS value_cons,
    sl.org_unit_id, sl.source_org_unit_id, sl.dest_org_unit_id,
    sl.calc_transfer_cls AS transfer_cls

```

```
FROM v_stock_ledger_classified sl
WHERE sl.enterprise_id = erp.current_enterprise_id();
```

For **department**→**sector** or **sector**→**enterprise** rollups on the **same tier**, aggregate over **v_consolidated_movements**: horizontals automatically drop, verticals remain.

10) GL posting hook + mapping

```
CREATE TABLE gl_map (
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
  CASCADE,
  movement movement_type NOT NULL,
  debit_acct TEXT NOT NULL,
  credit_acct TEXT NOT NULL,
  active BOOLEAN NOT NULL DEFAULT TRUE,
  PRIMARY KEY (enterprise_id, movement)
);

CREATE TABLE gl_posting (
  gl_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  enterprise_id UUID NOT NULL REFERENCES enterprise(enterprise_id) ON DELETE
  CASCADE,
  ledger_id UUID NOT NULL,
  posting_ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  account_dr TEXT NOT NULL,
  account_cr TEXT NOT NULL,
  amount_dom_money NOT NULL,
  currency TEXT NOT NULL,
  CHECK (amount >= 0)
);

CREATE INDEX ON gl_posting (enterprise_id, posting_ts DESC);
CREATE INDEX ON gl_posting (ledger_id);

-- Example proc: generate GL from ledger row (service should call within tx)
CREATE OR REPLACE FUNCTION gl_post_from_ledger(p_ledger_id uuid) RETURNS void
LANGUAGE plpgsql AS $$
DECLARE r record; m record; ccy TEXT;
BEGIN
  SELECT * INTO r FROM stock_ledger WHERE ledger_id = p_ledger_id AND
  enterprise_id = erp.current_enterprise_id();
  IF NOT FOUND THEN RAISE EXCEPTION 'Ledger % not found', p_ledger_id; END
  IF;

  -- skip non-value movements if desired
  IF r.value = 0 THEN RETURN; END IF;

  SELECT * INTO m FROM gl_map WHERE enterprise_id = r.enterprise_id AND
  movement = r.movement AND active;
  IF NOT FOUND THEN RAISE EXCEPTION 'GL map missing for movement %',
  r.movement; END IF;
```

```

    ccy := (SELECT base_currency FROM enterprise WHERE enterprise_id =
r.enterprise_id);

    INSERT INTO gl_posting(enterprise_id, ledger_id, account_dr, account_cr,
amount, currency)
    VALUES (r.enterprise_id, r.ledger_id, m.debit_acct, m.credit_acct,
ABS(r.value), ccy);

    UPDATE stock_ledger SET gl_posted = TRUE WHERE ledger_id = r.ledger_id;
END $$;

```

11) Report pack starters (compact; all rule-compliant)

Opening stock (as-of)

```

WITH last_day AS (SELECT (:from::date - 1)::date AS as_of)
SELECT b.org_unit_id, b.item_id, b.batch_id,
       b.units_balance AS opening_units, b.value_balance AS opening_value
FROM v_item_balance_asof b
JOIN last_day d ON b.as_of = d.as_of
WHERE b.enterprise_id = erp.current_enterprise_id()
      AND b.org_unit_id IN (SELECT org_unit_id FROM tmp_scope);

```

Purchases & landed cost

```

SELECT sl.org_unit_id, sl.item_id, sl.batch_id,
       SUM(sl.qty_units) qty,
       SUM(sl.value) base_value,
       SUM(COALESCE((sl.meta->>'duty')::numeric,0)
          + COALESCE((sl.meta->>'freight')::numeric,0)
          + COALESCE((sl.meta->>'insurance')::numeric,0)
          + COALESCE((sl.meta->>'other_lc')::numeric,0)) lc_value,
       SUM(sl.value
          + COALESCE((sl.meta->>'duty')::numeric,0)
          + COALESCE((sl.meta->>'freight')::numeric,0)
          + COALESCE((sl.meta->>'insurance')::numeric,0)
          + COALESCE((sl.meta->>'other_lc')::numeric,0)) total_landed
FROM stock_ledger sl
WHERE sl.enterprise_id = erp.current_enterprise_id()
      AND sl.org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND sl.movement='PURCHASE'
      AND sl.txn_date BETWEEN :from AND :to
GROUP BY sl.org_unit_id, sl.item_id, sl.batch_id;

```

Transfers (same-tier: horizontals auto-excluded)

```

SELECT source_org_unit_id, dest_org_unit_id, item_id, batch_id, transfer_cls,
       SUM(units_cons) qty_cons, SUM(value_cons) val_cons
FROM v_consolidated_movements

```

```

WHERE movement='TRANSFER'
  AND txn_date BETWEEN :from AND :to
  AND (source_org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
       OR dest_org_unit_id IN (SELECT org_unit_id FROM tmp_scope))
GROUP BY source_org_unit_id, dest_org_unit_id, item_id, batch_id,
transfer_cls;

```

Production I/O, Yield, Sales, Closing, Abnormal, Stocktake, Consignment, Expiry

(Use the versions from my previous message—they plug directly into this schema. They're already consolidation-safe when you source from v_consolidated_movements or ledger appropriately.)

12) Validations & business rules (DB-side)

```

-- FEFO enforcement helper: blocks issuing expired when policy forbids
CREATE OR REPLACE FUNCTION check_fefo_issue() RETURNS trigger AS $$
DECLARE exp DATE;
BEGIN
  IF NEW.movement IN ('ISSUE_TO_WO', 'SALE', 'TRANSFER') AND NEW.batch_id IS
NOT NULL THEN
    SELECT expiry_date INTO exp FROM batch WHERE batch_id = NEW.batch_id AND
enterprise_id = NEW.enterprise_id;
    IF exp IS NOT NULL AND exp < NEW.txn_date THEN
      RAISE EXCEPTION 'Cannot issue expired batch % (expiry % < txn %)',
NEW.batch_id, exp, NEW.txn_date;
    END IF;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_fefo_block ON stock_ledger;
CREATE TRIGGER trg_fefo_block
BEFORE INSERT ON stock_ledger
FOR EACH ROW EXECUTE FUNCTION check_fefo_issue();

-- Transfers must net zero per item enterprise-wide (soft validation view for
audit)
CREATE OR REPLACE VIEW v_transfer_integrity AS
SELECT enterprise_id, item_id, txn_date,
       SUM(CASE WHEN movement='TRANSFER' THEN qty_units ELSE 0 END) AS
qty_net,
       SUM(CASE WHEN movement='TRANSFER' THEN value ELSE 0 END) AS val_net
FROM stock_ledger
GROUP BY enterprise_id, item_id, txn_date
HAVING ABS(SUM(CASE WHEN movement='TRANSFER' THEN qty_units ELSE 0 END)) >
1e-9
       OR ABS(SUM(CASE WHEN movement='TRANSFER' THEN value ELSE 0 END)) > 1e-6;
-- Expect 0 rows for perfect netting

```

13) Performance notes

- **Partitioning:** ledger by month; create partitions ahead (cron job).
 - **MVs:** refresh `mv_item_daily_delta` on a schedule after ETL window.
 - **Indexes:** covered common filters; add partials by movement/date where needed.
 - **Workloads:** heavy reads on views; consider **TimescaleDB** chunks for time series.
 - **VACUUM:** autovacuum tuned for high-insert partitions.
 - **Connection pooling:** PgBouncer transaction pooling for web; session GUC set per request.
-

14) Job scheduling (pg_cron examples)

```
-- Refresh daily deltas at 02:30 local
-- SELECT cron.schedule('refresh_mv_item_daily_delta',
--   '30 2 * * *',
--   $$ REFRESH MATERIALIZED VIEW CONCURRENTLY mv_item_daily_delta; $$);

-- Create next 6 months partitions monthly
-- SELECT cron.schedule('roll_partitions',
--   '5 3 1 * *',
--   $$ CALL erp.ensure_future_partitions(6); $$);
```

Helper to create partitions:

```
CREATE OR REPLACE PROCEDURE erp.ensure_future_partitions(months_ahead int)
LANGUAGE plpgsql AS $$
DECLARE d DATE := date_trunc('month', current_date);
DECLARE i int := 0; part TEXT;
BEGIN
  WHILE i < months_ahead LOOP
    part := to_char(d, 'p_stock_ledger_YYYY_MM');
    EXECUTE format('
      CREATE TABLE IF NOT EXISTS %I PARTITION OF stock_ledger
      FOR VALUES FROM (%L) TO (%L);', part, d::date, (d + interval '1
month')::date);
    d := d + interval '1 month';
    i := i + 1;
  END LOOP;
END $$;
```

15) Minimal seed & usage checklist

1. **Insert tenant** in enterprise, set `erp.current_enterprise` GUC in app session.

2. Seed **UoM**, **org_unit** (with `tier_level`), **item**, **item_cost_policy**, **partners**, **tax_code**, **gl_map**.
 3. Load **FX rates** if you will accept non-base currency docs (service converts to base for `stock_ledger.value`).
 4. For each operation (purchase, transfer, issue, receipt, sale, adjustment), write one or more rows to **stock_ledger** with correct signs and metadata.
 5. Refresh MV: `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_item_daily_delta;`
 6. Run report queries (opening, purchases, transfers, production, yield, sales, closing, etc.) using:
 - o A **same-tier** `tmp_scope` of `org_unit_ids`,
 - o `v_consolidated_movements` so horizontals are excluded and verticals included.
 7. Generate GL with `SELECT gl_post_from_ledger(:ledger_id);` per txn (or batch).
 8. Monitor **v_transfer_integrity** (should be empty).
 9. Audit trail lands in `audit_log`.
-

16) Developer notes & guarantees

- **No duplication in consolidation:** horizontals excluded at the **view level**; verticals included and appear exactly once at same-tier rollups.
 - **Traceability:** every report can drill back to `stock_ledger.ledger_id` and its `doc_id`.
 - **Multi-currency:** store `stock_ledger.value` in **base currency**; keep original doc FX in `doc_header` meta if needed.
 - **Costing:** WAVG shipped; FIFO/LIFO layer table ready.
 - **Compliance:** Smart-Invoice payload storage included; tax code rates supported.
 - **Security:** per-tenant RLS on all main tables; require app to set `erp.current_enterprise`.
 - **Scale:** partitioned ledger; MVs; indexes for hot paths; cronable maintenance.
-

Here's a **production-grade FastAPI scaffold** that slots directly on top of the SQL you now have. It:

- Enforces **tenant isolation** by setting the `erp.current_enterprise` GUC per request (so RLS works).
 - Exposes clean **endpoints** for: posting ledger movements, GL posting, refreshing materialized views, and key **report packs** (opening, purchases+LC, transfers w/ your same-tier rule, production I/O, sales, closing, abnormal).
 - Uses **Async SQLAlchemy 2.0** + **asyncpg**, structured logging, env-driven config, and safe connection lifecycle.
 - Keeps **SQL in versioned files** (so DBAs can diff/approve).
 - Includes a tiny **smoke dataset** and **cURL examples** to run day-one.
-

1) Project layout

```
erp-fastapi/
├── app/
│   ├── main.py
│   ├── config.py
│   ├── db.py
│   ├── deps.py
│   ├── logging_conf.py
│   ├── schemas/
│   │   └── ledger.py
│   ├── routes/
│   │   ├── health.py
│   │   ├── admin.py           # refresh MVs, partitions, seeds
│   │   ├── ledger.py         # create movements
│   │   └── reports.py        # report packs
│   └── sql/
│       ├── reports/
│       │   ├── opening.sql
│       │   ├── purchases_lc.sql
│       │   ├── transfers.sql
│       │   ├── prod_io.sql
│       │   ├── sales_returns.sql
│       │   ├── closing_wavg.sql
│       │   └── abnormal.sql
│       └── admin/
│           ├── refresh_mv_item_daily_delta.sql
│           └── gl_post_from_ledger.sql      # wrapper call
├── .env.example
├── requirements.txt
├── README.md
└── docker-compose.yml
```

2) requirements.txt

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
pydantic==2.9.2
SQLAlchemy[asyncio]==2.0.35
asyncpg==0.29.0
python-dotenv==1.0.1
structlog==24.1.0
```

3) .env.example

```
APP_ENV=dev
APP_HOST=0.0.0.0
APP_PORT=8080

# Postgres
DATABASE_URL=postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb

# Required for RLS scoping; your gateway will set this per request (see
header below)
DEFAULT_ENTERPRISE_ID=00000000-0000-0000-0000-000000000000

# Optional hard auth (JWT in your gateway/ingress)
API_KEY=dev-key
```

4) app/config.py

```
from pydantic import BaseModel
import os

class Settings(BaseModel):
    app_env: str = os.getenv("APP_ENV", "dev")
    host: str = os.getenv("APP_HOST", "0.0.0.0")
    port: int = int(os.getenv("APP_PORT", "8080"))
    database_url: str = os.getenv("DATABASE_URL", "")
    default_enterprise_id: str = os.getenv("DEFAULT_ENTERPRISE_ID", "")
    api_key: str = os.getenv("API_KEY", "dev-key")

settings = Settings()
```

5) app/logging_conf.py

```
import logging, structlog

def setup_logging():
    logging.basicConfig(level=logging.INFO)
    structlog.configure(
        wrapper_class=structlog.make_filtering_bound_logger(logging.INFO),
```

```
        processors=[structlog.processors.TimeStamper(fmt="iso"),
structlog.processors.JSONRenderer()],
    )
```

6) app/db.py (async engine + tenant GUC helper)

```
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker,
AsyncSession
from sqlalchemy import text
from app.config import settings

engine = create_async_engine(
    settings.database_url,
    echo=False,
    pool_pre_ping=True,
    pool_size=10,
    max_overflow=20,
)

SessionLocal = async_sessionmaker(engine, expire_on_commit=False,
class_=AsyncSession)

async def set_enterprise_guc(session: AsyncSession, enterprise_id: str):
    # SET LOCAL applies for the duration of the transaction
    await session.execute(text("SET LOCAL exp.current_enterprise = :eid"),
{"eid": enterprise_id})
```

7) app/deps.py (per-request tenant & basic API key)

```
from fastapi import Header, HTTPException, Depends
from app.config import settings

async def get_enterprise_id(x_enterprise_id: str | None =
Header(default=None)):
    # Prefer explicit header, fallback to default (useful for dev/seeding)
    eid = x_enterprise_id or settings.default_enterprise_id
    if not eid:
        raise HTTPException(status_code=400, detail="Missing X-Enterprise-
Id")
    return eid

async def require_api_key(x_api_key: str | None = Header(default=None)):
    if settings.api_key and x_api_key != settings.api_key:
        raise HTTPException(status_code=401, detail="Unauthorized")
```

Gateway contract: send X-Enterprise-Id: <uuid> per request.

8) app/schemas/ledger.py (pydantic models)

```
from pydantic import BaseModel, Field
from typing import Optional, Literal
from decimal import Decimal

Movement = Literal[
    "OPENING", "PURCHASE", "PURCHASE_RETURN",
    "TRANSFER", "ISSUE_TO_WO", "RECEIPT_FROM_WO",
    "SALE", "SALES_RETURN", "ADJUSTMENT",
    "CONSIGNMENT_IN", "CONSIGNMENT_OUT", "COUNT_VARIANCE"
]

class LedgerIn(BaseModel):
    txn_date: str # YYYY-MM-DD
    org_unit_id: str
    item_id: str
    batch_id: str | None = None
    movement: Movement
    qty_units: Decimal = Field(default=0)
    qty_weight: Decimal = Field(default=0)
    qty_volume: Decimal = Field(default=0)
    value: Decimal = Field(default=0)
    source_org_unit_id: Optional[str] = None
    dest_org_unit_id: Optional[str] = None
    quality: Literal["FREE", "HOLD", "BLOCKED", "EXPIRED"] = "FREE"
    doc_id: Optional[str] = None
    reference_no: Optional[str] = None
    meta: dict = {}

class LedgerOut(BaseModel):
    ledger_id: str
    gl_posted: bool
```

9) app/routes/health.py

```
from fastapi import APIRouter

router = APIRouter(prefix="/health", tags=["health"])

@router.get("")
async def health():
    return {"status": "ok"}
```

10) app/routes/admin.py (refresh MVs, GL post)

```
from fastapi import APIRouter, Depends
from sqlalchemy import text
from app.db import SessionLocal, set_enterprise_guc
from app.deps import get_enterprise_id, require_api_key

router = APIRouter(prefix="/admin", tags=["admin"])

@router.post("/refresh/daily-delta", dependencies=[Depends(require_api_key)])
async def refresh_mv(enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await s.execute(text("REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_item_daily_delta"))
        return {"status": "refreshed"}

@router.post("/gl/post/{ledger_id}", dependencies=[Depends(require_api_key)])
async def gl_post_one(ledger_id: str, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await s.execute(text("SELECT gl_post_from_ledger(:lid)"), {"lid":
ledger_id})
        return {"status": "posted", "ledger_id": ledger_id}
```

11) app/routes/ledger.py (create movement)

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import text
from app.db import SessionLocal, set_enterprise_guc
from app.deps import get_enterprise_id, require_api_key
from app.schemas.ledger import LedgerIn, LedgerOut

router = APIRouter(prefix="/ledger", tags=["ledger"])

INSERT_SQL = """
INSERT INTO stock_ledger (
    enterprise_id, txn_date, org_unit_id, item_id, batch_id, movement,
    qty_units, qty_weight, qty_volume, value,
    source_org_unit_id, dest_org_unit_id, quality, doc_id, reference_no, meta
) VALUES (
    erp.current_enterprise_id(), :txn_date, :org_unit_id, :item_id, :batch_id,
:movement,
    :qty_units, :qty_weight, :qty_volume, :value,
    :source_org_unit_id, :dest_org_unit_id, :quality, :doc_id, :reference_no,
:meta::jsonb
```

```

)
RETURNING ledger_id, gl_posted
"""

@router.post("", response_model=LedgerOut,
dependencies=[Depends(require_api_key)])
async def create_ledger(entry: LedgerIn, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            try:
                res = await s.execute(text(INSERT_SQL), entry.model_dump())
                row = res.one()
                return {"ledger_id": str(row.ledger_id), "gl_posted":
row.gl_posted}
            except Exception as e:
                raise HTTPException(status_code=400, detail=str(e))

```

12) app/routes/reports.py (report packs)

```

from fastapi import APIRouter, Depends, Query
from sqlalchemy import text
from app.db import SessionLocal, set_enterprise_guc
from app.deps import get_enterprise_id

router = APIRouter(prefix="/reports", tags=["reports"])

# Helper: scope list as temp table
async def _set_scope(session, org_unit_ids: list[str]):
    await session.execute(text("CREATE TEMP TABLE IF NOT EXISTS tmp_scope
(org_unit_id uuid primary key) ON COMMIT DROP"))
    await session.execute(text("TRUNCATE tmp_scope"))
    if org_unit_ids:
        await session.execute(
            text("INSERT INTO tmp_scope(org_unit_id) SELECT
UNNEST(:ids)::uuid"),
            {"ids": org_unit_ids},
        )

async def _run_sql(session, path: str, params: dict):
    with open(path, "r", encoding="utf-8") as f:
        sql = f.read()
    res = await session.execute(text(sql), params)
    cols = list(res.keys())
    return [dict(zip(cols, row)) for row in res.fetchall()]

@router.get("/opening")
async def opening(as_of: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)

```

```

        return await _run_sql(s, "app/sql/reports/opening.sql", {"from":
as_of})

@router.get("/purchases")
async def purchases(frm: str, to: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/purchases_lc.sql",
{"from": frm, "to": to})

@router.get("/transfers")
async def transfers(frm: str, to: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/transfers.sql",
{"from": frm, "to": to})

@router.get("/production-io")
async def prod_io(frm: str, to: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/prod_io.sql", {"from":
frm, "to": to})

@router.get("/sales")
async def sales(frm: str, to: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/sales_returns.sql",
{"from": frm, "to": to})

@router.get("/closing")
async def closing(as_of: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/closing_wavg.sql",
{"to": as_of})

@router.get("/abnormal")
async def abnormal(frm: str, to: str, org_units: list[str] = Query(...),
enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:

```

```

        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)
            return await _run_sql(s, "app/sql/reports/abnormal.sql", {"from":
frm, "to": to})

```

13) app/sql/reports/*.sql (ready-to-run)

opening.sql

```

WITH last_day AS (SELECT (:from::date - 1)::date AS as_of)
SELECT b.org_unit_id, b.item_id, b.batch_id,
       b.units_balance AS opening_units,
       b.value_balance AS opening_value
FROM v_item_balance_asof b
JOIN last_day d ON b.as_of = d.as_of
WHERE b.enterprise_id = erp.current_enterprise_id()
      AND b.org_unit_id IN (SELECT org_unit_id FROM tmp_scope);

```

purchases_lc.sql

```

SELECT sl.org_unit_id, sl.item_id, sl.batch_id,
       SUM(sl.qty_units) qty,
       SUM(sl.value) base_value,
       SUM(COALESCE((sl.meta->>'duty')::numeric,0)
          + COALESCE((sl.meta->>'freight')::numeric,0)
          + COALESCE((sl.meta->>'insurance')::numeric,0)
          + COALESCE((sl.meta->>'other_lc')::numeric,0)) lc_value,
       SUM(sl.value
          + COALESCE((sl.meta->>'duty')::numeric,0)
          + COALESCE((sl.meta->>'freight')::numeric,0)
          + COALESCE((sl.meta->>'insurance')::numeric,0)
          + COALESCE((sl.meta->>'other_lc')::numeric,0)) total_landed
FROM stock_ledger sl
WHERE sl.enterprise_id = erp.current_enterprise_id()
      AND sl.org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND sl.movement='PURCHASE'
      AND sl.txn_date BETWEEN :from AND :to
GROUP BY sl.org_unit_id, sl.item_id, sl.batch_id;

```

transfers.sql (€ consolidation rule applied)

```

SELECT source_org_unit_id, dest_org_unit_id, item_id, batch_id, transfer_cls,
       SUM(units_cons) qty_cons,
       SUM(value_cons) val_cons
FROM v_consolidated_movements
WHERE enterprise_id = erp.current_enterprise_id()
      AND movement='TRANSFER'
      AND txn_date BETWEEN :from AND :to
      AND (source_org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
          OR dest_org_unit_id IN (SELECT org_unit_id FROM tmp_scope))

```

```
GROUP BY source_org_unit_id, dest_org_unit_id, item_id, batch_id,
transfer_cls;
```

prod_io.sql

```
WITH inputs AS (
  SELECT item_id, batch_id, SUM(qty_units) in_units, SUM(value) in_value
  FROM stock_ledger
  WHERE enterprise_id=erp.current_enterprise_id()
    AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
    AND movement='ISSUE_TO_WO'
    AND txn_date BETWEEN :from AND :to
  GROUP BY item_id, batch_id
),
outputs AS (
  SELECT item_id, batch_id, SUM(qty_units) out_units, SUM(value) out_value
  FROM stock_ledger
  WHERE enterprise_id=erp.current_enterprise_id()
    AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
    AND movement='RECEIPT_FROM_WO'
    AND txn_date BETWEEN :from AND :to
  GROUP BY item_id, batch_id
)
SELECT COALESCE(i.item_id,o.item_id) item_id,
       COALESCE(i.batch_id,o.batch_id) batch_id,
       COALESCE(in_units,0) in_units, COALESCE(in_value,0) in_value,
       COALESCE(out_units,0) out_units, COALESCE(out_value,0) out_value
FROM inputs i
FULL JOIN outputs o USING (item_id, batch_id);
```

sales_returns.sql

```
SELECT item_id, batch_id,
       SUM(CASE WHEN movement='SALE' THEN qty_units ELSE 0 END) AS
qty_sold,
       SUM(CASE WHEN movement='SALE' THEN value ELSE 0 END) AS
sales_value,
       SUM(CASE WHEN movement='SALES_RETURN' THEN qty_units ELSE 0 END) AS
qty_returned,
       SUM(CASE WHEN movement='SALES_RETURN' THEN value ELSE 0 END) AS
returns_value,
       SUM(CASE WHEN movement IN ('SALE','SALES_RETURN') THEN value ELSE 0
END) AS net_revenue
FROM stock_ledger
WHERE enterprise_id=erp.current_enterprise_id()
  AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
  AND movement IN ('SALE','SALES_RETURN')
  AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id;
```

closing_wavg.sql

```
WITH cost AS (
  SELECT enterprise_id, org_unit_id, item_id,
```

```

        MAX(as_of) AS as_of,
        MAX(wavg_unit_cost) FILTER (WHERE as_of <= :to::date) AS unit_cost
FROM v_item_cost_wavg
WHERE enterprise_id=erp.current_enterprise_id()
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND as_of <= :to::date
GROUP BY 1,2,3
),
bal AS (
  SELECT enterprise_id, org_unit_id, item_id, batch_id, units_balance
  FROM v_item_balance_asof
  WHERE enterprise_id=erp.current_enterprise_id()
        AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
        AND as_of = :to::date
)
SELECT b.org_unit_id, b.item_id, b.batch_id,
       b.units_balance AS closing_units,
       c.unit_cost,
       (b.units_balance * COALESCE(c.unit_cost,0)) AS closing_value
FROM bal b
LEFT JOIN cost c USING (enterprise_id, org_unit_id, item_id);

```

abnormal.sql

```

SELECT item_id, batch_id,
       SUM(qty_units) qty, SUM(value) value,
       meta->>'reason' AS reason
FROM stock_ledger
WHERE enterprise_id=erp.current_enterprise_id()
      AND org_unit_id IN (SELECT org_unit_id FROM tmp_scope)
      AND movement='ADJUSTMENT'
      AND (meta->>'abnormal')::boolean = true
      AND txn_date BETWEEN :from AND :to
GROUP BY item_id, batch_id, reason;

```

14) app/main.py

```

import uvicorn
from fastapi import FastAPI
from app.logging_conf import setup_logging
from app.routes import health, admin, ledger, reports
from app.config import settings

def create_app():
    setup_logging()
    app = FastAPI(title="ERP Reporting API", version="1.0.0")
    app.include_router(health.router)
    app.include_router(admin.router)
    app.include_router(ledger.router)
    app.include_router(reports.router)
    return app

app = create_app()

```

```
if __name__ == "__main__":
    uvicorn.run("app.main:app", host=settings.host, port=settings.port,
reload=True)
```

15) docker-compose.yml (optional local stack)

```
version: "3.9"
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: erp_user
      POSTGRES_PASSWORD: strongpass
      POSTGRES_DB: erpdb
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
  api:
    build: .
    command: uvicorn app.main:app --host 0.0.0.0 --port 8080 --reload
    environment:
      DATABASE_URL: postgresql+asyncpg://erp_user:strongpass@db:5432/erpdb
      DEFAULT_ENTERPRISE_ID: 00000000-0000-0000-0000-000000000000
      API_KEY: dev-key
    ports: ["8080:8080"]
    depends_on: ["db"]
volumes:
  pgdata:
```

16) Quick start

1. Apply the **SQL DDL** you already have (the production-ready starter from the previous step).
2. `cp .env.example .env` and set `DATABASE_URL`.
3. `pip install -r requirements.txt`
4. **Run:** `uvicorn app.main:app --host 0.0.0.0 --port 8080`
5. Seed minimal enterprise/org/items (or use your existing seed scripts).
6. Test health:

```
curl -s http://localhost:8080/health
```

17) Smoke flow (cURL)

Replace IDs with your real UUIDs; pass tenant header so RLS scopes correctly.

Create a purchase movement

```
curl -X POST http://localhost:8080/ledger \
  -H "Content-Type: application/json" \
  -H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000" \
  -H "X-API-Key: dev-key" \
  -d '{
    "txn_date": "2025-09-10",
    "org_unit_id": "<DEPT-UUID>",
    "item_id": "<ITEM-UUID>",
    "batch_id": "<BATCH-UUID>",
    "movement": "PURCHASE",
    "qty_units": "1000",
    "value": "50000",

    "meta": {
      "duty": "3000",
      "freight": "1000",
      "insurance": "500",
      "other_lc": "200"
    }
  }'
```

Refresh daily MV

```
curl -X POST http://localhost:8080/admin/refresh/daily-delta \
  -H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000" -H "X-API-Key: dev-key"
```

Run transfers report (same-tier: horizontals excluded, verticals included)

```
curl "http://localhost:8080/reports/transfers?frm=2025-09-01&to=2025-09-30&org_units=<DEPT-UUID>&org_units=<DEPT2-UUID>" \
  -H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000"
```

Closing stock (WAVG)

```
curl "http://localhost:8080/reports/closing?as_of=2025-09-30&org_units=<DEPT-UUID>" \
  -H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000"
```

18) Why this is “production-ready”

- **RLS** + per-request **GUC** = safe multi-tenant isolation.
 - **Async** DB w/ pooling; explicit **transactions**; **SET LOCAL** inside the tx.
 - **SQL kept in repo** (under `app/sql/`) for DB review/versioning.
 - **Consolidation rule enforced in views** (`v_consolidated_movements`) so all reports inherit: **include vertical, exclude horizontal** at same tier.
 - **Materialized views** and **refresh endpoints/jobs** for scale.
 - **Structured logs** and clear **error surfacing**.
 - **GL posting hook** ready; one call per ledger entry or batch.
-

Below is a **complete add-on pack** with:

1. **Alembic migrations** (init + first migration that creates everything)
2. **Seed script** (realistic data & flows)
3. **Audit Pack endpoint** (exports all report packs for a period and returns a ZIP)
4. Small glue updates to config/requirements/router

It all plugs straight into the FastAPI + SQL stack I gave you.

1) Add dependencies

requirements.txt (append)

```
alembic==1.13.2
pandas==2.2.2
openpyxl==3.1.5
python-multipart==0.0.9
```

2) Alembic setup

New files

```
erp-fastapi/
├── alembic.ini
├── migrations/
│   ├── env.py
│   ├── script.py.mako
│   └── versions/
│       └── 0001_init_schema.py
```

alembic.ini

```
[alembic]
script_location = migrations
sqlalchemy.url =
postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb
prepend_sys_path = .

[loggers]
keys = root,sqlalchemy,alembic

[handlers]
keys = console

[formatters]
```

```

keys = generic

[logger_root]
level = WARN
handlers = console

[logger_sqlalchemy]
level = WARN
handlers = console
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers = console
qualname = alembic

[handler_console]
class = StreamHandler
args = (sys.stdout,)
level = NOTSET
formatter = generic

[formatter_generic]
format = %(levelname)-5.5s [% (name)s] %(message)s

```

migrations/env.py

```

from __future__ import annotations
import asyncio
from logging.config import fileConfig
from sqlalchemy import pool
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import AsyncEngine, create_async_engine
from alembic import context
import os

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

DB_URL = os.getenv("DATABASE_URL", config.get_main_option("sqlalchemy.url"))

def run_migrations_offline():
    context.configure(
        url=DB_URL,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online():
    connectable = create_async_engine(DB_URL, poolclass=pool.NullPool)
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)
    await connectable.dispose()

```

```

def do_run_migrations(connection: Connection):
    context.configure(connection=connection)
    with context.begin_transaction():
        context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())

```

migrations/script.py.mako (stock Alembic template)

```

"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}
"""
from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision = ${repr(up_revision)}
down_revision = ${repr(down_revision)}
branch_labels = ${repr(branch_labels)}
depends_on = ${repr(depends_on)}

def upgrade() -> None:
    pass

def downgrade() -> None:
    pass

```

migrations/versions/0001_init_schema.py

Paste **all DDL** from the “maximum production-ready SQL starter” you approved (Sections 0–15), wrapped in Alembic’s `op.execute("""...""")`. Example shell:

```

from alembic import op
import sqlalchemy as sa

revision = "0001_init_schema"
down_revision = None
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("""
-- PASTE: extensions, domains, enums, schemas, functions, tables, views,
-- triggers, indexes, materialized views, procedures, etc.
-- This is the entire SQL starter you have, in order.
-- Example:
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS pgcrypto;

```

```
-- ...
-- (keep full body here)
"""

def downgrade() -> None:
    op.execute("""
-- Drop in reverse dependency order (optional). In practice, you'll add
-- real downgrades as needed. For now, keep as placeholder or controlled
teardown.
""")
```

Run:

```
alembic upgrade head
```

3) Seed script (realistic flows)

New file

```
erp-fastapi/scripts/seed.py
```

scripts/seed.py

```
import asyncio, os, uuid, random, datetime as dt
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy import text

DB_URL = os.getenv("DATABASE_URL",
"postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb")
ENTERPRISE_ID = os.getenv("DEFAULT_ENTERPRISE_ID", "00000000-0000-0000-0000-000000000000")

async def run():
    engine = create_async_engine(DB_URL, echo=False)
    async with engine.begin() as conn:
        # Set tenant GUC
        await conn.execute(text("SET LOCAL erp.current_enterprise = :eid"),
{"eid": ENTERPRISE_ID})

        # Seed enterprise (if empty)
        await conn.execute(text("""
INSERT INTO enterprise(enterprise_id, code, name, base_currency)
VALUES (:eid, 'ENT-01', 'Demo Enterprise', 'ZMW')
ON CONFLICT (enterprise_id) DO NOTHING;
"""), {"eid": ENTERPRISE_ID})

        # UOM
        for u in [("EA", "Each"), ("KG", "Kilogram"), ("L", "Litre")]:
            await conn.execute(text("INSERT INTO uom(uom_code, description)
VALUES (:c,:d) ON CONFLICT DO NOTHING"), {"c":u[0], "d":u[1]})
```

```

        # Org: 2 tiers, 3 depts in tier 2
        await conn.execute(text("""
INSERT INTO org_unit(org_unit_id, enterprise_id, name, code, tier_level,
parent_id)
VALUES
    (:s1, :eid, 'Production','PROD',1,NULL)
ON CONFLICT DO NOTHING;
        """), {"s1":str(uuid.uuid4()),"eid":ENTERPRISE_ID})
        # lookup prod id
        prod = (await conn.execute(text("SELECT org_unit_id FROM org_unit
WHERE code='PROD' AND enterprise_id=:eid"), {"eid":ENTERPRISE_ID})).scalar()
        depts = []
        for code,name in [("HSE-A","House A"),("HSE-B","House
B"),("PLANT","Processing Plant")]:
            oid = str(uuid.uuid4())
            depts.append(oid)
            await conn.execute(text("""
INSERT INTO org_unit(org_unit_id, enterprise_id, name, code, tier_level,
parent_id)
VALUES (:id, :eid, :n, :c, 2, :p) ON CONFLICT DO NOTHING
        """), {"id":oid,"eid":ENTERPRISE_ID,"n":name,"c":code,"p":prod})

        # Items & policies
        items=[]
        for sk,nm in [("FEED-50KG","Broiler Feed 50kg"),("BIRD-LIVE","Live
Bird"),("MEAT-PCK","Packaged Chicken")]:
            iid=str(uuid.uuid4()); items.append((iid,sk))
            await conn.execute(text("""
INSERT INTO item(item_id, enterprise_id, name, sku, base_uom, category,
expiry_days, serialized)
VALUES (:id,:eid,:n,:sku,'EA','POULTRY',90,false)
ON CONFLICT DO NOTHING
        """), {"id":iid,"eid":ENTERPRISE_ID,"n":nm,"sku":sk})
            await conn.execute(text("""
INSERT INTO item_cost_policy(item_id, method, standard_cost)
VALUES (:id,'WAVG',NULL) ON CONFLICT DO NOTHING
        """), {"id":iid})

        # Partners
        sup = str(uuid.uuid4())
        cust = str(uuid.uuid4())
        await conn.execute(text("""
INSERT INTO partner(partner_id, enterprise_id, name, type, currency)
VALUES (:id,:eid,'Feed Supplier','SUPPLIER','ZMW') ON CONFLICT DO NOTHING
        """), {"id":sup,"eid":ENTERPRISE_ID})
        await conn.execute(text("""
INSERT INTO partner(partner_id, enterprise_id, name, type, currency)
VALUES (:id,:eid,'Retail Chain','CUSTOMER','ZMW') ON CONFLICT DO NOTHING
        """), {"id":cust,"eid":ENTERPRISE_ID})

        # Tax & GL map
        await conn.execute(text("""
INSERT INTO tax_code(enterprise_id, tax_code, description, rate, is_vat)
VALUES (:eid,'VAT16','VAT 16%',0.16,true)
ON CONFLICT DO NOTHING
        """), {"eid":ENTERPRISE_ID})
        for mv,dr,cr in [

```

```

        ("PURCHASE", "1400-Inventory", "2100-AP"),
        ("SALE", "1100-AR", "4000-Revenue"),
        ("SALES_RETURN", "4001-SalesReturns", "1101-ARContra"),
        ("TRANSFER", "1400-Inventory", "1400-Inventory"),
        ("ISSUE_TO_WO", "1500-WIP", "1400-Inventory"),
        ("RECEIPT_FROM_WO", "1400-Inventory", "1500-WIP"),
        ("ADJUSTMENT", "5790-StockAdjust", "1400-Inventory")
    ]:
        await conn.execute(text("""
INSERT INTO gl_map(enterprise_id, movement, debit_acct, credit_acct, active)
VALUES (:eid,:mv,:dr,:cr,true)
ON CONFLICT (enterprise_id, movement) DO UPDATE SET
debit_acct=EXCLUDED.debit_acct, credit_acct=EXCLUDED.credit_acct, active=true
"""), {"eid":ENTERPRISE_ID,"mv":mv,"dr":dr,"cr":cr})

        # Doc headers
        today = dt.date.today()
        grn_id=str(uuid.uuid4())
        inv_id=str(uuid.uuid4())
        await conn.execute(text("""
INSERT INTO doc_header(doc_id, enterprise_id, doc_no, dtype, doc_date,
partner_id, currency, status)
VALUES (:id,:eid,'GRN-0001','GRN',:d,:pid,'ZMW','POSTED')
"""), {"id":grn_id,"eid":ENTERPRISE_ID,"d":today,"pid":sup})
        await conn.execute(text("""
INSERT INTO doc_header(doc_id, enterprise_id, doc_no, dtype, doc_date,
partner_id, currency, status)
VALUES (:id,:eid,'INV-0001','INV',:d,:pid,'ZMW','POSTED')
"""), {"id":inv_id,"eid":ENTERPRISE_ID,"d":today,"pid":cust})

        # Batches
        feed_item = items[0][0]
        live_item = items[1][0]
        meat_item = items[2][0]

        feed_batch=str(uuid.uuid4())
        bird_batch=str(uuid.uuid4())
        meat_batch=str(uuid.uuid4())

        await conn.execute(text("""
INSERT INTO batch(batch_id, enterprise_id, item_id, batch_no, mfg_date,
expiry_date)
VALUES (:id,:eid,:it,'FEEDLOT-01',:mfg,:exp)
"""), {"id":feed_batch,"eid":ENTERPRISE_ID,"it":feed_item,"mfg":today-
dt.timedelta(days=5),"exp":today+dt.timedelta(days=85)})

        await conn.execute(text("""
INSERT INTO batch(batch_id, enterprise_id, item_id, batch_no, mfg_date)
VALUES (:id,:eid,:it,'BATCH-BIRD-01',:mfg)
"""), {"id":bird_batch,"eid":ENTERPRISE_ID,"it":live_item,"mfg":today})

        await conn.execute(text("""
INSERT INTO batch(batch_id, enterprise_id, item_id, batch_no, mfg_date,
expiry_date)
VALUES (:id,:eid,:it,'MEAT-PACK-01',:mfg,:exp)

```

```

"""),
{"id":meat_batch,"eid":ENTERPRISE_ID,"it":meat_item,"mfg":today,"exp":today+d
t.timedelta(days=7)})

    # Stock movements:
    dept_house_a, dept_house_b, dept_plant = depts

    # Purchase FEED into House A
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, doc_id, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'PURCHASE',
    1000, 50000, :doc,
'{"duty":"3000","freight":"1000","insurance":"500","other_lc":"200"}')
"""), {"d":today, "ou":dept_house_a, "it":feed_item, "b":feed_batch,
"doc":grn_id})

    # Vertical transfer: Live birds from House A -> Plant (simulate after
growth)
    # issue feed to WO, receive birds from WO (simplified)
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'ISSUE_TO_WO',
    -800, -40000, '{"wo_no":"WO-BIRDS-01"}')
"""), {"d":today, "ou":dept_house_a, "it":feed_item, "b":feed_batch})
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'RECEIPT_FROM_WO',
    500, 60000, '{"wo_no":"WO-BIRDS-01"}')
"""), {"d":today, "ou":dept_house_a, "it":live_item, "b":bird_batch})

    # Vertical TRANSFER birds to PLANT
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, source_org_unit_id, dest_org_unit_id)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'TRANSFER',
    -500, -60000, :src, :dst)
"""), {"d":today, "ou":dept_house_a, "it":live_item, "b":bird_batch,
"src":dept_house_a, "dst":dept_plant})
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, source_org_unit_id, dest_org_unit_id)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'TRANSFER',
    500, 60000, :src, :dst)
"""), {"d":today, "ou":dept_plant, "it":live_item, "b":bird_batch,
"src":dept_house_a, "dst":dept_plant})

    # Process at PLANT: ISSUE birds to WO, RECEIVE meat packs
    await conn.execute(text("""

```

```

INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'ISSUE_TO_WO',
    -500, -60000, '{"wo_no":"WO-MEAT-01"}')
"""), {"d":today, "ou":dept_plant, "it":live_item, "b":bird_batch})
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'RECEIPT_FROM_WO',
    450, 90000, '{"wo_no":"WO-MEAT-01"}')
"""), {"d":today, "ou":dept_plant, "it":meat_item, "b":meat_batch})

    # Sale from PLANT
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, doc_id, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'SALE',
    -300, 180000, :inv, '{"vat_code":"VAT16"}')
"""), {"d":today, "ou":dept_plant, "it":meat_item, "b":meat_batch,
"inv":inv_id})

    # Optional: small abnormal adjustment at Plant
    await conn.execute(text("""
INSERT INTO stock_ledger(enterprise_id, txn_date, org_unit_id, item_id,
batch_id, movement,
    qty_units, value, meta)
VALUES (erp.current_enterprise_id(), :d, :ou, :it, :b, 'ADJUSTMENT',
    -5, -1000, '{"abnormal":true,"reason":"spoilage"}')
"""), {"d":today, "ou":dept_plant, "it":meat_item, "b":meat_batch})

    # Refresh MV
    await conn.execute(text("REFRESH MATERIALIZED VIEW CONCURRENTLY
mv_item_daily_delta"))

    await engine.dispose()
    print("✔ Seed complete")

if __name__ == "__main__":
    asyncio.run(run())

```

Run it

python scripts/seed.py

4) Audit Pack endpoint (ZIP all packs)

New file

app/routes/audit_pack.py

app/routes/audit_pack.py

```
import io, zipfile, json, datetime as dt
import pandas as pd
from fastapi import APIRouter, Depends, Query
from fastapi.responses import StreamingResponse
from sqlalchemy import text
from app.db import SessionLocal, set_enterprise_guc
from app.deps import get_enterprise_id

router = APIRouter(prefix="/reports", tags=["reports"])

SQL_FILES = {
    "opening": "app/sql/reports/opening.sql",
    "purchases_lc": "app/sql/reports/purchases_lc.sql",
    "transfers": "app/sql/reports/transfers.sql",
    "prod_io": "app/sql/reports/prod_io.sql",
    "sales_returns": "app/sql/reports/sales_returns.sql",
    "closing_wavg": "app/sql/reports/closing_wavg.sql",
    "abnormal": "app/sql/reports/abnormal.sql"
}

async def _set_scope(session, org_unit_ids):
    await session.execute(text("CREATE TEMP TABLE IF NOT EXISTS tmp_scope
(org_unit_id uuid primary key) ON COMMIT DROP"))
    await session.execute(text("TRUNCATE tmp_scope"))
    if org_unit_ids:
        await session.execute(text("INSERT INTO tmp_scope SELECT
UNNEST(:ids)::uuid"), {"ids": org_unit_ids})

async def _run_sql(session, path: str, params: dict):
    with open(path, "r", encoding="utf-8") as f:
        sql = f.read()
    res = await session.execute(text(sql), params)
    cols = list(res.keys())
    rows = [dict(zip(cols, r)) for r in res.fetchall()]
    return rows

@router.get("/audit-pack.zip")
async def audit_pack(
    frm: str,
    to: str,
    org_units: list[str] = Query(...),
    enterprise_id: str = Depends(get_enterprise_id),
):
    as_of = to
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as z:
        async with SessionLocal() as s:
            async with s.begin():
                await set_enterprise_guc(s, enterprise_id)
                await _set_scope(s, org_units)

                # Opening (as-of frm-1)
                opening = await _run_sql(s, SQL_FILES["opening"], {"from":
frm})
```

```

        purchases = await _run_sql(s, SQL_FILES["purchases_lc"],
{"from": frm, "to": to})
        transfers = await _run_sql(s, SQL_FILES["transfers"],
{"from": frm, "to": to})
        prod_io = await _run_sql(s, SQL_FILES["prod_io"], {"from":
frm, "to": to})
        sales = await _run_sql(s, SQL_FILES["sales_returns"],
{"from": frm, "to": to})
        closing = await _run_sql(s, SQL_FILES["closing_wavg"], {"to":
to})
        abnormal = await _run_sql(s, SQL_FILES["abnormal"], {"from":
frm, "to": to})

    # Write CSVs
    for name, data in [
        ("opening.csv", opening),
        ("purchases_lc.csv", purchases),
        ("transfers.csv", transfers),
        ("production_io.csv", prod_io),
        ("sales_returns.csv", sales),
        ("closing_wavg.csv", closing),
        ("abnormal.csv", abnormal),
    ]:
        df = pd.DataFrame(data)
        z.writestr(name, df.to_csv(index=False))

    # A JSON manifest (period, org_units, counts)
    manifest = {
        "enterprise_id": enterprise_id,
        "period": {"from": frm, "to": to, "as_of": as_of},
        "org_units": org_units,
        "counts": {
            "opening": len(opening),
            "purchases_lc": len(purchases),
            "transfers": len(transfers),
            "production_io": len(prod_io),
            "sales_returns": len(sales),
            "closing_wavg": len(closing),
            "abnormal": len(abnormal),
        },
        "notes": [
            "Same-tier consolidation rule applied in transfers:
include vertical, exclude horizontal.",
            "All values reported in enterprise base currency.",
        ],
    }
    z.writestr("manifest.json", json.dumps(manifest, indent=2))

    buf.seek(0)
    filename = f"audit_pack_{frm}_to_{to}.zip"
    return StreamingResponse(buf, media_type="application/zip",
headers={"Content-Disposition": f'attachment; filename="{filename}"'})

```

5) Wire routes & config

app/main.py (add router)

```
from app.routes import health, admin, ledger, reports, audit_pack
# ...
app.include_router(audit_pack.router)
```

6) Optional: tiny “create future partitions” admin endpoint

app/sql/admin/ensure_partitions.sql

```
CALL erp.ensure_future_partitions(6);
```

app/routes/admin.py (add)

```
@router.post("/partitions/future", dependencies=[Depends(require_api_key)])
async def ensure_partitions(enterprise_id: str = Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await s.execute(text("CALL erp.ensure_future_partitions(6)"))
    return {"status": "ok", "months_ahead": 6}
```

7) Quick runbook

```
# 1) Migrate
alembic upgrade head

# 2) Seed
python scripts/seed.py

# 3) Start API
uvicorn app.main:app --host 0.0.0.0 --port 8080

# 4) Audit pack (ZIP)
curl "http://localhost:8080/reports/audit-pack.zip?frm=2025-09-01&to=2025-09-30&org_units=<DEPT-UUID>&org_units=<DEPT2-UUID>" \
  -H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000" \
  -o audit_pack.zip
```

What you now have

- **Alembic:** one migration builds the entire schema (extensible).

- **Seed:** end-to-end flow across tiers: purchase → production → vertical transfer → processing → sale → abnormal.
- **Audit Pack:** zipped CSVs + manifest for the period; transfer report **always** respects the consolidation rule (**vertical included** / **horizontal excluded** at same tier).
- **Admin ops:** refresh MV, ensure partitions, GL post per ledger.

Here are the final add-ons:

- **Alembic autogenerate setup** (+ a short **migrations README** with best practices)
- New **GL-driven placeholders:** `/reports/financials` (P&L / BS / CF) and `/reports/ratios`
- An **Excel audit pack exporter:** `/reports/audit-pack.xlsx` (multi-sheet workbook)

Everything fits the stack you already have.

1) Alembic: enable autogenerate + README

You're SQL-first (raw DDL), so “autogenerate” won't detect objects unless we register *shadow metadata*. We'll keep autogenerate available for incremental tweaks, but retain **migration files as SQL** (recommended for DBAs).

1.1 `alembic.ini` — add options

```
[alembic]
script_location = migrations
sqlalchemy.url =
postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb
prepend_sys_path = .
revision_environment = true
```

1.2 `migrations/env.py` — wire a “shadow” `MetaData` (empty) and passthrough

```
from __future__ import annotations
import asyncio
from logging.config import fileConfig
from sqlalchemy import pool, MetaData
from sqlalchemy.engine import Connection
from sqlalchemy.ext.asyncio import AsyncEngine, create_async_engine
from alembic import context
import os

config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)
```

```

DB_URL = os.getenv("DATABASE_URL", config.get_main_option("sqlalchemy.url"))

# Shadow metadata: keep empty or register a few tables if you want autogen
diffs.
target_metadata = MetaData()

def run_migrations_offline():
    context.configure(
        url=DB_URL,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
        compare_type=True,
        compare_server_default=True,
    )
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online():
    connectable = create_async_engine(DB_URL, poolclass=pool.NullPool)
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)
    await connectable.dispose()

def do_run_migrations(connection: Connection):
    context.configure(
        connection=connection,
        target_metadata=target_metadata,
        compare_type=True,
        compare_server_default=True,
        render_as_batch=False,
    )
    with context.begin_transaction():
        context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())

```

1.3 Quick commands

```

# Create a new empty migration file you can fill with SQL
alembic revision -m "add xyz" # puts a new file in migrations/versions/

# Apply
alembic upgrade head

# Rollback 1 step
alembic downgrade -1

```

1.4 migrations/README.md (new)

```
# Migrations Best Practices

- We are SQL-first: put production DDL in migration `upgrade()` via
`op.execute("""...""")`.
- Always include:
  - explicit `CREATE INDEX` and `DROP INDEX`
  - trigger, function, and view definitions with `CREATE OR REPLACE`
  - partition creation procedures if needed
- Never break RLS: any new table must enable RLS and add tenant policy.
- For large MVs: prefer `REFRESH MATERIALIZED VIEW CONCURRENTLY`.
- Use `revision_environment = true` so env picks up DB URL from env vars.
- For test/dev: seed via `scripts/seed.py`, not migrations.

## Workflow

1. `alembic revision -m "feature"` to create a new stub.
2. Paste SQL into `upgrade()` (and an appropriate teardown into `downgrade()`
if safe).
3. `alembic upgrade head`
4. Run smoke reports; refresh MVs; verify `v_transfer_integrity` is empty.
```

2) GL-Driven placeholders: financials & ratios

Add very small **GL staging** so you can serve P&L/BS/CF and ratios quickly. These are placeholders that expect your accounting ETL to load monthly balances (or transactional postings) into a TB table.

2.1 SQL (new files under `app/sql/reports/`)

gl_trial_balance.sql (helper view materialization stub)

```
-- Expect a table or view populated by your GL:
-- gl_trial_balance(enterprise_id uuid, period date, account text, debit
dom_money, credit dom_money)
-- If you already have gl_posting -> accounts mapping, build TB from that in
the warehouse.
```

```
SELECT enterprise_id, period, account,
       SUM(debit) AS debit, SUM(credit) AS credit,
       SUM(debit - credit) AS net
FROM gl_trial_balance
WHERE enterprise_id = erp.current_enterprise_id()
  AND period BETWEEN :from AND :to
GROUP BY enterprise_id, period, account
ORDER BY period, account;
```

financials_pnl.sql

```
-- Map TB accounts to P&L lines with a mapping table (or a CASE).
-- Placeholder mapping table:
-- gl_to_fs_map(enterprise_id uuid, account text, fs_line text, fs_section
text)
-- fs_section: 'PNL','BS','CF'
```

```
WITH tb AS (
  SELECT period, m.fs_line, SUM(t.net) AS amount
  FROM gl_trial_balance t
  JOIN gl_to_fs_map m
    ON m.enterprise_id = t.enterprise_id AND m.account = t.account
  WHERE t.enterprise_id = exp.current_enterprise_id()
    AND m.fs_section = 'PNL'
    AND period BETWEEN :from AND :to
  GROUP BY period, m.fs_line
)
SELECT period, fs_line, amount
FROM tb
ORDER BY period, fs_line;
```

financials_bs.sql

```
WITH tb AS (
  SELECT period, m.fs_line, SUM(t.net) AS amount
  FROM gl_trial_balance t
  JOIN gl_to_fs_map m
    ON m.enterprise_id = t.enterprise_id AND m.account = t.account
  WHERE t.enterprise_id = exp.current_enterprise_id()
    AND m.fs_section = 'BS'
    AND period = :as_of::date
  GROUP BY period, m.fs_line
)
SELECT fs_line, amount
FROM tb
ORDER BY fs_line;
```

financials_cf_indirect.sql

```
-- Indirect CF: start from P&L Net Income + adjust for non-cash and working
capital deltas
-- Assumes you precompute this in warehouse or provide working capital views.
```

```
SELECT section, SUM(amount) AS amount
FROM cashflow_indirect_v
WHERE enterprise_id = exp.current_enterprise_id()
  AND period BETWEEN :from AND :to
GROUP BY section
ORDER BY section;
```

financials_ratios.sql

```
-- Ratios using derived financials; expect kpi_basis_v in warehouse
SELECT period,
  current_ratio,
  quick_ratio,
```

```

    debt_to_equity,
    gross_margin_pct,
    net_margin_pct,
    inventory_turns,
    dso_days,
    dpo_days,
    dih_days,
    cash_conversion_cycle
FROM kpi_basis_v
WHERE enterprise_id = erp.current_enterprise_id()
    AND period BETWEEN :from AND :to
ORDER BY period;

```

If you don't have `gl_trial_balance`, `gl_to_fs_map`, `cashflow_indirect_v`, or `kpi_basis_v` yet, add lightweight stubs pointing to your GL mart; the API will be ready to consume once they exist.

2.2 FastAPI routes (new endpoints)

Update `app/routes/reports.py` (append):

```

@router.get("/financials/pnl")
async def financials_pnl(frm: str, to: str, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            return await _run_sql(s, "app/sql/reports/financials_pnl.sql",
{"from": frm, "to": to})

@router.get("/financials/bs")
async def financials_bs(as_of: str, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            return await _run_sql(s, "app/sql/reports/financials_bs.sql",
{"as_of": as_of})

@router.get("/financials/cashflow")
async def financials_cf(frm: str, to: str, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            return await _run_sql(s,
"app/sql/reports/financials_cf_indirect.sql", {"from": frm, "to": to})

@router.get("/ratios")
async def financials_ratios(frm: str, to: str, enterprise_id: str =
Depends(get_enterprise_id)):
    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)

```

```
        return await _run_sql(s, "app/sql/reports/financials_ratios.sql",
{"from": frm, "to": to})
```

3) Excel Audit Pack exporter

New endpoint that mirrors the ZIP pack but returns a **single Excel workbook** (multiple sheets).
Uses `pandas.ExcelWriter(openpyxl)`.

3.1 New file `app/routes/audit_pack_xlsx.py`

```
import io, json
import pandas as pd
from fastapi import APIRouter, Depends, Query
from fastapi.responses import StreamingResponse
from sqlalchemy import text
from app.db import SessionLocal, set_enterprise_guc
from app.deps import get_enterprise_id

router = APIRouter(prefix="/reports", tags=["reports"])

SQL_FILES = {
    "opening": "app/sql/reports/opening.sql",
    "purchases_lc": "app/sql/reports/purchases_lc.sql",
    "transfers": "app/sql/reports/transfers.sql",
    "prod_io": "app/sql/reports/prod_io.sql",
    "sales_returns": "app/sql/reports/sales_returns.sql",
    "closing_wavg": "app/sql/reports/closing_wavg.sql",
    "abnormal": "app/sql/reports/abnormal.sql",
    # Optional GL sheets (will try, ignore if missing)
    "pnl": "app/sql/reports/financials_pnl.sql",
    "bs": "app/sql/reports/financials_bs.sql",
    "cashflow": "app/sql/reports/financials_cf_indirect.sql",
    "ratios": "app/sql/reports/financials_ratios.sql",
}

async def _set_scope(session, org_unit_ids):
    await session.execute(text("CREATE TEMP TABLE IF NOT EXISTS tmp_scope
(org_unit_id uuid primary key) ON COMMIT DROP"))
    await session.execute(text("TRUNCATE tmp_scope"))
    if org_unit_ids:
        await session.execute(text("INSERT INTO tmp_scope SELECT
UNNEST(:ids)::uuid", {"ids": org_unit_ids}))

async def _run_sql(session, path: str, params: dict):
    with open(path, "r", encoding="utf-8") as f:
        sql = f.read()
    try:
        res = await session.execute(text(sql), params)
        cols = list(res.keys())
        rows = [dict(zip(cols, r)) for r in res.fetchall()]
        return rows
    except Exception:
```

```

        # If optional GL sheets not yet set up, return None to skip
        return None

@router.get("/audit-pack.xlsx")
async def audit_pack_xlsx(
    frm: str,
    to: str,
    org_units: list[str] = Query(...),
    enterprise_id: str = Depends(get_enterprise_id),
):
    as_of = to
    buf = io.BytesIO()

    async with SessionLocal() as s:
        async with s.begin():
            await set_enterprise_guc(s, enterprise_id)
            await _set_scope(s, org_units)

            # Pull core sheets
            opening = await _run_sql(s, SQL_FILES["opening"], {"from": frm})
            purchases = await _run_sql(s, SQL_FILES["purchases_lc"], {"from":
frm, "to": to})
            transfers = await _run_sql(s, SQL_FILES["transfers"], {"from":
frm, "to": to})
            prod_io = await _run_sql(s, SQL_FILES["prod_io"], {"from": frm,
"to": to})
            sales = await _run_sql(s, SQL_FILES["sales_returns"], {"from":
frm, "to": to})
            closing = await _run_sql(s, SQL_FILES["closing_wavg"], {"to":
to})
            abnormal = await _run_sql(s, SQL_FILES["abnormal"], {"from": frm,
"to": to})

            # Optional GL sheets
            pnl = await _run_sql(s, SQL_FILES["pnl"], {"from": frm, "to":
to})

            bs = await _run_sql(s, SQL_FILES["bs"], {"as_of": to})
            cashflow = await _run_sql(s, SQL_FILES["cashflow"], {"from": frm,
"to": to})
            ratios = await _run_sql(s, SQL_FILES["ratios"], {"from": frm,
"to": to})

        with pd.ExcelWriter(buf, engine="openpyxl") as writer:
            def write_sheet(name, data):
                if data is None:
                    return
                df = pd.DataFrame(data)
                # Excel sheet name cap is 31 chars; keep names tight
                df.to_excel(writer, sheet_name=name[:31], index=False)

            write_sheet("Opening", opening)
            write_sheet("Purchases_LC", purchases)
            write_sheet("Transfers", transfers)
            write_sheet("Production_IO", prod_io)
            write_sheet("Sales_Returns", sales)
            write_sheet("Closing_WAVG", closing)
            write_sheet("Abnormal", abnormal)

```

```

# Optional financials
write_sheet("P&L", pnl)
write_sheet("Balance_Sheet", bs)
write_sheet("Cash_Flow", cashflow)
write_sheet("Ratios", ratios)

# Manifest sheet
manifest = {
    "enterprise_id": enterprise_id,
    "period": {"from": frm, "to": to, "as_of": as_of},
    "org_units": org_units,
    "notes": [
        "Same-tier consolidation: vertical transfers included,
horizontal transfers excluded.",
        "Inventory values in enterprise base currency.",
    ],
}
pd.DataFrame([manifest]).to_excel(writer, sheet_name="Manifest",
index=False)

buf.seek(0)
filename = f"audit_pack_{frm}_to_{to}.xlsx"
return StreamingResponse(
    buf,
    media_type="application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet",
    headers={"Content-Disposition": f'attachment;
filename="{filename}"'},
)

```

3.2 Wire the router

Add to app/main.py:

```

from app.routes import health, admin, ledger, reports, audit_pack,
audit_pack_xlsx
# ...
app.include_router(audit_pack_xlsx.router)

```

4) Minimal GL stubs (optional quick SQL to get you going)

If you don't have a GL warehouse yet, create simple placeholders so endpoints return something:

```

-- In your DB (temporary demo layer)
CREATE TABLE IF NOT EXISTS gl_trial_balance (
    enterprise_id uuid NOT NULL,
    period date NOT NULL,
    account text NOT NULL,

```

```

    debit numeric(20,6) NOT NULL DEFAULT 0,
    credit numeric(20,6) NOT NULL DEFAULT 0
);

CREATE TABLE IF NOT EXISTS gl_to_fs_map (
    enterprise_id uuid NOT NULL,
    account text NOT NULL,
    fs_line text NOT NULL,      -- e.g., 'Revenue', 'COGS', 'Inventory', 'Cash'
    fs_section text NOT NULL,   -- 'PNL', 'BS', 'CF'
    PRIMARY KEY (enterprise_id, account)
);

-- Cashflow & KPI basis quick stubs
CREATE VIEW IF NOT EXISTS cashflow_indirect_v AS
SELECT enterprise_id, period, section, SUM(amount) AS amount
FROM (
    SELECT enterprise_id, period, 'Operating'::text AS section, SUM(debit -
credit) AS amount
    FROM gl_trial_balance GROUP BY enterprise_id, period
) x
GROUP BY enterprise_id, period, section;

CREATE VIEW IF NOT EXISTS kpi_basis_v AS
SELECT
    enterprise_id, period,
    1.5::numeric AS current_ratio,
    1.2::numeric AS quick_ratio,
    0.6::numeric AS debt_to_equity,
    25.0::numeric AS gross_margin_pct,
    12.0::numeric AS net_margin_pct,
    8.0::numeric AS inventory_turns,
    35.0::numeric AS dso_days,
    30.0::numeric AS dpo_days,
    45.0::numeric AS dih_days,
    50.0::numeric AS cash_conversion_cycle
FROM (SELECT DISTINCT enterprise_id, period FROM gl_trial_balance) d;

```

Replace with your real GL pipeline as you wire it in.

5) Quick tests

- **P&L:**

```

curl "http://localhost:8080/reports/financials/pnl?frm=2025-09-01&to=2025-09-30" \
-H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000"

```

- **Ratios:**

```

curl "http://localhost:8080/reports/ratios?frm=2025-09-01&to=2025-09-30" \
-H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000"

```

- **Excel Audit Pack:**

```
curl "http://localhost:8080/reports/audit-pack.xlsx?frm=2025-09-01&to=2025-09-30&org_units=<DEPT-UUID>&org_units=<DEPT2-UUID>" \
-H "X-Enterprise-Id: 00000000-0000-0000-0000-000000000000" -o
audit_pack.xlsx
```

Done ✓

You now have:

- Alembic with both **SQL-first migrations** and **autogenerate scaffolding**
- **GL financials & ratios** endpoints ready to plug into your accounting mart
- A polished **Excel audit pack** (multi-sheet), alongside the existing **ZIP pack**
- All reports continue to respect your consolidation rule (**same-tier: include vertical, exclude horizontal**)

Here's the add-on bundle:

- **Pre-commit** (SQL linting + Python/JSON/YAML formatting)
- **Load testing** (k6 + Locust)
- **Observability** (Prometheus + Grafana + exporters) wired to your stack

Everything slots into the project you already have.

1) Pre-commit (SQL linting + formatting)

1.1 Add files

.pre-commit-config.yaml

```
repos:
  # 1) Basic hygiene
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
      - id: check-yaml
      - id: check-json
      - id: end-of-file-fixer
      - id: mixed-line-ending
      - id: trailing-whitespace

  # 2) Python formatting
  - repo: https://github.com/psf/black
```

```

rev: 24.8.0
hooks:
  - id: black
    language_version: python3.11

- repo: https://github.com/PyCQA/isort
  rev: 5.13.2
  hooks:
    - id: isort
      args: ["--profile", "black"]

# 3) SQL linting (PostgreSQL)
- repo: https://github.com/sqlfluff/sqlfluff
  rev: 3.1.0
  hooks:
    - id: sqlfluff-lint
      args: ["--dialect", "postgres"]
      files: ^(app/sql/|migrations/|.*\.sql$)
    - id: sqlfluff-fix
      args: ["--dialect", "postgres", "--exclude-rules", "L031"] # allow
implicit aliasing sometimes
      files: ^(app/sql/|migrations/|.*\.sql$)

# 4) YAML lint (infra files)
- repo: https://github.com/adrienverge/yamllint
  rev: v1.35.1
  hooks:
    - id: yamllint
      files: ^(docker-compose\.yaml|.*\.yaml|.*\.yml)$

```

.sqlfluff

```

[sqlfluff]
dialect = postgres
exclude_rules = L031,L034 # aliasing / select wildcards where acceptable in
views

[sqlfluff:rules]
max_line_length = 120
indent_unit = space
tab_space_size = 2

[sqlfluff:rules:layout.spacing]
closing_parenthesis_newline = true

```

1.2 Install & use

```

pip install pre-commit sqlfluff
pre-commit install # installs git hook
pre-commit run --all-files # run once on the repo

```

2) Load testing

Two options: **k6** (JS, easy in CI) and **Locust** (Python, rich).

2.1 k6

load/k6/basic-smoke.js

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: __ENV.VUS ? parseInt(__ENV.VUS) : 10,
  duration: __ENV.DURATION || '1m',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<800'],
  },
};

const BASE = __ENV.BASE || 'http://localhost:8080';
const EID = __ENV.EID || '00000000-0000-0000-0000-000000000000';
const API_KEY = __ENV.API_KEY || 'dev-key';
const OU = (__ENV.OU || '').split(','); // e.g. OU=uuid1,uuid2

export default function () {
  const params = { headers: { 'X-Enterprise-Id': EID, 'X-API-Key': API_KEY } };

  // Health
  let res = http.get(`${BASE}/health`, params);
  check(res, { 'health 200': (r) => r.status === 200 });

  // Opening
  res = http.get(`${BASE}/reports/opening?as_of=2025-09-30&org_units=${OU.map(encodeURIComponent).join('&org_units=')}`, params);
  check(res, { 'opening 200': (r) => r.status === 200 });

  // Transfers (consolidation rule path)
  res = http.get(`${BASE}/reports/transfers?frm=2025-09-01&to=2025-09-30&org_units=${OU.map(encodeURIComponent).join('&org_units=')}`, params);
  check(res, { 'transfers 200': (r) => r.status === 200 });

  // Closing
  res = http.get(`${BASE}/reports/closing?as_of=2025-09-30&org_units=${OU.map(encodeURIComponent).join('&org_units=')}`, params);
  check(res, { 'closing 200': (r) => r.status === 200 });

  sleep(1);
}
```

Run

```
docker run --rm -i loadimpact/k6 run - < load/k6/basic-smoke.js \
-e BASE=http://host.docker.internal:8080 \
-e EID=00000000-0000-0000-0000-000000000000 \
```

```
-e API_KEY=dev-key \  
-e OU=<DEPT-UUID>,<DEPT2-UUID> \  
-e VUS=20 -e DURATION=2m
```

2.2 Locust

load/locustfile.py

```
from locust import HttpUser, task, between  
import os, urllib.parse  
  
BASE = os.getenv("BASE", "http://localhost:8080")  
EID = os.getenv("EID", "00000000-0000-0000-0000-000000000000")  
API_KEY = os.getenv("API_KEY", "dev-key")  
OU = os.getenv("OU", "").split(",")  
  
hdrs = {"X-Enterprise-Id": EID, "X-API-Key": API_KEY}  
  
def qs(params):  
    return "&".join([f"{k}={urllib.parse.quote_plus(v)}" for k, v in params])  
  
class ERPUser(HttpUser):  
    wait_time = between(1, 3)  
  
    @task(2)  
    def opening(self):  
        ou = "&".join([f"org_units={u}" for u in OU])  
        self.client.get(f"/reports/opening?as_of=2025-09-30&{ou}",  
headers=hdrs)  
  
    @task(2)  
    def transfers(self):  
        ou = "&".join([f"org_units={u}" for u in OU])  
        self.client.get(f"/reports/transfers?frm=2025-09-01&to=2025-09-30&{ou}", headers=hdrs)  
  
    @task(1)  
    def closing(self):  
        ou = "&".join([f"org_units={u}" for u in OU])  
        self.client.get(f"/reports/closing?as_of=2025-09-30&{ou}",  
headers=hdrs)  
  
# Run:  
# locust -f load/locustfile.py --host=http://localhost:8080
```

3) Observability (Prometheus + Grafana)

We'll:

- instrument FastAPI with **Prometheus metrics**
- add **postgres_exporter** for DB

- add **node_exporter** + **cadvisor** for host/container metrics
- wire **Prometheus** and **Grafana** via a docker-compose **profile** observability

3.1 Instrument FastAPI

requirements.txt (append)

```
prometheus-fastapi-instrumentator==7.0.0
```

app/main.py (add after app creation)

```
from prometheus_fastapi_instrumentator import Instrumentator

def create_app():
    # ...
    app = FastAPI(title="ERP Reporting API", version="1.0.0")
    # routers...
    Instrumentator().instrument(app).expose(app, endpoint="/metrics",
    include_in_schema=False)
    return app
```

Now `/metrics` emits Prometheus metrics.

3.2 Prometheus & Grafana config

monitoring/prometheus.yml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'fastapi'
    metrics_path: /metrics
    static_configs:
      - targets: ['api:8080']    # service name from compose

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

monitoring/datasources.yml

```

apiVersion: 1
datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true

```

monitoring/dashboards.yml

```

apiVersion: 1
providers:
- name: Default
  orgId: 1
  folder: ERP
  type: file
  disableDeletion: false
  updateIntervalSeconds: 10
  options:
    path: /var/lib/grafana/dashboards

```

monitoring/dashboards/fastapi-overview.json (minimal starter)

```

{
  "title": "FastAPI Overview",
  "editable": true,
  "panels": [
    {
      "type": "graph",
      "title": "Requests p50/p95",
      "targets": [
        { "expr": "histogram_quantile(0.5, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))" },
        { "expr": "histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))" }
      ]
    },
    {
      "type": "graph",
      "title": "Requests per second",
      "targets": [
        { "expr": "sum(rate(http_requests_total[1m]))" }
      ]
    }
  ],
  "schemaVersion": 39
}

```

(You can also import off-the-shelf dashboards later: Node Exporter, cAdvisor, Postgres Exporter.)

3.3 Docker Compose (observability profile)

Append to your existing `docker-compose.yml`:

```
services:
  # --- existing services db, api ...

  postgres-exporter:
    image: prometheuscommunity/postgres-exporter:v0.15.0
    profiles: ["observability"]
    environment:
      DATA_SOURCE_NAME:
postgres://erp_user:strongpass@db:5432/erpdb?sslmode=disable
    depends_on: ["db"]
    networks: ["default"]
    ports:
      - "9187:9187"

  node-exporter:
    image: prom/node-exporter:v1.8.2
    profiles: ["observability"]
    pid: "host"
    network_mode: "host"
    command:
      - '--path.rootfs=/host'
    volumes:
      - '/:/host:ro,rslave'

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:v0.47.2
    profiles: ["observability"]
    ports:
      - "8081:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker:/var/lib/docker:ro

  prometheus:
    image: prom/prometheus:v2.54.1
    profiles: ["observability"]
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    ports:
      - "9090:9090"
    depends_on: ["api", "postgres-exporter", "node-exporter", "cadvisor"]

  grafana:
    image: grafana/grafana:11.1.3
    profiles: ["observability"]
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
      GF_USERS_DEFAULT_THEME: light
    ports:
      - "3000:3000"
    volumes:
```

```

-
./monitoring/datasources.yml:/etc/grafana/provisioning/datasources/datasources.yml:ro
-
./monitoring/dashboards.yml:/etc/grafana/provisioning/dashboards/dashboards.yml:ro
- ./monitoring/dashboards:/var/lib/grafana/dashboards:ro
  depends_on: ["prometheus"]

networks:
  default:
    driver: bridge

```

Bring up observability stack

```

docker compose --profile observability up -d
# Grafana: http://localhost:3000 (admin/admin)
# Prometheus: http://localhost:9090
# cAdvisor: http://localhost:8081

```

4) Small API hardening for metrics & health (optional)

- Rate-limit or firewall `/metrics` if exposing publicly.
- Add a simple 200 “readiness” check that pings DB.

app/routes/health.py (add a DB ping)

```

from sqlalchemy import text
from app.db import SessionLocal

@router.get("/ready")
async def ready():
    try:
        async with SessionLocal() as s:
            await s.execute(text("SELECT 1"))
            return {"ready": True}
    except Exception as e:
        return {"ready": False, "error": str(e)}

```

Prometheus can then use `/ready` for blackbox if needed.

5) CI suggestions (optional)

- Run **pre-commit** in CI:

```
pre-commit run --all-files
```

- Run **k6 smoke** on every PR (short duration, 2–3 VUs) to catch obvious regressions.
 - Exporter dashboards: import official JSONs into Grafana for deeper DB/host views.
-

What you have now

- **Pre-commit:** consistent SQL/Python/YAML quality gate.
- **Load tests:** k6 + Locust, parameterized for tenant header + org unit scope.
- **Observability:** Prometheus scrapes FastAPI `/metrics`, Postgres exporter, node/cAdvisor; Grafana pre-provisioned.

All of this continues to respect your core consolidation logic (same-tier consolidation: **vertical transfers included, horizontal transfers excluded**) because the logic lives in your SQL views and report queries the load tests call.

Here are the **Makefile** and **GitHub Actions** you asked for, wired to everything we built (migrations, seed, reports, pre-commit, load tests, observability hooks). Copy-paste into your repo and tweak IDs as needed.

Makefile

```
# ==== Config ====
PY=python
UV=uvicorn app.main:app --host 0.0.0.0 --port 8080
DB?=postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb
EID?=00000000-0000-0000-0000-000000000000
OU?=
API_KEY?=dev-key

export DATABASE_URL=$(DB)
export DEFAULT_ENTERPRISE_ID=$(EID)
export API_KEY

# ==== Dev env ====
.PHONY: venv install precommit
venv:
    $(PY) -m venv .venv
    . .venv/bin/activate; pip install -U pip

install:
    pip install -r requirements.txt
    pre-commit install

precommit:
    pre-commit run --all-files

# ==== DB / Migrations ====
.PHONY: mig-init mig-new migrate downgrade seed refresh mv partitions
mig-init:
    alembic init migrations

mig-new:
    @[ "$$(m)" ] || (echo 'Usage: make mig-new m="message"; exit 1)
    alembic revision -m "$$(m)"

migrate:
    alembic upgrade head

downgrade:
    @[ "$$(rev)" ] || (echo 'Usage: make downgrade rev="base|-1|<rev>";
exit 1)
    alembic downgrade $$(rev)

seed:
    $(PY) scripts/seed.py
```

```

refresh:
    curl -s -X POST http://localhost:8080/admin/refresh/daily-delta \
        -H "X-Enterprise-Id: $(EID)" -H "X-API-Key: $(API_KEY)"

partitions:
    curl -s -X POST http://localhost:8080/admin/partitions/future \
        -H "X-Enterprise-Id: $(EID)" -H "X-API-Key: $(API_KEY)"

# ==== API ====
.PHONY: run
run:
    $(UV)

# ==== Reports (quick curls) ====
.PHONY: report-opening report-transfers report-closing audit-zip audit-xlsx
report-opening:
    @[ "$(as_of)" ] || (echo 'Usage: make report-opening as_of=YYYY-MM-DD'; exit 1)
    curl -s "http://localhost:8080/reports/opening?as_of=$(as_of)$(foreach u,$(OU),&org_units=$(u))" \
        -H "X-Enterprise-Id: $(EID)" | jq .

report-transfers:
    @[ "$(from)" ] && [ "$(to)" ] || (echo 'Usage: make report-transfers from=YYYY-MM-DD to=YYYY-MM-DD'; exit 1)
    curl -s
    "http://localhost:8080/reports/transfers?frm=$(from)&to=$(to)$(foreach u,$(OU),&org_units=$(u))" \
        -H "X-Enterprise-Id: $(EID)" | jq .

report-closing:
    @[ "$(as_of)" ] || (echo 'Usage: make report-closing as_of=YYYY-MM-DD'; exit 1)
    curl -s "http://localhost:8080/reports/closing?as_of=$(as_of)$(foreach u,$(OU),&org_units=$(u))" \
        -H "X-Enterprise-Id: $(EID)" | jq .

audit-zip:
    @[ "$(from)" ] && [ "$(to)" ] || (echo 'Usage: make audit-zip from=YYYY-MM-DD to=YYYY-MM-DD'; exit 1)
    curl -s "http://localhost:8080/reports/audit-pack.zip?frm=$(from)&to=$(to)$(foreach u,$(OU),&org_units=$(u))" \
        -H "X-Enterprise-Id: $(EID)" -o audit_pack_$(from)_$(to).zip && ls -lh audit_pack_$(from)_$(to).zip

audit-xlsx:
    @[ "$(from)" ] && [ "$(to)" ] || (echo 'Usage: make audit-xlsx from=YYYY-MM-DD to=YYYY-MM-DD'; exit 1)
    curl -s "http://localhost:8080/reports/audit-pack.xlsx?frm=$(from)&to=$(to)$(foreach u,$(OU),&org_units=$(u))" \
        -H "X-Enterprise-Id: $(EID)" -o audit_pack_$(from)_$(to).xlsx && ls -lh audit_pack_$(from)_$(to).xlsx

# ==== Observability stack ====
.PHONY: obs-up obs-down
obs-up:
    docker compose --profile observability up -d

```

```

obs-down:
    docker compose --profile observability down

# ==== Load tests ====
.PHONY: k6 locust
k6:
    docker run --rm -i loadimpact/k6 run - < load/k6/basic-smoke.js \
        -e BASE=http://host.docker.internal:8080 \
        -e EID=$(EID) -e API_KEY=$(API_KEY) -e OU=$(OU) \
        -e VUS=10 -e DURATION=1m

locust:
    locust -f load/locustfile.py --headless -u 20 -r 5 --run-time 1m \
        --host=http://localhost:8080 \
        --env BASE=http://localhost:8080 EID=$(EID) API_KEY=$(API_KEY)
OU=$(OU)

```

Examples:

- make migrate seed run
- make report-transfers from=2025-09-01 to=2025-09-30
OU=<DEPT1>,<DEPT2>
- make obs-up then open Grafana <http://localhost:3000> (admin/admin)



`.github/workflows/ci.yml`

CI runs on PRs/pushes:

- Sets up Python
- Spins up Postgres service
- Runs Alembic **migrate**, seeds demo data
- Hits a few **report endpoints** (FastAPI on ephemeral port)
- Runs **pre-commit** (lint/format)
- Runs a short **k6** smoke (optional, docker-in-docker)
- Uploads **audit pack** as artifact

```

name: ci

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build-test:
    runs-on: ubuntu-latest
    services:

```

```

postgres:
  image: postgres:15
  env:
    POSTGRES_USER: erp_user
    POSTGRES_PASSWORD: strongpass
    POSTGRES_DB: erpdb
  ports: ["5432:5432"]
  options: >-
    --health-cmd="pg_isready -U erp_user"
    --health-interval=5s
    --health-timeout=5s
    --health-retries=10

env:
  DATABASE_URL:
postgresql+asyncpg://erp_user:strongpass@localhost:5432/erpdb
  DEFAULT_ENTERPRISE_ID: 00000000-0000-0000-0000-000000000000
  API_KEY: ci-key

steps:
  - uses: actions/checkout@v4

  - name: Set up Python
    uses: actions/setup-python@v5
    with:
      python-version: "3.11"

  - name: Cache pip
    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }
      restore-keys: |
        ${ runner.os }-pip-

  - name: Install deps
    run: |
      pip install -r requirements.txt
      pip install pre-commit sqlfluff

  - name: Migrate DB
    run: alembic upgrade head

  - name: Seed demo data
    run: python scripts/seed.py

  - name: Start API
    run: |
      nohup uvicorn app.main:app --host 0.0.0.0 --port 8080 >/tmp/api.log
2>&1 &
      sleep 5
      curl -s http://localhost:8080/health

  - name: Smoke reports (curl)
    run: |
      # get any org_unit from seed

```

```

        OU=$(psql "postgresql://erp_user:strongpass@localhost:5432/erpdb" -
At -c "SELECT org_unit_id FROM org_unit LIMIT 2" | tr '\n' ',' | sed
's/,,$//')
        echo "Using OU=${OU}"
        curl -s "http://localhost:8080/reports/opening?as_of=2025-09-
30&org_units=${OU//,/}&org_units=}" \
        -H "X-Enterprise-Id: ${DEFAULT_ENTERPRISE_ID}" | head
        curl -s "http://localhost:8080/reports/transfers?frm=2025-09-
01&to=2025-09-30&org_units=${OU//,/}&org_units=}" \
        -H "X-Enterprise-Id: ${DEFAULT_ENTERPRISE_ID}" | head

- name: Generate audit pack (xlsx)
  run: |
    OU=$(psql "postgresql://erp_user:strongpass@localhost:5432/erpdb" -
At -c "SELECT org_unit_id FROM org_unit LIMIT 2" | tr '\n' ',' | sed
's/,,$//')
    curl -s "http://localhost:8080/reports/audit-pack.xlsx?frm=2025-09-
01&to=2025-09-30&org_units=${OU//,/}&org_units=}" \
    -H "X-Enterprise-Id: ${DEFAULT_ENTERPRISE_ID}" -o audit_pack.xlsx
    ls -lh audit_pack.xlsx

- name: Upload artifact
  uses: actions/upload-artifact@v4
  with:
    name: audit-pack-xlsx
    path: audit_pack.xlsx

- name: Pre-commit
  run: pre-commit run --all-files

k6-smoke:
  runs-on: ubuntu-latest
  needs: build-test
  if: github.event_name == 'pull_request' || github.ref ==
'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Install k6
      run: |
        sudo apt-get update
        sudo apt-get install -y ca-certificates gnupg
        curl -s https://dl.k6.io/key.gpg | sudo gpg --dearmor -o
/usr/share/keyrings/k6-archive-keyring.gpg
        echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg]
https://dl.k6.io/deb stable main" | sudo tee /etc/apt/sources.list.d/k6.list
        sudo apt-get update && sudo apt-get install -y k6
    - name: Start API & DB (compose)
      run: |
        docker compose up -d db
        sleep 5
        pip install -r requirements.txt
        alembic upgrade head
        python scripts/seed.py
        nohup uvicorn app.main:app --host 0.0.0.0 --port 8080 >/tmp/api.log
2>&1 &
        sleep 5
    - name: Run k6 smoke

```

```
env:
  BASE: http://localhost:8080
  EID: 00000000-0000-0000-0000-000000000000
  API_KEY: ci-key
run: |
  OU=$(psql "postgresql://erp_user:strongpass@localhost:5432/erpdb" -
At -c "SELECT org_unit_id FROM org_unit LIMIT 2" | tr '\n' ',' | sed
's/,,$//')
  VUS=10 DURATION=30s OU=$OU BASE=$BASE EID=$EID API_KEY=$API_KEY k6
run load/k6/basic-smoke.js
```

Notes:

- CI uses a Postgres **service** and local FastAPI.
- We fetch 2 org units from seed to exercise **same-tier consolidation** on transfers.
- The k6 job is separate for clarity; you can fold it into the first job if preferred.

That's it ✓

You now have:

- A **Makefile** with one-liners for migrations, seeds, running, reports, audit packs, observability, and load tests.
- A **CI pipeline** that lints, migrates, seeds, smoke-tests core endpoints, generates an **Excel audit pack** artifact, and runs a brief **k6** load.

If you want, I can also add a **Make target** to build & push Docker images with tags from git SHA, and a **staging deploy** example (Compose or Helm).

You said:

Add

ChatGPT said:

Awesome — here are turnkey additions for:

- Docker image build & push (Makefile + Dockerfile)
- Staging deploy with **Compose or Helm** (your pick)
- CI workflow to build/push the image and (optionally) deploy to staging

Everything reuses the app you already have.

1) Docker image

Dockerfile

```
# syntax=docker/dockerfile:1.7
FROM python:3.11-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1
WORKDIR /app

# System deps
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential curl tini && rm -rf /var/lib/apt/lists/*

# Python deps first (better cache)
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt

# App
COPY app app
COPY app/sql app/sql
COPY scripts scripts
COPY alembic.ini alembic.ini
COPY migrations migrations

# Non-root
RUN useradd -m appuser
USER appuser

EXPOSE 8080
ENTRYPOINT ["/usr/bin/tini", "--"]
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

2) Makefile — build, push, run, helm

Append these targets to your existing **Makefile**:

```
# ==== Docker ====
IMG?=ghcr.io/your-org/erp-reporting
TAG?=$(shell git rev-parse --short=12 HEAD)

docker-build:
    docker build -t $(IMG):$(TAG) -t $(IMG):latest .

docker-push:
    docker push $(IMG):$(TAG)
    docker push $(IMG):latest

docker-run:
    docker run --rm -p 8080:8080 \
        -e DATABASE_URL="$(DB)" \
```

```

        -e DEFAULT_ENTERPRISE_ID="$ (EID) " \
        -e API_KEY="$ (API_KEY) " \
        $ (IMG):$ (TAG)

# ==== Helm (staging) ====
NS?=erp-staging
REL?=erp-reporting
KCTX?=your-kube-context

helm-install:
    helm upgrade --install $ (REL) deploy/helm/erp-reporting \
        --namespace $ (NS) --create-namespace \
        --set image.repository=$ (IMG) \
        --set image.tag=$ (TAG) \
        --set env.DATABASE_URL="$ (DB) " \
        --set env.DEFAULT_ENTERPRISE_ID="$ (EID) " \
        --set env.API_KEY="$ (API_KEY) "

helm-uninstall:
    helm uninstall $ (REL) -n $ (NS)

helm-status:
    kubectl --context $ (KCTX) -n $ (NS) get pods,svc,ingress

```

3) Staging with Compose (option A)

docker-compose.staging.yml

```

version: "3.9"
services:
  api:
    image: ${IMAGE:-ghcr.io/your-org/erp-reporting:latest}
    environment:
      DATABASE_URL: ${DATABASE_URL}
      DEFAULT_ENTERPRISE_ID: ${DEFAULT_ENTERPRISE_ID}
      API_KEY: ${API_KEY}
    ports: ["8080:8080"]
    restart: unless-stopped

```

Run:

```

# one-time
export IMAGE=ghcr.io/your-org/erp-reporting:<tag>
export
DATABASE_URL=postgres+asyncpg://erp_user:strongpass@dbhost:5432/erpdb
export DEFAULT_ENTERPRISE_ID=00000000-0000-0000-0000-000000000000
export API_KEY=staging-key

docker compose -f docker-compose.staging.yml up -d

```

4) Staging with Helm (option B)

Repo layout:

```
deploy/helm/erp-reporting/  
├─ Chart.yaml  
├─ values.yaml  
└─ templates/  
    ├─ deployment.yaml  
    ├─ service.yaml  
    ├─ hpa.yaml  
    └─ ingress.yaml
```

deploy/helm/erp-reporting/Chart.yaml

```
apiVersion: v2  
name: erp-reporting  
description: ERP Reporting API  
type: application  
version: 0.1.0  
appVersion: "1.0.0"
```

deploy/helm/erp-reporting/values.yaml

```
replicaCount: 2  
  
image:  
  repository: ghcr.io/your-org/erp-reporting  
  tag: latest  
  pullPolicy: IfNotPresent  
  
service:  
  type: ClusterIP  
  port: 8080  
  
env:  
  DATABASE_URL: "" # set via --set or sealed secret  
  DEFAULT_ENTERPRISE_ID: "00000000-0000-0000-0000-000000000000"  
  API_KEY: "staging-key"  
  
resources:  
  limits:  
    cpu: "500m"  
    memory: "512Mi"  
  requests:  
    cpu: "200m"  
    memory: "256Mi"  
  
autoscaling:  
  enabled: true  
  minReplicas: 2  
  maxReplicas: 6  
  targetCPUUtilizationPercentage: 70
```

```

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: erp-staging.example.com
      paths:
        - path: /
          pathType: Prefix
  tls: [] # e.g., [{hosts: ["erp-staging.example.com"], secretName: erp-staging-tls}]

```

deploy/helm/erp-reporting/templates/deployment.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "erp-reporting.fullname" . }}
  labels: {{- include "erp-reporting.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels: {{- include "erp-reporting.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels: {{- include "erp-reporting.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 8080
          env:
            - name: DATABASE_URL
              value: "{{ .Values.env.DATABASE_URL }}"
            - name: DEFAULT_ENTERPRISE_ID
              value: "{{ .Values.env.DEFAULT_ENTERPRISE_ID }}"
            - name: API_KEY
              value: "{{ .Values.env.API_KEY }}"
          readinessProbe:
            httpGet:
              path: /health/ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 20
      resources: {{- toYaml .Values.resources | nindent 12 }}

```

deploy/helm/erp-reporting/templates/service.yaml

```

apiVersion: v1

```

```

kind: Service
metadata:
  name: {{ include "erp-reporting.fullname" . }}
spec:
  type: {{ .Values.service.type }}
  selector: {{- include "erp-reporting.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: 8080

```

deploy/helm/erp-reporting/templates/hpa.yaml

```

{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "erp-reporting.fullname" . }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "erp-reporting.fullname" . }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{
.Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}

```

deploy/helm/erp-reporting/templates/ingress.yaml

```

{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "erp-reporting.fullname" . }}
  annotations:
    kubernetes.io/ingress.class: {{ .Values.ingress.className | quote }}
spec:
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ .host | quote }}
      http:
        paths:
          {{- range .paths }}
          - path: {{ .path }}
            pathType: {{ .pathType }}
            backend:
              service:
                name: {{ include "erp-reporting.fullname" $ }}

```

```

        port:
          number: {{ $.Values.service.port }}
      {{- end }}
    {{- end }}
    {{- if .Values.ingress.tls }}
    tls: {{- toYaml .Values.ingress.tls | nindent 4 }}
    {{- end }}
  {{- end }}

```

(Optional) Add `_helpers.tpl` with `fullname`, `labels`, `selectorLabels`—or inline labels for brevity.

Deploy:

```

make docker-build docker-push TAG=$(git rev-parse --short=12 HEAD)
make helm-install TAG=$(git rev-parse --short=12 HEAD) \

```

```

DB='postgresql+asyncpg://erp_user:strongpass@postgres.default.svc:5432/erpdb' \
\
EID=00000000-0000-0000-0000-000000000000 \
API_KEY=staging-key

```

5) CI: Build & push image (+ optional Helm deploy)

Add a separate workflow or extend your existing `ci.yml`. Here's a focused **docker+deploy** workflow.

.github/workflows/docker-deploy.yml

```

name: docker-deploy

on:
  push:
    branches: [ main ]
    workflow_dispatch:

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: your-org/erp-reporting

jobs:
  docker:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - name: Log in to GHCR

```

```

    uses: docker/login-action@v3
    with:
      registry: ${ env.REGISTRY }
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }

- name: Extract metadata (tags, labels)
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
    tags: |
      type=sha,format=short
      type=ref,event=branch
      type=raw,value=latest,enable=${ is_default_branch }

- name: Build and push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ${ steps.meta.outputs.tags }
    labels: ${ steps.meta.outputs.labels }

deploy-staging:
  needs: docker
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4

    # Set kubeconfig (use OIDC or store kubeconfig as a secret)
    - name: Set up kubectl
      uses: azure/setup-kubectl@v4
      with:
        version: 'v1.30.0'

    - name: Write kubeconfig
      run: |
        mkdir -p $HOME/.kube
        echo "${ secrets.KUBE_CONFIG_STAGING }" > $HOME/.kube/config
        chmod 600 $HOME/.kube/config

    - name: Set Helm
      uses: azure/setup-helm@v4
      with:
        version: 'v3.14.4'

    - name: Deploy (Helm upgrade)
      env:
        IMAGE: ghcr.io/${ env.IMAGE_NAME }
        TAG: ${ github.sha }
      run: |
        helm upgrade --install erp-reporting deploy/helm/erp-reporting \
          --namespace erp-staging --create-namespace \
          --set image.repository=$IMAGE \
          --set image.tag=${ TAG::12 } \

```

```
--set env.DATABASE_URL="${{ secrets.STAGING_DATABASE_URL }}" \
--set env.DEFAULT_ENTERPRISE_ID="${{ secrets.STAGING_EID }}" \
--set env.API_KEY="${{ secrets.STAGING_API_KEY }}"
```

Secrets you need:

- KUBE_CONFIG_STAGING — kubeconfig for your staging cluster (or switch to OIDC-based auth)
 - STAGING_DATABASE_URL — e.g., postgresql+asyncpg://user:pass@host:5432/db
 - STAGING_EID — the enterprise UUID
 - STAGING_API_KEY — API key for staging
-

6) Quick sanity checklist

- Build & run locally:
- `make docker-build TAG=$(git rev-parse --short=12 HEAD)`
- `make docker-run TAG=$(git rev-parse --short=12 HEAD)`
- Staging via Compose (simple VM):
- `IMAGE=ghcr.io/your-org/erp-reporting:latest docker compose -f docker-compose.staging.yml up -d`
- Staging via Helm (cluster):
- `make helm-install`
- `make helm-status`

Everything continues to preserve your core consolidation rule (same-tier consolidation: **vertical transfers included, horizontal transfers excluded**) since it's embedded in the SQL views all these deploy paths ship.

Here are production-ready add-ons for:

- Cert-Manager TLS (Let's Encrypt) and HTTPS Ingress
- Sealed Secrets (encrypt DATABASE_URL/API_KEY, etc.)
- Canary rollouts with **Argo Rollouts** (Helm-switchable)

Everything snaps into your existing Helm chart (deploy/helm/erp-reporting), with safe defaults and copy-paste YAML.

1) Cert-Manager TLS + HTTPS Ingress

1.1 Install Cert-Manager (once per cluster)

```
kubectl apply -f https://github.com/cert-manager/cert-
manager/releases/download/v1.15.3/cert-manager.yaml
# wait for pods in cert-manager ns to be Ready
```

1.2 ClusterIssuer (Let's Encrypt HTTP-01)

Create `infra/cert-manager/clusterissuer-lets-encrypt-prod.yaml`:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    email: ops@example.com          # <-- change me
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: le-account-key
    solvers:
      - http01:
          ingress:
            class: nginx
```

Apply:

```
kubectl apply -f infra/cert-manager/clusterissuer-lets-encrypt-prod.yaml
```

1.3 Helm: enable TLS on your Ingress

Edit `deploy/helm/erp-reporting/values.yaml`:

```
ingress:
  enabled: true
  className: nginx
  hosts:
    - host: erp-staging.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: erp-staging-tls          # cert-manager will create/renew
it
      hosts:
        - erp-staging.example.com
  certManager:
    clusterIssuer: letsencrypt-prod      # <-- add this block
```

Patch `templates/ingress.yaml` to include annotations for cert-manager:

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```

metadata:
  name: {{ include "erp-reporting.fullname" . }}
  annotations:
    kubernetes.io/ingress.class: {{ .Values.ingress.className | quote }}
    {{- if .Values.ingress.certManager.clusterIssuer }}
    cert-manager.io/cluster-issuer: {{
.Values.ingress.certManager.clusterIssuer | quote }}
    {{- end }}
spec:
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ .host | quote }}
      http:
        paths:
          {{- range .paths }}
          - path: {{ .path }}
            pathType: {{ .pathType }}
            backend:
              service:
                name: {{ include "erp-reporting.fullname" $ }}
                port:
                  number: {{ $.Values.service.port }}
          {{- end }}
        {{- end }}
    {{- if .Values.ingress.tls }}
    tls: {{- toYaml .Values.ingress.tls | nindent 4 }}
    {{- end }}
  {{- end }}

```

That's it—certs will issue/auto-renew.

2) Sealed Secrets (encrypt env like DATABASE_URL/API_KEY)

2.1 Install Sealed-Secrets Controller (once)

```
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.27.2/controller.yaml
```

2.2 Helm support for loading a Secret

Add to values.yaml:

```

secret:
  enabled: true
  name: erp-reporting-secrets
  # when using SealedSecrets, this Secret is created by the controller; app
  references it.

```

```

    keys:
      DATABASE_URL: ""          # leave empty here; populated by
SealedSecret
      DEFAULT_ENTERPRISE_ID: "00000000-0000-0000-0000-000000000000"
      API_KEY: ""              # from SealedSecret

```

Update templates/deployment.yaml to **prefer Secret env (secure) over plain .Values.env:**

```

envFrom:
  - secretRef:
      name: {{ .Values.secret.name }}
env:
  # fallback for any not provided via secret (optional)
  - name: DEFAULT_ENTERPRISE_ID
    value: "{{ .Values.env.DEFAULT_ENTERPRISE_ID }}"

```

Note: remove the explicit env: for DATABASE_URL and API_KEY since they'll come from the Secret.

2.3 Create a SealedSecret (CLI)

First, create a **temporary** vanilla Secret locally (not committed):

```

kubectl create secret generic erp-reporting-secrets \
  --from-
literal=DATABASE_URL='postgresql+asyncpg://user:pass@host:5432/erpdb' \
  --from-literal=API_KEY='staging-key' \
  --from-literal=DEFAULT_ENTERPRISE_ID='00000000-0000-0000-0000-000000000000' \
  --dry-run=client -o yaml > plain-secret.yaml

```

Seal it:

```

kubeseal --format yaml < plain-secret.yaml > deploy/helm/erp-
reporting/templates/sealedsecret.yaml

```

The generated file will look like:

```

apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: erp-reporting-secrets
  namespace: erp-staging
spec:
  encryptedData:
    DATABASE_URL: AgBd...
    API_KEY: AgF3...
    DEFAULT_ENTERPRISE_ID: AgH9...

```

Commit sealedsecret.yaml. When applied, the controller creates/update the Secret named erp-reporting-secrets that your Deployment references.

To rotate values, regenerate `plain-secret.yaml` and re-seal.

3) Canary Rollouts with Argo Rollouts

3.1 Install Argo Rollouts (once)

```
kubectl create namespace argo-rollouts
kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-rollouts/releases/download/v1.7.2/install.yaml
# optional dashboard:
kubectl argo rollouts dashboard
```

3.2 Helm toggle: use Rollout instead of Deployment

Add to `values.yaml`:

```
rollouts:
  enabled: true
  canary:
    steps:
      - setWeight: 10
      - pause: { duration: 60 }      # seconds
      - setWeight: 30
      - pause: { duration: 60 }
      - setWeight: 60
      - pause: { duration: 60 }
      - setWeight: 100
    trafficRouting:
      nginx: {}                    # use NGINX ingress for splitting
      # Optional basic analysis template name (create separately)
      analysis: null
```

3.3 New template: `templates/rollout.yaml`

```
{{- if .Values.rollouts.enabled }}
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: {{ include "erp-reporting.fullname" . }}
  labels: {{- include "erp-reporting.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels: {{- include "erp-reporting.selectorLabels" . | nindent 6 }}
  strategy:
    canary:
      canaryService: {{ include "erp-reporting.fullname" . }}-canary
      stableService: {{ include "erp-reporting.fullname" . }}-stable
```

```

    trafficRouting:
      nginx: {}
    steps:
      {{- toYaml .Values.rollouts.canary.steps | nindent 6 }}
      {{- if .Values.rollouts.canary.analysis }}
    analysis:
      templates:
        - templateName: {{ .Values.rollouts.canary.analysis }}
      {{- end }}
    template:
      metadata:
        labels: {{- include "erp-reporting.selectorLabels" . | nindent 8 }}
      spec:
        containers:
          - name: api
            image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
            imagePullPolicy: {{ .Values.image.pullPolicy }}
            ports:
              - containerPort: 8080
            envFrom:
              - secretRef:
                  name: {{ .Values.secret.name }}
            env:
              - name: DEFAULT_ENTERPRISE_ID
                value: "{{ .Values.env.DEFAULT_ENTERPRISE_ID }}"
            readinessProbe:
              httpGet: { path: /health/ready, port: 8080 }
              initialDelaySeconds: 5
              periodSeconds: 10
            livenessProbe:
              httpGet: { path: /health, port: 8080 }
              initialDelaySeconds: 10
              periodSeconds: 20
            resources: {{- toYaml .Values.resources | nindent 12 }}
      {{- end }}

```

3.4 Split Services for stable/canary

Create `templates/service-stable.yaml` and `templates/service-canary.yaml`:

```

{{- if .Values.rollouts.enabled }}
apiVersion: v1
kind: Service
metadata:
  name: {{ include "erp-reporting.fullname" . }}-stable
spec:
  selector: {{- include "erp-reporting.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: 8080
---
apiVersion: v1
kind: Service
metadata:

```

```

    name: {{ include "erp-reporting.fullname" . }}-canary
spec:
  selector: {{- include "erp-reporting.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: 8080
{{- end }}

```

And **modify** `templates/ingress.yaml` so the backend points at **the stable Service** name when rollouts are enabled:

```

# inside backend.service.name:
{{- if .Values.rollouts.enabled -}}
{{ include "erp-reporting.fullname" . }}-stable
{{- else -}}
{{ include "erp-reporting.fullname" . }}
{{- end }}

```

NGINX Ingress + Rollouts will steer traffic weights between `-stable` and `-canary` based on the Rollout strategy.

3.5 Disable the old Deployment when Rollouts are on

Wrap the original `templates/deployment.yaml` with:

```

{{- if not .Values.rollouts.enabled }}
# ... existing Deployment here ...
{{- end }}

```

4) (Optional) Simple analysis template for Rollouts

Create `templates/analysis-template.yaml`:

```

{{- if and .Values.rollouts.enabled .Values.rollouts.canary.analysis }}
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: {{ .Values.rollouts.canary.analysis }}
spec:
  metrics:
    - name: http-95th-latency
      interval: 30s
      count: 2
      successCondition: result < 0.8
      provider:
        prometheus:

```

```
    address: http://prometheus.monitoring.svc.cluster.local:9090
    query: |
      histogram_quantile(0.95,
sum(rate(http_request_duration_seconds_bucket{job="fastapi"}[1m])) by (le))
{{- end }}
```

Set in values.yaml:

```
rollouts:
  enabled: true
  canary:
    analysis: erp-canary-analysis
    steps:
      - setWeight: 20
      - pause: { duration: 60 }
      - setWeight: 50
      - pause: { duration: 60 }
      - setWeight: 100
```

5) CI touches for the new bits

5.1 CI deploy uses SealedSecret + TLS

Your `docker-deploy.yml` Helm step already sets env via `--set`. With SealedSecrets, you **don't** pass secrets via `--set`; instead, ensure the `SealedSecret` is in the target namespace:

```
- name: Apply sealed secrets
  run: |
    kubectl -n erp-staging apply -f deploy/helm/erp-
reporting/templates/sealedsecret.yaml
```

Optionally, ensure the **ClusterIssuer** exists:

```
- name: Ensure ClusterIssuer
  run: kubectl apply -f infra/cert-manager/clusterissuer-lets-encrypt-
prod.yaml
```

If you enable rollouts in staging:

```
- name: Install Argo Rollouts CRDs (once per cluster)
  run: |
    kubectl apply -n argo-rollouts -f https://github.com/argoproj/argo-
rollouts/releases/download/v1.7.2/install.yaml || true
```

And pass Helm switch:

```
--set rollouts.enabled=true
```

6) Makefile helpers (optional)

Append:

```
# ==== Sealed Secrets ====
seal:
    @[ "$(file)" ] || (echo 'Usage: make seal file=plain-secret.yaml';
exit 1)
    kubeseal --format yaml < $(file) > deploy/helm/erp-
reporting/templates/sealedsecret.yaml && echo "Wrote sealedsecret.yaml"

apply-secrets:
    kubectl -n $(NS) apply -f deploy/helm/erp-
reporting/templates/sealedsecret.yaml

# ==== Rollouts ====
rollouts-dashboard:
    kubectl argo rollouts dashboard
```

Summary

- **TLS:** Cert-Manager issuer + Helm ingress annotations = automatic HTTPS, auto-renew.
- **Secrets:** Sealed Secrets keeps credentials out of git in plaintext; Deployment consumes Secret via `envFrom`.
- **Canary:** Argo Rollouts controlled by `values.yaml` switch; NGINX traffic split; optional Prometheus analysis gate.
- **CI/Make:** helper targets and workflow snippets to apply `SealedSecret`, ensure `ClusterIssuer`, and use rollouts in staging.

1) Environment override files

Create these next to your chart: `deploy/helm/erp-reporting/values-staging.yaml` and `values-prod.yaml`.

deploy/helm/erp-reporting/values-staging.yaml

```
# --- STAGING overrides ---
replicaCount: 2

image:
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 8080

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: erp-staging.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: erp-staging-tls
      hosts: [erp-staging.example.com]
  certManager:
    clusterIssuer: letsencrypt-prod

# Prefer SealedSecret-sourced env (Secret name must exist in namespace)
secret:
  enabled: true
  name: erp-reporting-secrets-staging

env:
  # fallback / non-secret env only
  DEFAULT_ENTERPRISE_ID: "00000000-0000-0000-0000-000000000000"

resources:
  requests: { cpu: "200m", memory: "256Mi" }
  limits:   { cpu: "500m", memory: "512Mi" }

autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 6
  targetCPUUtilizationPercentage: 70

# Canary disabled in staging by default (flip to true if you want to test)
rollouts:
```

enabled: false

deploy/helm/erp-reporting/values-prod.yaml

```
# --- PRODUCTION overrides ---
replicaCount: 4

image:
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 8080

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: erp.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: erp-prod-tls
      hosts: [erp.example.com]
  certManager:
    clusterIssuer: letsencrypt-prod

secret:
  enabled: true
  name: erp-reporting-secrets-prod

env:
  DEFAULT_ENTERPRISE_ID: "00000000-0000-0000-0000-000000000000"

resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits: { cpu: "1500m", memory: "1Gi" }

autoscaling:
  enabled: true
  minReplicas: 4
  maxReplicas: 12
  targetCPUUtilizationPercentage: 65

# Enable canary with Argo Rollouts in prod
rollouts:
  enabled: true
  canary:
    steps:
      - setWeight: 10
      - pause: { duration: 60 }
      - setWeight: 30
      - pause: { duration: 120 }
      - setWeight: 60
      - pause: { duration: 180 }
```

```
- setWeight: 100
analysis: erp-canary-analysis
```

Remember to create **two SealedSecrets**, one per namespace:

- erp-reporting-secrets-staging **in** erp-staging
 - erp-reporting-secrets-prod **in** erp-prod
- (Use the same `templates/sealedsecret.yaml` pattern, or keep separate files like `sealedsecret-staging.yaml` / `sealedsecret-prod.yaml`.)
-

2) Helmfile

Create `deploy/helm/helmfile.yaml` to declaratively manage both envs.

```
helmDefaults:
  wait: true
  atomic: true
  cleanupOnFail: true
  timeout: 600

repositories:
- name: local
  url: file://../helm

releases:
# === STAGING ===
- name: erp-reporting
  namespace: erp-staging
  chart: ./erp-reporting
  values:
    - ./erp-reporting/values.yaml
    - ./erp-reporting/values-staging.yaml
  set:
    - name: image.repository
      value: {{ requiredEnv "IMAGE_REPO" | default "ghcr.io/your-org/erp-reporting" }}
    - name: image.tag
      value: {{ requiredEnv "IMAGE_TAG" | default "latest" }}
  hooks:
    - events: ["presync"]
      showlogs: true
      command: kubectl
      args:
        - apply
        - -n
        - erp-staging
        - -f
        - ./erp-reporting/templates/sealedsecret.yaml

# === PRODUCTION ===
```

```

- name: erp-reporting
  namespace: erp-prod
  chart: ./erp-reporting
  values:
    - ./erp-reporting/values.yaml
    - ./erp-reporting/values-prod.yaml
  set:
    - name: image.repository
      value: {{ requiredEnv "IMAGE_REPO" | default "ghcr.io/your-org/erp-reporting" }}
    - name: image.tag
      value: {{ requiredEnv "IMAGE_TAG" | default "latest" }}
  hooks:
    - events: ["presync"]
      showlogs: true
      command: kubectl
      args:
        - apply
        - -n
        - erp-prod
        - -f
        - ./erp-reporting/templates/sealedsecret.yaml

```

You can keep a single `sealedsecret.yaml` if it's **cluster-wide encrypted** and namespaced properly; otherwise keep separate sealed secrets per env and point the hook to the right file (e.g., `sealedsecret-staging.yaml`, `sealedsecret-prod.yaml`).

3) Makefile helpers for Helmfile

Append these to your existing Makefile:

```

# ==== Helmfile ====
KCTX?=your-kube-context
IMAGE_REPO?=ghcr.io/your-org/erp-reporting
IMAGE_TAG?=$(shell git rev-parse --short=12 HEAD)

helmfile-staging:
    @echo "Deploying STAGING with IMAGE=$(IMAGE_REPO):$(IMAGE_TAG)"
    IMAGE_REPO=$(IMAGE_REPO) IMAGE_TAG=$(IMAGE_TAG) \
    kubectl config use-context $(KCTX) && \
    helmfile -f deploy/helm/helmfile.yaml --selector namespace=erp-staging
apply

helmfile-prod:
    @echo "Deploying PROD with IMAGE=$(IMAGE_REPO):$(IMAGE_TAG)"
    IMAGE_REPO=$(IMAGE_REPO) IMAGE_TAG=$(IMAGE_TAG) \
    kubectl config use-context $(KCTX) && \
    helmfile -f deploy/helm/helmfile.yaml --selector namespace=erp-prod
apply

helmfile-diff:

```

```
IMAGE_REPO=$(IMAGE_REPO) IMAGE_TAG=$(IMAGE_TAG) \
helmfile -f deploy/helm/helmfile.yaml diff
```

Usage:

- make helmfile-staging KCTX=<ctx> IMAGE_TAG=\$(git rev-parse --short HEAD)
- make helmfile-prod KCTX=<ctx> IMAGE_TAG=\$(git rev-parse --short HEAD)

4) GitHub Actions: deploy via Helmfile

Add a simple job to your existing deploy workflow (or a new one). This example targets **staging** on push to main, using **Helmfile** and your KUBE_CONFIG_STAGING secret.

```
# .github/workflows/deploy-helmfile.yml
name: deploy-helmfile

on:
  push:
    branches: [ main ]
    workflow_dispatch:

jobs:
  staging:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup kubectl
        uses: azure/setup-kubectl@v4
        with:
          version: v1.30.0

      - name: Setup Helm
        uses: azure/setup-helm@v4
        with:
          version: v3.14.4

      - name: Setup Helmfile
        run: |
          curl -sSL
          https://github.com/helmfile/helmfile/releases/download/v0.157.0/helmfile_0.15
          7.0_linux_amd64.tar.gz \
            | tar xz && sudo mv helmfile /usr/local/bin/

      - name: Write kubeconfig (staging)
        run: |
          mkdir -p $HOME/.kube
          echo "${{ secrets.KUBE_CONFIG_STAGING }}" > $HOME/.kube/config
          chmod 600 $HOME/.kube/config
```

```

- name: Ensure ClusterIssuer & SealedSecrets CRD
  run: |
    kubectl apply -f infra/cert-manager/clusterissuer-lets-encrypt-
prod.yaml || true
    kubectl apply -f https://github.com/bitnami-labs/sealed-
secrets/releases/download/v0.27.2/controller.yaml || true

- name: Deploy STAGING via Helmfile
  env:
    IMAGE_REPO: ghcr.io/your-org/erp-reporting
    IMAGE_TAG: ${ github.sha }
  run: |
    helmfile -f deploy/helm/helmfile.yaml \
      --selector namespace=erp-staging apply

```

For **production**, mirror this with `KUBE_CONFIG_PROD` and `--selector namespace=erp-prod`, and protect with manual approvals.

5) Quick ops runbook

- **TLS & DNS:** Point `erp-staging.example.com` and `erp.example.com` to your ingress LB. Cert-Manager will mint certs via the ClusterIssuer.
 - **Secrets:** Generate and seal per namespace:
 - # STAGING
 - `kubectl create secret generic erp-reporting-secrets-staging \`
 - `-n erp-staging \`
 - `--from-literal=DATABASE_URL='...' \`
 - `--from-literal=API_KEY='...' \`
 - `--from-literal=DEFAULT_ENTERPRISE_ID='00000000-0000-0000-0000-000000000000' \`
 - `--dry-run=client -o yaml > plain-staging.yaml`
 - `kubeseal --format yaml < plain-staging.yaml > deploy/helm/erp-reporting/templates/sealedsecret.yaml`
 - # PROD (or create `sealedsecret-prod.yaml` and reference it in the prod hook)
 - **Promote:** Push to main → CI builds/pushes image → run `deploy-helmfile staging`. When ready, run `make helmfile-prod` locally or trigger a prod workflow.
-

ERP Reporting Walk-Through

1. Login & Landing

UX Flow:

- User signs in via secure login (SSO, 2FA supported).
- Role-based dashboards (Finance, Operations, QA, etc.).
- Onboarding tooltip for first-time users.

UI Mock:

- Clean login page (logo, username, password, “Sign in with Google/SSO”).
- Optional 2FA code input.
- Landing dashboard tiles:
 -  Inventory
 -  Finance
 -  Operations
 -  Reports & Audit
 -  Admin/Settings

2. Main Dashboard

UX Flow:

- High-level KPIs (Opening Stock, Closing Stock, Purchases, Sales, COGS, Profit).
- Quick alerts (abnormal losses, expiring batches, consignment due).
- Drill-down links into report packs.

UI Mock:

- Top KPI cards (with color indicators):
 - Opening Stock Value
 - Purchases this period
 - Production Yield %
 - Sales & Invoices
 - Profit / Loss

- Left nav sidebar:
 - Reports (Inventory, Finance, Audit)
 - Transfers (Horizontal/Vertical)
 - Quality / Expiry
 - Settings
- Notifications bell 🔔 with alerts (e.g., “3 batches expiring in 7 days”).

3. 📁 Report Pack Menu

UX Flow:

- User clicks **Reports** → chooses type (Stock, Sales, Yield, Loss/Gain, Financials).
- Same-tier consolidation rules applied (✓ vertical, ✗ horizontal).
- Search & filter bar:
 - Period (from–to date)
 - Org Unit(s) (multi-select)
 - Tiers (dropdown: site → dept → process)
 - Currency / UoM toggle

UI Mock:

- Tile grid (like a report library):
 - 📦 Opening/Closing Stock
 - 🛒 Purchases & Landed Cost
 - 🔄 Interdepartment Transfers
 - 🏭 Production I/O & Yield
 - 📄 Sales & Invoices
 - ⚠️ Abnormal Losses/Gains
 - 📊 Stocktake Variances
 - 📁 Audit Pack (download all)

4. 📦 Inventory Report (Example Drill-Down)

UX Flow:

- Select “Closing Stock” → opens report with:
 - Hierarchical table (Sector > Dept > Batch > Item).
 - Columns: UoM (units, weight, volume), Cost, Value.
 - Expand/collapse by tier.
 - Export options: XLSX, PDF, JSON.

UI Mock:

- Left filter panel (date, dept, UoM).
 - Main pane: interactive table.
 - Toggle: Group by **Sector** or **Item**.
 - Graph view option: stock distribution pie/bar chart.
-

5. Transfers (Horizontal / Vertical)

UX Flow:

- Choose Transfers Report.
- Tabbed: **Vertical Transfers** | **Horizontal Transfers**.
- Vertical included in consolidation; horizontal tracked separately.
- User can filter: Source Dept, Destination Dept, Item, Batch.

UI Mock:

- Timeline bar showing transfer flow between depts.
 - Table: Source → Destination | Qty | Cost | Transfer Price | Timestamp.
 - Visual Sankey chart of flows (optional).
-

6. Production & Yield

UX Flow:

- Select Production I/O report.
- Show inputs (feeds, materials) vs outputs (finished goods).
- Highlight variances (Yield %, Losses).
- Drill-down into Work Order detail.

UI Mock:

- Two panels: Inputs (left) vs Outputs (right).
 - KPIs: Std vs Actual Yield %, Loss/Gain.
 - Variance chart (waterfall bar).
 - WO detail modal (batch, line, operator, cost).
-

7. Finance (P&L, BS, CF)

UX Flow:

- Finance tab auto-consolidates GL with stock/operational data.
- Reports:
 - Profit & Loss
 - Balance Sheet
 - Cash Flow
 - Financial Ratios

UI Mock:

- Tabs across top: P&L | BS | CF | Ratios.
 - P&L table: Revenue → COGS → Gross Profit → Expenses → Net.
 - Balance Sheet tree view: Assets / Liabilities / Equity.
 - Ratio cards: Liquidity, Gross Margin %, Debt/Equity.
-

8. Exceptions & Audit Pack

UX Flow:

- User clicks Audit Pack → system bundles:
 - Trial balance tie-outs
 - Eliminations schedules
 - Abnormal loss/gain register
 - Stocktake adjustments
 - Consignment & FEFO logs
- Download: ZIP (Excel + PDF).

UI Mock:

- Download screen: “Generate Audit Pack”.
 - Progress bar (async).
 - Success modal: “Audit Pack ready — download as XLSX / PDF / ZIP”.
-

9. Analytics / BI

UX Flow:

- Optional BI dashboard overlay for trends:
 - Inventory ageing
 - Sales trends
 - Yield over time
- Custom report builder (drag-drop fields).

UI Mock:

- Canvas with chart widgets.
 - Save/share dashboards.
 - Export to CSV/PowerBI.
-

10. ⚙ Settings / Admin

UX Flow:

- Configure consolidation rules, UoM, GL mappings.
- Role-based access to reports.
- Audit trail for every export.

UI Mock:

- Settings tabs: Consolidation | UoM | GL Mapping | Permissions.
 - Toggle switches for same-tier rules (✓ vertical, ✗ horizontal).
 - Role matrix (User × Report × Permission).
-

🎨 UI/UX Design Principles

- **Consistency:** Same filter bar across all reports.
 - **Clarity:** Always reconcile back to TB/GL.
 - **Extensibility:** Work for poultry, mining, retail, etc.
 - **Auditability:** Every download stamped with user, timestamp, parameters.
 - **Cross-device:** Responsive UI (desktop, tablet, mobile).
-

🌐 ERIK – Full User Experience Walk-Through (With Branding)

1. 🔑 Login & Branding Splash

- **Branding:** Centered **ERIK logo** (modern green E mark). Footer watermark: “Powered by Greenomic Enterprise”.
 - **UX:** Username + Password OR SSO (Google/Microsoft/Okta). Optional 2FA prompt.
 - **UI Mock:**
 - Green gradient background (ERIK logo glowing).
 - Login card with “Enterprise Resource & Intelligence Kernel”.
 - Greenomic Enterprise globe-leaf logo as a small watermark in the bottom-right.
-

2. 🏠 Landing Dashboard

- **Top Bar:** ERIK logo (left), user profile (right).
 - **Side Nav:** Inventory 📦, Operations 🏭, Finance 💰, Reports 📊, Audit 🔍, Settings ⚙️.
 - **Main Tiles:**
 - Opening Stock
 - Purchases
 - Transfers (Vertical/Horizontal)
 - Production Yield
 - Sales/Invoices
 - Closing Stock
 - Profit & Loss
 - **Branding:** Greenomic Enterprise banner in footer (“Rooted in Zambia. Growing in Unity.”).
-

3. 📁 Report Pack Library

- **UX:** Users select report category with consistent filters (date range, tiers, org units, currency/UoM).
- **UI:**
 - Grid of reports (with icons + ERIK green styling).

- Each report tile has a “Run” button + “Export” dropdown (XLSX/PDF/JSON).
 - Reports include:** Opening/Closing Stock, Purchases, Transfers, Yield, Sales, Losses/Gains, Variances, Financial Statements, Audit Pack.
-

4. 📦 Inventory Drill-Down

- UX:** Expandable tree (Enterprise → Sector → Department → Batch → Item).
 - UI Mock:**
 - Table columns: U/W/V, Cost, Value, Expiry, Lot/Serial.
 - FEFO warning badges ●.
 - Export buttons aligned top-right.
 - Branding:** “Greenomic Enterprise Quality Seal” watermark on printed/exported reports.
-

5. ↻ Transfers (Horizontal vs Vertical)

- Vertical Transfers:** Included in consolidation (shown in green flow arrows).
 - Horizontal Transfers:** Logged but excluded from consolidation (grey dashed arrows).
 - UI Mock:**
 - Sankey diagram flow visualization.
 - Tab switch: Vertical / Horizontal.
 - Table view for auditors.
-

6. 🏭 Production & Yield

- UX:** Compare inputs vs outputs. Show yield efficiency, variances, abnormal losses.
 - UI:**
 - Left panel: Inputs (feeds, materials).
 - Right panel: Outputs (finished goods).
 - KPIs: Std vs Actual Yield %, Loss Value.
 - Waterfall variance chart.
 - Branding:** ERIK green palette, Greenomic logo watermark on charts.
-

7. 💰 Finance Center

- P&L:** Hierarchical Revenue → COGS → GP → Opex → Net.

- **Balance Sheet:** Assets/Liabilities/Equity expandable tree.
 - **Cash Flow:** Operating/Investing/Financing toggles.
 - **Ratios:** KPI cards (Liquidity, ROA, D/E, Gross Margin %).
 - **Branding:** ERIK styled finance tables with Greenomic Enterprise footer.
-

8. Exceptions & Audit Pack

- **UX:** Auto-generated binder containing:
 - TB tie-outs
 - Eliminations schedules
 - Abnormal loss/gain registers
 - Consignment logs
 - Expiry/FEFO reports
 - **UI Mock:**
 - “Generate Audit Pack” button (with async progress bar).
 - Download as XLSX, PDF, or ZIP.
 - **Branding:** Digital stamp “Generated by ERIK • Certified by Greenomic Enterprise”.
-

9. Analytics / BI

- **UX:** Drag-drop custom report builder.
 - **UI Mock:**
 - Widgets for stock ageing, sales trend, yield efficiency.
 - Save/share dashboards.
 - **Branding:** Greenomic globe-leaf background watermark on analytics exports.
-

10. Admin & Settings

- **UX:**
 - Configure consolidation rules (✓ vertical / ✗ horizontal).
 - GL mappings.
 - Role-based access.
 - Audit logs.
- **UI Mock:**
 - Tab layout: Rules | UoM | GL | Permissions | Logs.
 - Role matrix grid with toggle switches.
- **Branding:** ERIK interface; Greenomic accreditation for compliance modules.

Design Principles (with ERIK/Greenomic)

- **ERIK = App Face** (dashboards, reports, UX flow).
 - **Greenomic = Institutional Seal** (watermarks, certificates, compliance).
 - **Consistent Green Palette** (gradient from light to deep emerald).
 - **Audit-Ready UX** (exports always include timestamp, user, enterprise ID).
 - **Cross-Industry Neutral** (works for poultry, mining, retail, government).
-

User Management & Roles screen.

It will include:

- **User directory table** (Name, Role, Department, Last Login, Status).
- **Role assignment matrix** (checkmarks/toggles for Reports, Finance, Inventory, Audit Pack).
- **Invite new user modal** with fields for email, department, tier, and role.
- **Expand/collapse tree** to drill down: Enterprise → Sector → Department → Team.

GL Mapping Editor.

It will include:

- **Mapping table** (ERP Item/Transaction Type → GL Account Code → Account Name).
- **Search + Auto-suggest** for GL accounts.
- **Drag-and-drop interface** for reassigning mappings.
- **Validation alerts** (e.g., unmapped items highlighted in red).
- **Export/Import buttons** (Excel, JSON for bulk mappings).

GL Mapping Editor mock with:

- Mapping table (ERP Item → GL Code → Account Name).
- Search/autocomplete for accounts.
- Drag & drop reassignments.
- Validation alerts for unmapped/duplicate codes.
- Import/Export (Excel/JSON).

Workflow Approvals Modal.

It will feature:

- **Multi-step approval chain** (e.g., Initiator → Manager → Finance → Audit).
- Each step with status indicators: ☐ Pending, ☐ Approved, ☒ Rejected.
- **Approval actions** (Approve, Reject, Request Changes, Add Comment).
- **Audit trail sidebar** (who approved/rejected + timestamp).
- **Configurable rules**: thresholds (e.g., >\$50k requires Finance + Audit).

Audit Logs & Activity Monitoring Dashboard.

It will include:

- **Chronological activity feed** (user logins, approvals, exports, edits).
- Filters (by user, module, action type, date range).
- **Color-coded risk flags** (green = normal, yellow = unusual, red = critical).
- **Drill-down option** to see details of each action (before/after values, IP, device).
- Export logs to CSV/PDF/JSON.

Stock Ageing Dashboard.

It will include:

- **Aging buckets** (0–30 days, 31–60, 61–90, 91+).
- Filters by **Item, Batch, Department, Sector**.
- **Bar/heatmap visualization** of stock distribution across aging buckets.
- Alerts for **near-expiry and expired batches** (FEFO compliance).
- Drill-down to see **lot/serial details**.
- Export as XLSX/PDF/JSON.

Stock Ageing Dashboard mock will now also feature:

- **Ageing buckets** (0–30, 31–60, 61–90, 91+ days).
- Bar/heatmap visualization.
- Filters (Item, Batch, Dept, Sector).
- **Expiry alerts** with FEFO compliance badges.
- **Drill-down lot/serial details.**
- **Predictive depletion forecast** chart (time series: expected run-out date based on consumption trends).
- Export options (XLSX, PDF, JSON).

Stock Count & Variance Screen.

It will include:

- **Stock count entry table** (system qty vs physical count).
- Automatic **variance calculation** (shortage/surplus in qty & value).
- **Color-coded alerts** (green = match, red = shortage, blue = surplus).
- **Batch/lot tracking** integrated with FEFO.
- Audit trail of who performed the count and when.
- Export option (XLSX/PDF).

Consignment & FEFO Manager screen.

It will include:

- **Consignment In/Out tracking** (with supplier/customer references).
- **FEFO (First-Expiry-First-Out) compliance dashboard:**
 - Near-expiry warnings
 - Expired batch alerts
 - Auto-suggestions for priority dispatch
- **Lot/serial tracking** tied to expiry dates.
- Filters for **Supplier, Warehouse, Department, Batch.**
- Export as XLSX/PDF/JSON.

Landed Cost Allocator screen.

It will include:

- **Import purchase order / shipment details** (items, quantities, base costs).
- Additional cost entry fields: freight, insurance, duties, handling, other overheads.
- **Automated cost allocation** across items (by qty, weight, volume, or value).
- Variance summary showing adjusted landed cost vs base purchase cost.
- Export options (Excel/PDF).

Landed Cost Allocator screen mock will now include:

- **Shipment & PO details** (items, base costs, quantities).
- Fields for **freight, insurance, duties, handling, overheads**.
- **Allocation logic** selector (by qty, weight, volume, or value).
- **Variance summary** (base cost vs landed cost).
- **Comparison chart** (side-by-side bar chart of base vs landed cost per item).
- Export options (Excel, PDF).

Serial/Lot Tracking & Quality Holds screen.

It will include:

- **Lot/Serial register** (Item, Lot/Serial number, Mfg date, Expiry date, Location).
- **Status tags:** Available, On Hold (QA), Released, Expired.
- **Quality Hold panel** with reason codes (e.g., pending inspection, failed test, regulatory block).
- **Drill-down** to movement history of each lot/serial.
- **Search & filter** by lot, item, expiry, supplier.
- Export options (Excel/PDF/JSON).

QA Release Workflow in the **Serial/Lot Tracking & Quality Holds** screen.

The mock will show:

- **Lot/Serial register** with columns (Item, Lot/Serial #, Mfg Date, Expiry, Location).
- **Status tags:** ☐ Available, ☐ On Hold, ☒ Expired.
- **Quality Hold panel** with reason codes.
- **Drill-down history** (movements, transfers, inspections).
- **QA Release Workflow modal:** Approve, Reject, Comment, Digital Sign-off.
- **Search & filter** by batch, expiry, supplier.
- Export to XLSX/PDF/JSON.

Scenario Costing & Activity-Based Overhead Allocation screen.

It will include:

- **Costing methods selector** (FIFO, LIFO, Weighted Avg, Standard, Scenario).
- **Scenario Costing workspace**: users enter assumptions (e.g., ↑ raw material price 10%, ↓ labor 5%).
- **Comparison table**: Standard vs Scenario costs, variance % by item/department.
- **Activity-Based Overhead Allocation panel**: overhead pools (Utilities, Maintenance, QA, Transport) allocated based on drivers (machine hours, labor hours, units, weight).
- **Graph view**: stacked bar chart of cost breakdown before vs after scenario.
- Export options (Excel/PDF/JSON).

Save Scenario & Compare Scenarios features in the **Scenario Costing & Overhead Allocation** screen.

The mock will show:

- **Costing selector** (FIFO, LIFO, Wt. Avg, Standard, Scenario).
- **Scenario workspace** with editable assumptions.
- **Comparison table** (Base vs Scenario 1 vs Scenario 2).
- **Save Scenario** button + list of stored scenarios.
- **Overhead Allocation panel** with cost drivers.
- **Stacked bar chart** comparing allocations before/after.
- Export to Excel/PDF/JSON.

Scenario Costing & Overhead Allocation mock will now also include a **sensitivity slider** for real-time simulations.

The final design will feature:

- Costing method selector (FIFO, LIFO, Wt. Avg, Standard, Scenario).
- Scenario workspace with editable assumptions.
- **Comparison table:** Base vs Scenario 1 vs Scenario 2.
- **Save Scenario** + **Compare Scenarios** list.
- Overhead allocation panel (pools + drivers).
- **Sensitivity slider** to adjust $\pm\%$ on key drivers (materials, utilities, labor) with instant recalculation.
- Graph view: stacked bar chart before vs after.
- Export (Excel, PDF, JSON).

Abnormal Losses & Gains Report screen.

It will include:

- **Variance register** showing shortages, damages, wastage, and unexpected gains.
- Columns: Item, Batch, Dept, Qty (U/W/V), Value, Reason, Approver.
- **Color-coded alerts** (red = loss, green = gain).
- **Approval workflow**: losses require manager sign-off, gains require investigation.
- **Drill-down** into source transaction (transfer, production, stocktake).
- Export options (Excel, PDF, JSON).

Abnormal Losses & Gains Report screen mock will now also include a **trend analysis chart**.

It will feature:

- **Variance register** (Item, Batch, Dept, Qty, Value, Reason, Approver).
- Color-coded alerts (● Loss, □ Gain).
- **Approval workflow** for sign-offs.
- Drill-down into source transactions.
- **Trend analysis chart** (loss/gain trends by department, supplier, or time period).
- Export to Excel/PDF/JSON.

Surplus & Shortage Report screen.

It will include:

- **Comparison table:** System Stock vs Physical Stock vs Allocated Orders.
- Highlighted variances: ☐ Surplus (excess stock), ☒ Shortage (stock deficit).
- **Filters:** Item, Dept, Warehouse, Date range.
- **Root-cause panel:** shows whether the variance came from transfers, miscounts, production yield variance, or expired stock.
- **Graph view:** bar chart showing surplus/shortage by department.
- Export options: Excel / PDF / JSON.

Surplus & Shortage Report screen mock will now also include a **recommendations panel**.

It will feature:

- **Comparison table** (System vs Physical vs Allocated).
- Variance highlights: ☐ Surplus, ☒ Shortage.
- Filters (Item, Dept, Warehouse, Date).
- **Root-cause panel** (transfers, miscounts, yield, expiry).
- **Bar chart** for surplus/shortage by department.
- **Recommendations panel** (e.g., auto-suggest reallocation, reorder, or write-off).
- Export to Excel/PDF/JSON.

Surplus & Shortage Report screen mock will now include:

- **Comparison table** (System vs Physical vs Allocated).
- Variance highlights (□ Surplus, ● Shortage).
- Filters (Item, Dept, Warehouse, Date).
- Root-cause analysis panel.
- Bar chart of variances by department.
- **Recommendations panel** with suggested actions (reallocate, reorder, write-off).
- **“Simulate Action” button** to preview financial & stock impact before committing.
- Export options (Excel, PDF, JSON).

Balance Sheet mock so that:

- **Total Assets = Total Equity + Liabilities** is **visibly enforced** on the diagram.
- A **balancing indicator/checkmark** will confirm reconciliation.
- If there's a mismatch, it will highlight in red (variance shown).
- Sections: **Assets (Current + Non-current)** on the left, **Liabilities + Equity** on the right.
- Totals aligned at the bottom with bold styling.

Balance Sheet mock will now include:

- **Assets vs Liabilities + Equity layout** with totals **guaranteed equal**.
- **Balancing indicator** (✓ green if equal, ✗ red if mismatch).
- Detailed breakdown: Current / Non-current Assets, Current / Non-current Liabilities, Equity.
- **Variance line** if mismatch occurs.
- **Toggle switch** at the top for **Consolidated vs Departmental** view.

Balance Sheet mock will now include:

- **Totals guaranteed equal** (Assets = Liabilities + Equity).
- **Balancing indicator** (✓ balanced / ✗ variance).
- **Breakdowns:** Current vs Non-current Assets, Current vs Non-current Liabilities, Equity.
- **Toggle switch:** Consolidated vs Departmental.
- **Drill-down expand/collapse tree** for accounts (e.g., Assets → Inventory → Raw Materials / WIP / Finished Goods).
- **Variance line** if mismatch occurs.

Balance Sheet mock will now feature:

- **Guaranteed reconciliation:** Total Assets = Total Liabilities + Equity.
- **Balancing indicator** (✓ balanced / ✗ mismatch).
- **Breakdowns:** Current vs Non-current Assets, Current vs Non-current Liabilities, Equity.
- **Toggle:** Consolidated vs Departmental.
- **Drill-down tree** for sub-accounts (e.g., Assets → Inventory → Raw Materials / WIP / Finished Goods).
- **Variance line** if mismatch occurs.
- **Reconciliation Report quick link** beneath totals, leading to a detailed tie-out with the Trial Balance.

Cash Flow Statement screen.

It will include:

- **Sections:**
 - Cash Flows from Operating Activities
 - Cash Flows from Investing Activities
 - Cash Flows from Financing Activities
- **Totals panel:** Net Cash Inflow/Outflow, Opening Balance, Closing Balance.
- **Reconciliation line:** Closing balance ties directly to Cash in the Balance Sheet.
- **Expand/Collapse tree** for detailed transactions under each activity.
- **Toggle switch** for Consolidated vs Departmental.
- **Charts:** bar/line graph showing inflows and outflows by category.
- **Export options:** Excel, PDF, JSON.

Cash Flow Statement mock will now include:

- **Sections:** Operating, Investing, Financing cash flows.
- **Totals panel:** Net Cash Flow, Opening Balance, Closing Balance.
- **Reconciliation line:** Closing balance = Balance Sheet cash.
- **Expand/Collapse tree** for drill-down.
- **Toggle:** Consolidated vs Departmental view.
- **Forecast Mode toggle** → overlay projections next to actuals.
- **Charts:** line/bar combo showing inflows vs outflows.
- **Export:** Excel, PDF, JSON.

Cash Flow Statement mock will now include a **scenario selector** alongside Forecast Mode.

Final design will feature:

- **Sections:** Operating, Investing, Financing activities.
- **Totals panel:** Net Cash Flow, Opening Balance, Closing Balance.
- **Reconciliation line** with Balance Sheet tie-out.
- **Expand/Collapse tree** for detailed transactions.
- **Consolidated vs Departmental toggle.**
- **Forecast Mode toggle.**
- **Scenario selector:** Best Case, Base Case, Worst Case.
- **Charts:** actual vs forecast (with scenario overlays).
- Export options (Excel, PDF, JSON).

Financial Ratios & KPI Dashboard screen.

It will include:

- **Key ratios grouped** by category:
 - Liquidity (Current Ratio, Quick Ratio, Cash Ratio)
 - Profitability (Gross Margin, Net Margin, ROA, ROE)
 - Efficiency (Inventory Turnover, Asset Turnover, DSO, DPO)
 - Leverage (Debt-to-Equity, Interest Coverage)
- **Visualizations:**
 - KPI cards with ratio values + color coding (green = healthy, red = concern).
 - Trend lines for ratios over time.
 - Benchmarks vs industry standards.
- **Scenario toggle** → compare ratios across Best Case, Base Case, Worst Case (from Cash Flow).
- **Expand/Collapse panels** for each ratio group.
- Export options: Excel, PDF, JSON.

Financial Ratios & KPI Dashboard mock will now also include a **composite Financial Health Score**.

Final design will feature:

- **Grouped ratios:**
 - Liquidity → Current, Quick, Cash Ratios.
 - Profitability → Gross Margin, Net Margin, ROA, ROE.
 - Efficiency → Inventory Turnover, Asset Turnover, DSO, DPO.
 - Leverage → Debt-to-Equity, Interest Coverage.
- **KPI cards** with ratio values + traffic-light indicators.
- **Trend lines** to show movement over time.
- **Benchmarks** vs industry averages.
- **Scenario toggle:** Best / Base / Worst case.
- **Expand/Collapse panels** for clean navigation.
- **Composite Financial Health Score** → single index (e.g., 78/100) shown as a gauge meter.
- Export options: Excel, PDF, JSON.

Asset Register & Records screen.

It will include:

- **Asset master list:** Asset ID, Name, Category, Department, Acquisition Date, Cost, Depreciation method.
- **Depreciation tracker:** Accumulated depreciation, Net book value.
- **Expand/Collapse tree:** Assets grouped by category (Buildings, Vehicles, Machinery, IT Equipment, etc.).
- **Location & Custodian fields:** track where asset is and who is responsible.
- **Lifecycle status:** Active, Under Maintenance, Disposed.
- **Integration** with GL accounts (auto-post depreciation, disposal gains/losses).
- **Charts:** Pie chart of asset distribution, Line chart of depreciation trend.
- Export options: Excel, PDF, JSON.

Asset Register & Records screen mock will now also include a **Revaluation & Impairment module**.

Final design will feature:

- **Asset master list** with full details (ID, Name, Category, Dept, Acquisition Date, Cost, Depreciation method).
- **Depreciation tracker** with accumulated depreciation + net book value.
- **Expand/Collapse tree** for asset categories (Buildings, Vehicles, Machinery, IT, etc.).
- **Location & Custodian** tracking.
- **Lifecycle status** (Active, Maintenance, Disposed).
- **Revaluation & Impairment panel:**
 - Record upward/downward adjustments.
 - Auto-update NBV (Net Book Value).
 - Post impact to GL.
- **Integration** with GL accounts (Depreciation, Revaluation Surplus, Disposal).
- **Charts:**
 - Pie chart of asset distribution by category.
 - Line chart of depreciation trend.
- Export (Excel, PDF, JSON).

Audit Pack Generation & Export screen.

It will include:

- **Pre-configured report pack builder:** Trial Balance, GL, Journals, P&L, Balance Sheet, Cash Flow, Variance Reports.
- **Audit evidence attachments:** scanned invoices, receipts, contracts linked to transactions.
- **Expand/Collapse tree** for selecting which reports to include.
- **Validation check:** confirms all included reports reconcile (e.g., P&L → Retained Earnings → Balance Sheet).
- **Export options:** ZIP bundle (Excel, PDF, JSON, Audit XML).
- **Audit trail sidebar** showing who generated/exported the pack and when.
- **Compliance flags** (ZRA Smart Invoice, IFRS, GAAP, IPSAS).

Audit Pack Generation & Export screen mock will now also include a **“Share with Auditor” button** for secure distribution.

The final design will feature:

- **Report pack builder** (TB, GL, Journals, P&L, BS, CF, Variances).
- **Audit evidence attachments** linked to transactions.
- **Expand/Collapse tree** for report selection.
- **Validation check** to ensure full reconciliation.
- **Export bundle**: ZIP (Excel, PDF, JSON, Audit XML).
- **Audit trail sidebar** (who generated, when).
- **Compliance flags**: ZRA Smart Invoice, IFRS, GAAP, IPSAS.
- **One-click “Share with Auditor”** → generates secure link/email with expiry.

Audit Pack Generation & Export screen mock next, with all the features we agreed on:

- Report pack builder (TB, GL, Journals, P&L, BS, CF, Variances)
- Attachments (invoices, receipts, contracts)
- Expand/Collapse tree for selection
- Reconciliation validation check
- Export as ZIP (Excel, PDF, JSON, Audit XML)
- Audit trail sidebar
- Compliance flags (ZRA Smart Invoice, IFRS, GAAP, IPSAS)
- **Share with Auditor button** (secure link/email)

Enterprise Consolidation Dashboard.

It will include:

- **Multi-level consolidation:** Enterprise → Sector → Department → Unit.
- **Tree view with expand/collapse** for drilling down into subsidiaries and departments.
- **Cross-check panel:** ensures no double-counting (horizontal transfers excluded, vertical transfers included).
- **KPIs at the top:** Consolidated Revenue, Expenses, Net Profit, Assets, Liabilities, Equity.
- **Charts:**
 - Pie chart of contribution by department/sector.
 - Line chart of consolidated trends over time.
- **Toggle:** Actuals vs Forecast, Consolidated vs Standalone view.
- Export options: Excel, PDF, JSON.

Enterprise Consolidation Dashboard mock will now also include a **Reconciliation Exceptions** panel.

Final design will feature:

- **Multi-level consolidation:** Enterprise → Sector → Department → Unit.
- **Expand/Collapse tree view** for drill-down.
- **Cross-check logic:** Horizontal transfers excluded, vertical transfers included.
- **Top KPIs:** Revenue, Expenses, Net Profit, Assets, Liabilities, Equity.
- **Charts:** Pie (department contribution), Line (trend over time).
- **Toggle:** Actuals vs Forecast, Consolidated vs Standalone.
- **Reconciliation Exceptions panel:** highlights mismatches, unposted eliminations, or unbalanced entries.
- **Export:** Excel, PDF, JSON.

Purchases & Landed Cost Reports screen.

It will include:

- **Purchases register:** Supplier, Invoice No., Date, Item, Qty, Base Cost.
- **Landed Cost panel:** freight, insurance, duties, handling, overhead allocations.
- **Comparison table:** Base Cost vs Landed Cost per item.
- **Variance highlights** (color-coded differences).
- **Charts:** Bar chart of landed vs base cost, Trend line of purchase cost evolution.
- **Filters:** Supplier, Item, Department, Date range.
- **Export options:** Excel, PDF, JSON.
- **Compliance flags:** ZRA Smart Invoice, VAT, Duties.

Purchases & Landed Cost Reports screen mock will now also include a **Supplier Performance panel**.

Final design will feature:

- **Purchases register** (Supplier, Invoice, Date, Item, Qty, Base Cost).
- **Landed Cost panel** (freight, insurance, duties, handling, overhead).
- **Comparison table**: Base vs Landed Cost per item.
- **Variance highlights** with color codes.
- **Charts**:
 - Bar chart → Base vs Landed Cost
 - Trend line → Purchase cost evolution over time
- **Filters**: Supplier, Item, Dept, Date range.
- **Supplier Performance panel**:
 - On-time delivery %
 - Cost variance history
 - Quality rating (stars or score)
- **Compliance flags**: ZRA Smart Invoice, VAT, Duties.
- Export options (Excel, PDF, JSON).

Sales & Invoicing Dashboard.

It will include:

- **Sales register:** Customer, Invoice No., Date, Items, Qty, Unit Price, Total.
- **Invoice status tracker:** Paid, Pending, Overdue.
- **Charts:**
 - Bar chart → Sales by department/region
 - Line chart → Sales trend over time
 - Pie chart → Product/category contribution
- **Filters:** Customer, Product, Department, Date range.
- **Tax compliance panel:** VAT, Withholding Tax, ZRA Smart Invoice QR integration.
- **Receivables aging panel** (0–30, 31–60, 61–90, 91+).
- Export options: Excel, PDF, JSON.

Sales & Invoicing Dashboard mock will now also include a **Customer Performance panel**.

Final design will feature:

- **Sales register** (Customer, Invoice No., Date, Item, Qty, Unit Price, Total).
- **Invoice status tracker** (☐ Paid, ☐ Pending, ☒ Overdue).
- **Charts:**
 - Bar → Sales by department/region
 - Line → Sales trend over time
 - Pie → Product/category contribution
- **Filters:** Customer, Product, Dept, Date.
- **Tax compliance panel:** VAT, WHT, ZRA Smart Invoice QR.
- **Receivables aging panel** (0–30, 31–60, 61–90, 91+).
- **Customer Performance panel:**
 - Purchase frequency
 - Avg. payment days
 - Credit utilization %
- Export options (Excel, PDF, JSON).

Yield & Production Output Dashboard.

It will include:

- **Production register:** Inputs (materials, labor, overhead) vs Outputs (finished goods, by-products, wastage).
- **Yield efficiency metrics:** Standard yield %, Actual yield %, Variance %.
- **Charts:**
 - Waterfall chart → Inputs → Outputs → Variance.
 - Line chart → Yield trends over time.
- **Filters:** Department, Batch, Product, Date range.
- **Variance alerts:** Abnormal losses/gains flagged.
- **Scenario costing toggle** → simulate different yield assumptions.
- Export options: Excel, PDF, JSON.

Yield & Production Output Dashboard mock will now also include a **Benchmark Comparison Panel**.

Final design will feature:

- **Production register:** Inputs vs Outputs (finished goods, by-products, wastage).
- **Yield efficiency metrics:** Standard %, Actual %, Variance %.
- **Charts:**
 - Waterfall → Inputs → Outputs → Variances.
 - Line → Yield trends over time.
- **Filters:** Department, Batch, Product, Date.
- **Variance alerts** (abnormal losses/gains).
- **Scenario costing toggle** (simulate assumptions).
- **Benchmark comparison panel:** vs industry averages, vs prior periods.
- Export (Excel, PDF, JSON).

Yield & Production Output Dashboard mock now with all the agreed features:

- Production register (inputs vs outputs)
- Yield efficiency metrics (Std %, Actual %, Variance %)
- Waterfall + line charts
- Variance alerts
- Scenario costing toggle
- Benchmark comparison panel (industry + prior periods)
- Export options (Excel, PDF, JSON)

Abnormal Losses & Gains Report screen.

It will include:

- **Losses & Gains register:** Event ID, Date, Item/Batch, Dept, Qty/Weight/Volume, Value.
- **Categories:** Spoilage, Breakage, Theft, Expiry, Revaluation Gains/Losses.
- **Variance tracker:** Expected vs Actual stock position.
- **Charts:**
 - Bar → Abnormal losses/gains by department.
 - Line → Trend of incidents over time.
- **Exception alerts:** Highlight variances above tolerance thresholds.
- **Root cause notes:** Allow attaching explanation documents/photos.
- **Export options:** Excel, PDF, JSON.

Abnormal Losses & Gains Report mock will now also include a **Corrective Actions panel**.

Final design will feature:

- **Losses & Gains register** (Event ID, Date, Item/Batch, Dept, Qty/Weight/Volume, Value).
- **Categories:** Spoilage, Breakage, Theft, Expiry, Revaluation Gains/Losses.
- **Variance tracker:** Expected vs Actual stock position.
- **Charts:**
 - Bar → Losses/gains by department.
 - Line → Trend over time.
- **Exception alerts:** Variances above tolerance flagged.
- **Root cause notes:** attach explanations/photos/docs.
- **Corrective Actions panel:** Assign tasks, deadlines, responsible managers.
- Export (Excel, PDF, JSON).

Abnormal Losses & Gains Report mock now with:

- Register of events (ID, Date, Item/Batch, Dept, Qty/Weight/Volume, Value)
- Categories: Spoilage, Breakage, Theft, Expiry, Revaluation Gains/Losses
- Variance tracker (Expected vs Actual)
- Charts (Bar + Line)
- Exception alerts (above thresholds)
- Root cause notes with attachments
- **Corrective Actions panel** (tasks, deadlines, manager assignments)
- Export options (Excel, PDF, JSON)

Surplus & Shortage Report screen.

It will include:

- Register view: Event ID, Item/Batch, Department, Expected Qty, Actual Qty, Variance (+/-), Value impact.
- Filters: By sector, department, item, date range.
- Consolidated metrics:
 - Total shortages (units + value)
 - Total surpluses (units + value)
 - Net effect
- Visualization:
 - Heatmap → hotspots of shortages
 - Pie chart → surplus vs shortage proportion
- Alerts: Highlight systemic variances across multiple cycles.
- Export (Excel, PDF, JSON).

🚪 Applying ERIK ERP Logic to Hotel Operations

1. Design Goals & Core Principles

- **Consistency:** The same transaction and reporting backbone used for manufacturing/agribusiness maps to hotel services.
- **Multi-Tier Structure:** Hotels have sectors (Accommodation, F&B, Events, Spa, etc.), departments within sectors (e.g., Room Service, Housekeeping, Restaurant, Kitchen, Bar), and tiers (front-office vs back-office).
- **Unit of Measure (UoM):** Instead of just units, weights, and volumes, hotel UoM includes **room-nights, covers (meals), guest-hours, occupancy %, service packages**.
- **Stock + Services Integration:** Hotels consume inventory (linen, food, beverages, toiletries) and generate outputs (rooms occupied, meals served, events booked).

2. Organisational Model (Variable Tiers)

- **Sector 1:** Accommodation (Front Desk, Housekeeping, Laundry).
- **Sector 2:** Food & Beverage (Restaurant, Kitchen, Bar).
- **Sector 3:** Events & Conferencing.
- **Sector 4:** Wellness (Spa, Gym, Pool).
- **Sector 5:** Back-office (Finance, HR, Maintenance).
- Each sector can run **independently or consolidated** into hotel-wide reporting.

3. Items, Batches & UoM

- **Items:**
 - Consumables → Linen, Toiletries, Food ingredients, Beverages.
 - Assets → Rooms, Conference halls, Spa equipment, Gym machines.
- **Batches:**
 - Perishable items (kitchen & bar stock) → lot-tracked with expiry (FEFO).
 - Service allocations → e.g., Room #101 occupancy from 01–03 Oct is a “batch”.
- **UoM Examples:**
 - Rooms → Room-nights.
 - Meals → Covers served.
 - Drinks → Bottles/ml.
 - Spa → Hours booked.

4. Stock Ledger & Movement Taxonomy

- **Purchases:** Food supplies, toiletries, linen, cleaning supplies.
 - **Interdepartment Horizontal Transfers:** Kitchen ingredients moved between restaurants/bars.
 - **Vertical Transfers:**
 - Inventory → Service consumption (toiletries issued to rooms, wine issued to bar, linen issued to housekeeping).
 - Service → Revenue (room occupied → posted to guest folio).
 - **Outputs:**
 - Accommodation nights sold.
 - Meals served.
 - Events booked.
 - **Sales/Invoices:** Guest folios consolidate multi-department charges.
-

5. Costing & Transfer Pricing

- **Direct Costs:** Food, beverage, laundry, utilities.
 - **Indirect/Overheads:** Staff salaries, maintenance, energy allocation.
 - **Transfer Pricing:**
 - Linen issued from laundry to rooms = costed.
 - Wine issued from store to bar = costed.
 - **Scenario Costing:** Simulate promotions (e.g., “3 nights for 2” → test profitability).
-

6. Reports

Stock & Service Reports

- Opening & Closing stocks (kitchen, bar, laundry).
- Consumption reports (ingredients, linen, toiletries).
- Room Occupancy reports (available vs sold room-nights).
- Meal covers served by day/shift.

Financial Reports

- Profit & Loss → segmented by Accommodation, F&B, Events, Spa.
- Balance Sheet → Hotel assets, prepaid bookings, liabilities.
- Cash Flow → Bookings vs payments vs expenses.

- Yield Reports → RevPAR (Revenue per Available Room), Food Cost %, Beverage Cost %, Event profitability.
 - Surplus/Shortage → linen losses, minibar variances, food wastage.
 - Abnormal Losses & Gains → Spoilage, theft, guest walk-outs, complimentary services.
-

7. Consolidation Rules

- **Within a Tier (same level):**
 - F&B restaurants consolidate into total F&B sales.
 - Vertical transfers included (ingredients to bar → cost flows through).
 - Horizontal transfers ignored to prevent duplication.
 - **Cross-Tier:**
 - Accommodation + F&B + Events + Spa = Hotel-wide consolidated accounts.
 - **No Duplication:** E.g., wine issued from store to bar → only consumed once.
-

8. Audit Pack

- Night Audit = Daily financial & stock reconciliation.
 - Variance report = Actual vs Expected revenue per department.
 - Activity-based allocation = Overheads split across rooms, F&B, events.
 - Export for auditors (PDF, Excel, JSON).
-

9. Cross-Industry Extensibility

- The exact same engine works for:
 - Resorts → add sectors like Golf, Marina.
 - Hospitals → wards = rooms, patients = guests, treatments = services.
 - Airlines → seats = rooms, meals = covers, crew = housekeeping/restaurant staff.
-

10. Developer Tasks Checklist

- Extend **Items table** to include service UoM (room-nights, covers).
- Build **Night Audit automation** (auto-close of daily transactions).
- Implement **Consolidation rules engine** (exclude horizontals, include verticals).
- Add **Hotel-specific KPIs**: RevPAR, ADR (Average Daily Rate), Occupancy %, Food & Beverage cost %.

- Link guest folios to multi-department charges.
 - Ensure **Smart Invoice & OCR** for supplier invoices (kitchen, laundry).
-

✓ With this approach, **ERIK ERP can fully manage a hotel's operations** the same way it handles stock-intensive businesses — only difference is the UoM and categorization of outputs (services instead of physical goods).

🌀 Hotel ERP Mockups Roadmap

1. **Unified Hotel Dashboard**
 - KPIs: Occupancy %, ADR (Average Daily Rate), RevPAR, F&B Cost %, Event Utilization.
 - Tiles for Accommodation, F&B, Events, Spa, Finance.
 2. **Room Occupancy & Revenue Report**
 - Room-nights sold vs available.
 - Heatmap of occupancy per room/floor.
 - Revenue per room type (standard, deluxe, suite).
 3. **F&B Consumption & Costing Report**
 - Covers served.
 - Food & Beverage stock usage.
 - Variance vs standards.
 - Gross margin by menu category.
 4. **Night Audit & Variance Dashboard**
 - End-of-day reconciliation.
 - Actual vs expected sales.
 - Variances flagged for investigation.
 5. **Events & Conferencing Profitability Report**
 - Booked vs available slots.
 - Cost of services (catering, AV equipment).
 - Profitability per event.
 6. **Spa & Wellness Utilization Report**
 - Hours booked vs capacity.
 - Revenue by service.
 - Staff allocation efficiency.
 7. **Hotel P&L & Balance Sheet**
 - Departmental segmented reporting.
 - Consolidation into full hotel P&L.
 - Balance Sheet with assets (hotel property, fixtures), liabilities, equity.
-

Unified Hotel Dashboard (Hotel ERP) – Design Specs

Top KPIs (cards at the top):

-  **Occupancy %** (e.g., 82%)
-  **ADR (Average Daily Rate)**
-  **RevPAR (Revenue per Available Room)**
-  **F&B Cost %** (Actual vs Standard)
-  **Event Utilization %**
-  **Spa Utilization %**

Tiles/Sections (expandable):

1. **Accommodation** – Room availability, bookings, cancellations, revenue.
2. **Food & Beverage (F&B)** – Covers served, food cost %, beverage cost %.
3. **Events & Conferencing** – Bookings, revenue, cost allocation.
4. **Spa & Wellness** – Hours booked, capacity utilization, revenue.
5. **Finance** – Cash flow summary, P&L snapshot, variances.

Visuals:

- **Bar chart:** Revenue by department.
- **Line chart:** Occupancy trends (daily/weekly/monthly).
- **Pie chart:** Revenue contribution by department.
- **Heatmap:** Room occupancy by floor/wing.

Controls:

- Date range filter.
 - Actual vs Forecast toggle.
 - Export: Excel, PDF, JSON.
-

Room Occupancy & Revenue Report.

🌀 Room Occupancy & Revenue Report – Design Specs

Core Panels:

- **Occupancy Summary:**
 - Total rooms available vs sold.
 - Occupancy % (with daily, weekly, monthly toggle).
- **Revenue Summary:**
 - Total room revenue.
 - ADR (Average Daily Rate).
 - RevPAR (Revenue per Available Room).
- **Occupancy Heatmap:**
 - Visual grid by floor/wing showing booked vs vacant rooms.
- **Trends & Charts:**
 - Line chart → Occupancy over time.
 - Bar chart → Revenue by room type (standard, deluxe, suite).

Controls & Filters:

- Date range, room type, floor/wing, booking source (OTA, direct, corporate).
- Actual vs forecast toggle.

Special Features:

- Cancellation & no-show tracker.
 - Variance alerts (if occupancy < target threshold).
 - Export → Excel, PDF, JSON.
-

Room Occupancy & Revenue Report mock now with all the agreed features:

- Occupancy summary (rooms available vs sold, %)
- Revenue summary (ADR, RevPAR)
- Heatmap of room occupancy (by floor/wing)
- Line chart → occupancy trend
- Bar chart → revenue by room type
- Cancellation & no-show tracker
- Variance alerts (below target occupancy)
- Filters (date range, room type, booking source)
- Export (Excel, PDF, JSON)

Food & Beverage (F&B) Consumption & Costing Report.

🌀 F&B Consumption & Costing Report – Design Specs

Core Panels:

- **Consumption Summary:**
 - Total covers served (breakfast, lunch, dinner, bar).
 - Ingredient consumption vs standards.
- **Costing Summary:**
 - Food cost % (actual vs target).
 - Beverage cost % (actual vs target).
 - Variance report (favorable/unfavorable).
- **Charts:**
 - Bar chart → Food & Beverage revenue by category.
 - Line chart → Cost % trend over time.
 - Pie chart → Contribution by menu category (starters, mains, drinks, etc.).
- **Waste & Variance Tracker:**
 - Abnormal losses (spoilage, wastage, theft).
 - Surplus/shortage reporting.

Controls & Filters:

- Date range, outlet (restaurant, bar, room service), menu category.
- Actual vs forecast toggle.

Special Features:

- Supplier linkage → ingredients consumed vs delivered.
 - Scenario costing toggle (menu simulations, pricing tests).
 - Export → Excel, PDF, JSON.
-

F&B Consumption & Costing Report mock now with all the agreed features:

- Consumption summary (covers served, ingredient usage vs standards)
- Costing summary (food cost %, beverage cost %, variances)
- Charts:
 - Bar → F&B revenue by category
 - Line → Cost % trend
 - Pie → Contribution by menu category
- Waste & variance tracker (spoilage, shortages, theft)
- Supplier linkage (ingredients consumed vs delivered)
- Scenario costing toggle (menu pricing simulations)
- Filters (date range, outlet, menu category)
- Export (Excel, PDF, JSON)

Night Audit & Variance Dashboard.

Night Audit & Variance Dashboard – Design Specs

Core Panels:

- **Daily Closure Summary:**
 - Rooms sold vs expected.
 - Revenue posted by department (Rooms, F&B, Events, Spa).
 - Variance vs forecast.
- **Variance Analysis:**
 - Expected vs actual occupancy.
 - Expected vs actual revenue per department.
 - Exceptions flagged (under-reported sales, missing postings).
- **Charts:**
 - Line chart → Daily occupancy trend.
 - Bar chart → Actual vs Expected revenue by department.
 - Pie chart → Revenue breakdown by source.
- **Reconciliation:**
 - Folio balances (guests).
 - POS integration checks (restaurant, bar, spa).
- **Controls & Filters:**
 - Date picker (select audit date).
 - Drill-down by department.

Special Features:

- “Lock Day” function after audit completed.
 - Exception log export (Excel, PDF, JSON).
 - Compliance check (Smart Invoice reconciliation).
-

Night Audit & Variance Dashboard mock now with all the agreed features:

- Daily closure summary (rooms sold, revenue by dept, variance vs forecast)
- Variance analysis (expected vs actual occupancy & revenue)
- Charts:
 - Line → Daily occupancy trend
 - Bar → Actual vs Expected revenue by dept
 - Pie → Revenue breakdown by source
- Reconciliation (guest folio balances, POS checks)
- Controls (date picker, department drill-down)
- Special features: “Lock Day” + Exception log export
- Compliance checks (Smart Invoice)

Events & Conferencing Profitability Report – Design Specs

Core Panels:

- **Events Register:** Event ID, Client, Date, Venue, Capacity, Status (Booked/Completed).
- **Revenue Summary:** Venue charges, Catering, Equipment rental, Other services.
- **Costing Summary:** Food & beverage costs, staffing, AV/equipment, overhead allocation.
- **Profitability Metrics:** Gross profit per event, Margin %, Variance vs budget.
- **Charts:**
 - Bar → Revenue vs Cost per event.
 - Line → Event profitability trends.
 - Pie → Contribution of events to total hotel revenue.
- **Utilization Metrics:** % of available venues booked, Average spend per event.

Controls & Filters:

- Date range, venue, client type (corporate, social, NGO, etc.).
- Actual vs forecast toggle.

Special Features:

- Attach contracts, invoices, and receipts per event.
 - Scenario costing (e.g., pricing adjustments, package simulation).
 - Export (Excel, PDF, JSON).
-

Events & Conferencing Profitability Report mock now with:

- Events register (Event ID, Client, Date, Venue, Capacity, Status)
- Revenue summary (venue, catering, equipment, other services)
- Costing summary (F&B, staff, AV/equipment, overheads)
- Profitability metrics (gross profit, margin %, budget variance)
- Charts:
 - Bar → Revenue vs Cost per event
 - Line → Profitability trends
 - Pie → Contribution to total hotel revenue
- Utilization metrics (% venues booked, avg spend per event)
- Filters (date, venue, client type, actual vs forecast)
- Special features: Attachments + Scenario costing
- Export options (Excel, PDF, JSON)

Spa & Wellness Utilization Report.

Spa & Wellness Utilization Report – Design Specs

Core Panels:

- **Booking Summary:** Service type (massage, facial, gym, pool, yoga), Slots available vs booked, Utilization %.
- **Revenue Summary:** Revenue per service type, Avg spend per guest, Package vs single service revenue.
- **Staff Allocation:** Therapist/Instructor hours vs utilization.
- **Charts:**
 - Bar → Revenue by service type.
 - Line → Utilization trends over time.
 - Pie → Contribution of spa/wellness to overall hotel revenue.
- **Variance Tracker:** Actual vs expected utilization.

Controls & Filters:

- Date range, service type, staff, guest category (walk-in, in-house, membership).
- Actual vs forecast toggle.

Special Features:

- Package management (bundled services).
 - Attachments (guest consent forms, staff schedules).
 - Export (Excel, PDF, JSON).
-

Hotel P&L and Balance Sheet Report.

Hotel P&L & Balance Sheet – Design Specs

Profit & Loss Statement (P&L):

- **Revenue Segments:** Accommodation, F&B, Events, Spa, Other services.
- **COGS/Direct Costs:** Food, beverages, laundry, spa consumables.
- **Gross Profit & Margin %.**
- **Operating Expenses:** Salaries, utilities, maintenance, marketing.
- **Net Profit Before/After Tax.**
- **Charts:**
 - Bar → Revenue vs Expenses by department.
 - Line → Profit trends over time.

Balance Sheet:

- **Assets:** Current (cash, receivables, inventories) + Non-current (property, equipment).
- **Liabilities:** Current (payables, accruals) + Non-current (loans).
- **Equity:** Share capital, retained earnings.
- **Balancing Indicator:** ✓ Assets = Liabilities + Equity.
- **Expand/Collapse Tree:** Drill into asset & liability classes.
- **Reconciliation Report link:** Tie-out with Trial Balance.

Controls & Features:

- Date range, Consolidated vs Departmental toggle.
 - Export (Excel, PDF, JSON).
-

Hotel Cash Flow Statement – Design Specs

Core Sections:

- **Operating Activities:** Guest payments, supplier payments, payroll, utilities.
- **Investing Activities:** Renovations, equipment purchases, asset disposals.
- **Financing Activities:** Loan drawdowns/repayments, equity injections, dividends.
- **Totals Panel:** Net cash inflow/outflow, opening balance, closing balance.

Special Features:

- **Reconciliation line** → Closing balance ties to hotel Balance Sheet cash.
- **Forecast Mode toggle** → Future cash flow projections.
- **Scenario Selector** → Best Case, Base Case, Worst Case.
- **Charts:**
 - Line chart → Actual vs Forecast inflows/outflows.
 - Bar chart → Cash flow breakdown by activity.

Controls & Filters:

- Date range, department, actual vs forecast.
 - Consolidated vs Departmental toggle.
 - Export (Excel, PDF, JSON).
-

ERP Setup & Reporting for Hospitals and Clinics

Design Goals & Core Principles

1. **Unified but Flexible Framework**
 - Support hospitals, clinics, and multi-site health systems.
 - Variable number of **tiers (sectors → departments → units → wards)**.
2. **Consistent Movement & Transfer Logic**
 - **Horizontal Transfers** (same tier): e.g., drugs reallocated between wards.
 - **Vertical Transfers** (different tiers): e.g., pharmacy stock to wards, or lab results → patient record → billing.
3. **Transparency & Compliance**
 - Every transaction reconciles back to **stock ledgers, patient accounts, and GL**.
 - FEFO/expiry handling for drugs & consumables.
 - Audit pack for regulators (health ministry, ZRA, insurance boards).

Organisational Model (Variable Tiers)

- **Tier 1: Sectors** → Clinical (Outpatient, Inpatient, Surgery), Diagnostics, Pharmacy, Admin.
- **Tier 2: Departments** → Cardiology, Pediatrics, Radiology, Pharmacy, Lab, Theatre.
- **Tier 3: Units/Wards** → ICU, OPD, Maternity Ward, Lab benches, Sub-pharmacies.

Each sector/department can run standalone reports **and** consolidate into hospital-wide reports.

Item & Service Batches + Units of Measure

- **Drugs & Consumables** → Tracked by batch, expiry date, lot, UoM (pills, bottles, syringes).
 - **Services** → Measured in hours, sessions, or test counts.
 - **Assets** → Medical equipment, beds, ambulances.
 - **Patients** → "Stock movement" is mirrored in treatment flows (e.g., patient intake → service → billing).
-

Report Packs

1. **Stock & Pharmacy Reports**
 - Opening & closing drug stock.
 - Purchases (from suppliers).
 - Consignments in/out (shared procurement).
 - Returns, expiry, FEFO enforcement.
 - Surplus/shortage & variance vs expected usage.
 2. **Patient Care Reports**
 - Admissions, discharges, occupancy by ward.
 - Outpatient visits by specialty.
 - Diagnostic test volumes & turnaround times.
 3. **Financial Reports**
 - Patient billing & invoices.
 - Insurance claims vs collections.
 - Cash flow statements (by department & consolidated).
 - Profit & Loss (by specialty).
 - Balance Sheet (hospital as an enterprise).
 - Ratios → Bed Occupancy %, Average Length of Stay (ALOS), Cost per Patient.
 4. **Operational Reports**
 - Theatre utilization rates.
 - Doctor & nurse workload.
 - Equipment uptime/downtime.
 - Referral patterns (internal vs external).
 5. **Audit & Compliance Reports**
 - Controlled drug usage & disposal logs.
 - NHIMA/insurance claim reports.
 - Staff overtime & payroll linkages.
 - Patient safety metrics (adverse events, infection rates).
-

Consolidation Rules

- **Same Tier (Horizontal):** *Do not consolidate patient transfers across wards, only net usage is reported.*
 - **Cross-Tier (Vertical):** Pharmacy → Ward → Patient is consolidated into **net drug consumption**.
 - **Financial Reconciliation:**
 - Each departmental report reconciles back to **general ledger**.
 - Patient invoices → A/R aging → Hospital revenue line in P&L.
 - Cash & receivables reconcile to Balance Sheet.
-

□ Costing & Transfer Pricing

- Drugs → Landed cost allocation (purchase + freight + storage).
 - Services → Activity-based costing (lab tests, radiology scans).
 - Patient charges → Transfer pricing when service provided by another department (e.g., surgery consumes theatre, pharmacy, lab).
-

▣ Example Calculations

- **Drug Variance:** Expected stock = Opening + Purchases – Issues. Actual < Expected = Loss (spoilage/theft).
 - **Bed Occupancy:** Occupied bed days ÷ Available bed days × 100.
 - **ALOS:** Total inpatient days ÷ Discharges.
 - **Cost per Patient:** (Direct costs + Overheads) ÷ # of patients.
-

▣ UI/UX: How Users Run These

- **Dashboards:**
 - Doctor view: patient counts, procedures done.
 - Admin view: bed occupancy, drug usage, revenues.
 - Finance view: departmental P&L + consolidated.
 - **Reports:**
 - On-demand or scheduled (daily, monthly, annually).
 - Export formats (Excel, PDF, JSON).
 - **Controls:**
 - Expand/collapse trees by sector/department.
 - Actual vs forecast toggles.
-

✂ Developer Tasks Checklist

- Build **Stock Ledger schema** for drugs & consumables.
- Create **Service Ledger** for tests, treatments, procedures.
- Implement **Patient Ledger** linking care events to billing.
- Ensure **multi-UoM support** (pills, doses, ml, hours).
- Set up **audit pack** generation for regulators.
- Build **reconciliation jobs** (stock ↔ billing ↔ GL).
- Implement **expiry & FEFO logic**.
- Deploy **variance alerts** (drug shortages, overuse).

Final Notes

- This ERP design ensures **clinical accuracy + financial compliance**.
 - Can scale from **small clinics** to **multi-hospital networks**.
 - Extensible across industries (agriculture, hotels, hospitals, schools) with the same **ledger + transfer + consolidation framework**.
-

Unified Hospital Dashboard – Design Specs

Top KPIs (cards at the top):

-  **Bed Occupancy %** (e.g., 78%)
-  **Average Length of Stay (ALOS)**
-  **Revenue by Department**
-  **Pharmacy Stock Variance %**
-  **Lab Test Turnaround Time**
-  **Claim Collection Rate (Insurance/NHIMA)**

Tiles/Sections (expandable with drill-down):

1. **Admissions & Discharges** – Patient inflows/outflows by ward.
2. **Pharmacy & Consumables** – Stock, expiry alerts, drug usage vs expected.
3. **Diagnostics (Lab & Imaging)** – Test volumes, utilization %, TAT.
4. **Theatre & Procedures** – Utilization %, cancellations, complications.
5. **Finance** – Revenue, expenses, departmental P&L, consolidated P&L.

Visuals:

- **Bar chart:** Revenue contribution by department (Outpatient, Inpatient, Pharmacy, Diagnostics, Surgery).
- **Line chart:** Bed occupancy trends over time.
- **Pie chart:** Expense breakdown (Salaries, Consumables, Utilities, Admin).
- **Heatmap:** Bed occupancy per ward/unit.

Controls:

- Date range filter.
 - Consolidated vs Departmental toggle.
 - Actual vs Forecast toggle.
 - Export: Excel, PDF, JSON.
-

Unified Hospital Dashboard mock now with:

- **KPIs:** Bed Occupancy %, ALOS, Revenue by Dept, Pharmacy Variance %, Lab TAT, Claim Collection Rate
- **Expandable tiles:** Admissions, Pharmacy, Diagnostics, Theatre, Finance
- **Visuals:** Bar (Revenue by dept), Line (Occupancy trend), Pie (Expense breakdown), Heatmap (Beds by ward)
- **Controls:** Date range, Consolidated vs Departmental, Actual vs Forecast, Export options

Hospital/Clinic Patient Admissions & Discharges Report Screen.

Patient Admissions & Discharges – Design Specs

Core Sections:

- **Summary KPIs:**
 - Total Admissions (by date range)
 - Total Discharges
 - Average Length of Stay
 - Readmission Rate %
- **Admissions Table:**
 - Patient ID, Name, Age, Gender, Admission Date, Ward/Unit, Diagnosis, Admitting Doctor.
- **Discharges Table:**
 - Patient ID, Discharge Date, Outcome (Recovered, Referred, Deceased), Ward.
- **Charts & Visuals:**
 - **Line chart:** Admissions vs Discharges trend.
 - **Bar chart:** Admissions by Department/Ward.
 - **Heatmap:** Bed Occupancy status by ward.

Controls:

- Date range picker.
 - Department/Ward filter.
 - Consolidated vs Departmental toggle.
 - Export to Excel, PDF, JSON.
-

Hospital/Clinic Patient Admissions & Discharges Report mock now with:

- **KPIs:** Admissions, Discharges, Avg Length of Stay, Readmission Rate
- **Tables:** Admissions details + Discharges details
- **Visuals:** Line (Admissions vs Discharges trend), Bar (Admissions by ward), Heatmap (Bed occupancy)
- **Controls:** Date range, ward filter, consolidated toggle, export options

Hospital Pharmacy & Consumables Report Screen.

Pharmacy & Consumables Report – Design Specs

KPIs (Top Cards):

- Opening Stock Value
- Purchases (Qty & Value)
- Issues/Consumption (by ward/department)
- Expired/Returned Stock
- Closing Stock Balance
- Variance % (Expected vs Actual)

Tables:

- **Stock Ledger:** Item, Batch No, Expiry, UoM, Opening, Purchases, Issues, Adjustments, Closing.
- **Expiry Alerts:** Item, Qty, Expiry Date, Ward/Dept.

Charts & Visuals:

- **Bar chart:** Drug usage by department.
- **Pie chart:** Stock distribution by category (Antibiotics, Analgesics, Consumables, Equipment).
- **Line chart:** Monthly stock movement trends.

Controls:

- Date range picker.
 - Department/Ward filter.
 - Expiry $\leq$ 30 days toggle.
 - Export options (Excel, PDF, JSON).
-

Hospital Pharmacy & Consumables Report mock with:

- **KPIs:** Opening stock, Purchases, Issues, Expiry>Returns, Closing balance, Variance %
- **Tables:** Stock Ledger + Expiry Alerts
- **Charts:** Bar (Usage by dept), Pie (Stock category), Line (Trends)
- **Controls:** Date range, ward filter, expiry toggle, export

Hospital Diagnostics (Lab & Imaging) Report Screen.

Diagnostics Report – Design Specs

KPIs (Top Cards):

- Total Lab Tests Conducted
- Total Imaging Procedures (X-ray, CT, MRI, Ultrasound)
- Average Turnaround Time (TAT)
- Critical Test Alerts (%)
- Revenue from Diagnostics

Tables:

- **Lab Tests Table:** Test ID, Type, Department, Ordered By, Result Time, Status.
- **Imaging Table:** Procedure ID, Patient ID, Type (X-ray/CT/MRI), Technician, Report Issued (Y/N).

Charts & Visuals:

- **Bar chart:** Test volume by type (Blood, Urine, Imaging categories).
- **Line chart:** TAT trend over time.
- **Pie chart:** Revenue share by test type.

Controls:

- Date range picker.
 - Department filter (Lab, Imaging, Both).
 - Urgent/Emergency toggle.
 - Export options (Excel, PDF, JSON).
-

Hospital Theatre & Procedures Utilization Report Screen.

Theatre & Procedures Report – Design Specs

KPIs (Top Cards):

- Total Surgeries/Procedures
- Theatre Utilization %
- Cancellation Rate %
- Average Procedure Duration
- Complication/Adverse Event Rate %

Tables:

- **Procedure Log:** Procedure ID, Patient ID, Surgeon, Type, Duration, Outcome, Dept/Ward.
- **Cancellation Log:** Reason, Date, Department, Surgeon.

Charts & Visuals:

- **Bar chart:** Procedures by department (General, Ortho, Cardiac, Neuro, etc.).
- **Line chart:** Utilization trend across days/weeks.
- **Pie chart:** Cancellation reasons distribution.

Controls:

- Date range picker.
 - Department/specialty filter.
 - Elective vs Emergency toggle.
 - Export: Excel, PDF, JSON.
-

Hospital Finance Dashboard (P&L, Balance Sheet, Cash Flow) — consolidated and departmental.

Hospital Finance Dashboard – Design Specs

KPIs (Top Cards):

- Total Revenue (by dept & consolidated)
- Operating Expenses (OPEX)
- Net Profit/Loss
- Cash Balance
- Accounts Receivable (Insurance + Patients)
- Accounts Payable (Suppliers)

Reports/Sections:

- 1. Profit & Loss Statement (P&L):**
 - Revenue by department (Inpatient, Outpatient, Diagnostics, Pharmacy).
 - Direct Costs (drugs, consumables, lab kits).
 - Operating Expenses (salaries, utilities, maintenance).
 - Net Profit Before/After Tax.
- 2. Balance Sheet:**
 - Assets (cash, receivables, equipment).
 - Liabilities (payables, loans).
 - Equity.
 - ✓ Assets = Liabilities + Equity (visual validation).
- 3. Cash Flow Statement:**
 - Operating, Investing, Financing activities.
 - Actual vs Forecast toggle.
 - Scenario selector (Best, Base, Worst case).

Charts & Visuals:

- Bar chart → Revenue vs Expenses per department.
- Line chart → Profitability trend.
- Pie chart → Expense distribution.
- Heatmap → AR aging by payer (insurance vs patients).

Controls:

- Date range picker.
 - Consolidated vs Departmental toggle.
 - Export to Excel, PDF, JSON.
-

Audit Pack & Compliance Export – Design Specs

KPIs (Top Cards):

- Total Reports Generated
- Pending Compliance Checks
- Last Audit Date
- Next Scheduled Audit

Audit Pack Sections:

- **Financial Reports:** P&L, Balance Sheet, Cash Flow, Ratios.
- **Operational Reports:** Admissions, Theatre, Diagnostics, Pharmacy.
- **Compliance Reports:** NHIMA claims, ZRA Smart Invoice exports, Drug expiry logs.
- **Variance & Exception Logs:** Revenue shortfalls, drug shortages, insurance claim rejections.

Features:

- **Export Options:** Excel, PDF, JSON, XML (for regulatory systems).
- **Scenario Pack:** Ability to generate audit packs by department or consolidated.
- **One-click Zip Export:** All reports bundled with metadata and audit trail.

Charts & Visuals:

- Pie chart → Reports by type (Financial vs Operational vs Compliance).
- Bar chart → Exceptions by department.
- Line chart → Audit readiness trend over time.

Controls:

- Date range picker.
 - Consolidated vs Departmental export.
 - Regulator-specific format toggle (e.g., NHIMA, ZRA, IFRS).
-

Let's adapt the **ERIK ERP framework** you've built (stocks, transfers, consolidation, reports, audit packs) into a **Police Service operational and financial management setup**.

ERP Setup & Reporting for Police Service Operations

Design Goals & Core Principles

1. **Unified Command & Control** — capture data across **stations, units, and divisions**.
 2. **Resource Accountability** — ensure full tracking of personnel, assets (vehicles, firearms, uniforms), and consumables (fuel, rations, ammunition).
 3. **Operational Transparency** — enforce clear reporting on arrests, deployments, patrols, and investigations.
 4. **Financial & Compliance Integration** — link budgets, expenses, and grants with GL, audit, and government reporting.
 5. **Scalable Structure** — support **national HQ → regional HQ → stations → units**.
-

Organisational Model (Variable Tiers)

- **Tier 1: National Headquarters** – Policy, budget, consolidated reporting.
- **Tier 2: Divisions** – e.g., Traffic, CID, Forensics, Riot Control.
- **Tier 3: Stations** – local operational reporting, custody, case management.
- **Tier 4: Units/Departments** – patrol teams, investigation squads, logistics, armoury.

Each level can report **individually** or as part of **consolidated roll-ups**.

Resources & Units of Measure

- **Personnel** → Officers, detectives, admin staff (measured in FTEs, hours worked).
 - **Assets** → Vehicles, radios, IT systems, firearms, uniforms.
 - **Consumables** → Fuel (litres), ammo (rounds), rations (kg), medical kits (units).
 - **Operations** → Patrol hours, cases handled, arrests, traffic stops.
-

Report Packs

1. **Operational Reports**
 - **Patrol & Deployment:** hours, km covered, hotspots.
 - **Crime & Arrests:** cases registered, arrests made, charges filed.
 - **Traffic & Road Safety:** traffic stops, tickets issued, accidents investigated.
 - **Custody Management:** detainees in/out, cell occupancy %.
 - **Forensics & Investigations:** evidence logged, cases resolved, backlog status.
 2. **Resource & Asset Reports**
 - **Fleet Management:** opening balance (vehicles), fuel usage, repairs, closing balance.
 - **Armoury & Ammunition:** opening stock, issues, returns, shortages.
 - **Uniforms & Equipment:** procurement, distribution, losses, disposals.
 3. **Financial Reports**
 - **Budget Utilization:** allocations vs expenditure by division.
 - **Expenditure Reports:** fuel, rations, overtime, maintenance.
 - **Revenue Reports:** fines, penalties, service fees.
 - **P&L Equivalent** (for internal performance).
 - **Balance Sheet & Cash Flow** (for government audits).
 4. **Compliance & Audit Reports**
 - **Firearm Logs** (serial, issue, return, disposal).
 - **Custody Logs** (chain of custody, human rights compliance).
 - **Case Evidence Chain.**
 - **Audit Pack** for Ministry of Home Affairs, AG's office, donors.
-

Consolidation Rules

- **Same Tier (Horizontal):** Station-to-station transfers (e.g., moving vehicles) are **excluded** from consolidated totals.
 - **Cross-Tier (Vertical):** HQ → Division → Station → Unit is consolidated into **net** usage.
 - **Financial Reconciliation:**
 - Budgets roll up from stations to HQ.
 - Fleet fuel logs reconcile to procurement & supplier invoices.
 - Ammunition issues reconcile to armoury stock.
-

Costing & Transfer Pricing

- **Fuel & Rations** → Allocated by patrol hours, vehicle usage, or headcount.
- **Cases/Investigations** → Activity-based costing (investigation hours, lab tests).

- **Fines/Revenue** → Allocated per unit/station, consolidated at HQ.
-

Example Calculations

- **Patrol Efficiency:** $\text{Km covered} \div \text{litres of fuel used}$.
 - **Case Clearance Rate:** $\text{Cases closed} \div \text{Cases registered} \times 100$.
 - **Cell Occupancy:** $\text{Detainees} \div \text{Available cell spaces} \times 100$.
 - **Budget Utilization %:** $\text{Actual spend} \div \text{Budget allocation} \times 100$.
-

UI/UX: How Users Run These

- **Dashboards:**
 - Commander view → deployments, arrests, cases.
 - Logistics officer → fleet & armoury stock.
 - Finance officer → budgets, spend, compliance.
 - **Reports:** on-demand or scheduled (daily crime stats, monthly fuel reports, quarterly audits).
 - **Controls:** expand/collapse by region, unit, or station.
-

Developer Tasks Checklist

- Create **Personnel Ledger** (deployments, shifts, overtime).
 - Build **Asset Ledger** (vehicles, firearms, equipment).
 - Implement **Operations Ledger** (cases, arrests, patrol hours).
 - Set up **Stock Ledger** (ammo, fuel, rations).
 - Integrate with **GL & audit pack** generation.
 - Automate **compliance reports** (firearm logs, custody).
 - Deploy alerts for **variance exceptions** (e.g., missing ammo, overspent fuel).
-

Final Notes

- This ERP setup ensures **operational accountability + financial transparency**.
- Scales from **single police station** to **entire national force**.
- Supports **government integration** (budgets, audits, donor reporting).
- Can integrate with external systems (RTSA, courts, forensic labs).

Unified Police Service Dashboard – Design Specs

KPIs (Top Cards):

-  **Patrol Coverage %** (hours or km covered vs planned)
-  **Active Deployments** (officers on duty vs available)
-  **Case Clearance Rate %** (closed vs registered)
-  **Fleet Utilization %** (vehicles available vs in repair)
-  **Armoury Variance %** (expected vs actual ammunition)
-  **Budget Utilization %**

Expandable Sections (Tiles):

1. **Crime & Arrests** – reported crimes, arrests, charges filed, case backlog.
2. **Patrol & Deployment** – routes covered, hotspots, officer shift allocation.
3. **Fleet & Fuel** – opening stock of vehicles, fuel issued, km covered, repairs.
4. **Armoury & Equipment** – firearms issued/returned, ammo consumption, shortages.
5. **Finance & Compliance** – budget vs spend, fines collected, donor funding, consolidated P&L & cash flow.
6. **Custody & Detention** – detainees in/out, occupancy rates, human rights compliance log.

Visuals:

- **Bar chart:** Crime types vs solved cases.
- **Line chart:** Patrol coverage trend over time.
- **Pie chart:** Budget spend breakdown (fuel, salaries, equipment, training).
- **Heatmap:** Crime hotspots by geography.

Controls:

- Date range filter.
- Region/Station/Division selector.
- Actual vs Planned toggle.
- Export: Excel, PDF, JSON.

Crime & Arrests Report – Design Specs

KPIs (Top Cards):

- Total crimes reported
- Arrests made
- Charges filed
- Clearance rate %
- Case backlog

Report Sections:

- **Crime Register:** Case ID, type, date, reporting station, status.
- **Arrests Log:** Arrest ID, officer, suspect, charges, custody status.
- **Charges Filed:** Case-to-charge mapping, prosecution updates.
- **Backlog Report:** Pending investigations, court cases.

Visuals:

- Bar chart → Crime categories (theft, assault, fraud, etc.).
- Line chart → Crime trend (daily/weekly/monthly).
- Pie chart → Arrests by outcome (charged, released, ongoing).

Controls:

- Date range picker.
 - Division/station filter.
 - Export (PDF, Excel, JSON).
 - Drill-down: Click crime type → view detailed cases.
-

🔍 Patrol & Deployment Report – Design Specs

KPIs (Top Cards):

- Total patrol hours
- Patrol coverage % (planned vs actual routes)
- Active officers deployed
- Vehicles in operation vs total fleet
- Hotspot coverage %

Report Sections:

- **Patrol Log:** Patrol ID, officers, route, start/end time, km covered, outcome.
- **Deployment Summary:** Division, number of officers deployed, unit type (foot, vehicle, mounted, special).
- **Exceptions Report:** Missed patrols, route deviations, equipment failures.

Visuals:

- Line chart → Patrol coverage trend over time.
- Heatmap → Hotspot coverage on map view.
- Bar chart → Deployments by unit type.
- Pie chart → Vehicle utilization (active vs idle vs in repair).

Controls:

- Date range picker.
 - Division/station filter.
 - Patrol type filter (foot, vehicle, special).
 - Export: Excel, PDF, JSON.
-

ERIK Setup for a Diversified Farming Company

1. Design Goals & Core Principles

- **Unified ERP & Reporting:** Cover livestock, crops, processing, wholesale, retail, and land sales under one logical framework.
- **Variable Tiers:** Each sector (livestock, crops, processing, retail, land sales) has departments and sub-departments at different tiers.
- **Full Traceability:** From feed inputs and veterinary medicine in poultry → to chicken meat in retail shops.
- **Multi-UoM Support:** Units, weights (kg, tons), volume (liters), and monetary values all reconciled.
- **Consolidation Rules:** Eliminate duplication; vertical transfers included, horizontal transfers excluded at same-tier consolidation.

2. Organisational Model

- **Top Tier (Enterprise):** Farming Company consolidated entity.
- **Mid Tier (Sectors):** Poultry, Cattle, Goats, Pigs, Crops, Processing, Retail, Land.
- **Lower Tier (Departments):**
 - Poultry → Hatchery, Layers, Broilers.
 - Cattle → Dairy, Beef.
 - Crops → Wheat, Soya, Potatoes.
 - Processing → Abattoir, Chicken Plant, Beef Plant, Pork Plant.
 - Retail → Butcheries, Shops.
 - Land → Sales, Development.

Each unit posts its own reports but also consolidates upward.

3. Items, Batches & Units of Measure

- **Livestock:** Tracked as batches (flocks, herds). UoM = headcount, weight (kg), liters (milk).
- **Crops:** Tracked in hectares planted, yields in tons.
- **Inputs:** Feed, medicines, fertilizers, seeds → tracked with FEFO and expiry.

- **Processing Outputs:** Carcasses, cuts, milk packs → by batch, lot, and serial.
 - **Sales:** Retail in kg/liters, wholesale in bulk weight/volume.
 - **Land:** Sq. meters/hectares, converted to monetary value.
-

4. 📊 Stock Ledger & Movement Taxonomy

- **Opening Stock** → At each batch start.
 - **Purchases** → Inputs: feed, drugs, seeds, fertilizer.
 - **Interdepartment Transfers:**
 - Vertical → Poultry live birds → Abattoir. Milk → Processing plant. Wheat → Milling.
 - Horizontal → Feed between poultry houses (excluded at same-tier consolidation).
 - **Outputs:** Production yields (milk liters, carcass weight, harvested tons).
 - **Sales:** Wholesale & retail invoices.
 - **Closing Stock:** Livestock in herd/flock, crops in silos, finished goods in warehouse.
-

5. 💰 Costing & Transfer Pricing

- **Livestock:** Cost = feed + vet + housing overhead ÷ units.
 - **Crops:** Cost = seed + fertilizer + irrigation + labor ÷ tons harvested.
 - **Processing:** Transfer price = livestock carrying cost → to abattoir, plus processing overhead allocated.
 - **Retail/Wholesale:** Landed cost allocator = processing cost + packaging + transport.
 - **Land Sales:** Land acquisition + development cost → matched with sales revenue.
-

6. 📄 Report Packs

Production & Operations

- Poultry yield report (chicks → broilers → kg of meat).
- Dairy yield report (liters per cow).
- Crop yield report (hectare → tons).
- Abattoir/processing output report (live weight in → carcass out).

Financial & Stock

- Stock reports (feed, livestock, crops, processed meat, land bank).
- Purchases report (inputs, medicines, fertilizers, packaging).

- Sales report (wholesale, retail, land).
- Surplus/shortage, abnormal losses/gains (disease outbreaks, spoilage).

Enterprise Consolidation

- Profit & Loss, Balance Sheet, Cash Flow.
 - Financial Ratios (gross margin per sector, ROA, livestock conversion rates).
 - Audit Pack (supporting docs for compliance).
-

7. ∞ GL & Finance Integration

- Journals post directly from operational transactions:
 - Purchases → Debit Inventory, Credit Payables.
 - Sales → Debit Receivables/Cash, Credit Revenue.
 - Transfers → Debit Receiving Dept, Credit Sending Dept (eliminated on consolidation).
 - Multi-currency supported (domestic + export).
 - IFRS/GAAP toggles for reporting.
-

8. ✓ Validation & Controls

- **Biometric/role approvals** for transfers & disposals.
 - **Batch reconciliations** → every flock/herd must reconcile feed used vs weight gained.
 - **Crop reconciliations** → expected vs actual yields.
 - **Processing reconciliations** → live weight vs carcass vs cuts.
 - **Land reconciliations** → registered plots vs sold units.
-

9. 🏠 Cross-Industry Extensibility

- **Agriculture** → already covered with crops & livestock.
 - **Processing/Manufacturing** → abattoirs & plants map to factory logic.
 - **Retail/Wholesale** → butcheries align with distribution templates.
 - **Real Estate** → land sales = same as property management in ERP.
-

10. 📱 UI/UX Considerations

- **Unified dashboard** → overview of all sectors (livestock, crops, processing, retail, land).
 - **Drill-downs** → from enterprise → sector → department → batch.
 - **Expand/Collapse Trees** → to move between cattle/dairy vs poultry/broilers, etc.
 - **Filters** → by species, crop, department, location.
 - **Exports** → PDF, Excel, JSON for regulators & auditors.
-

11. Developer Tasks Checklist

- Define schema for livestock, crops, processing, retail, land.
 - Implement stock movement tables (purchases, transfers, outputs, sales).
 - Automate costing engines (batch costing, landed cost allocation, overhead allocation).
 - Build report pack SQL views (sector level, enterprise level).
 - Integrate GL posting engine (with transfer eliminations).
 - Create UI dashboards (summary + drill-down + exports).
 - Test with sample poultry, cattle, crop, and land transactions.
-

12. Example Calculation

- **Poultry:** 1,000 chicks @ \$1 each = \$1,000 opening.
 - Feed & medicine = \$5,000.
 - Mortality = 50 birds.
 - Remaining 950 birds → 1.5kg average weight = 1,425kg meat.
 - Cost per kg = $(\$1,000 + \$5,000) \div 1,425 = \$4.21/\text{kg}$.
 - Transfer to abattoir at \$4.21/kg → further overheads applied → retail sells at margin.
-

13. Final Notes

- **Same ERIK logic applies:** Input → Process → Output → Transfer → Sale → Consolidation.
- Livestock = similar to WIP manufacturing.
- Crops = project costing (seasonal).
- Processing = vertical transfer + overhead allocation.
- Retail = distribution logic.
- Land = property management logic.

With this model, a farming enterprise can track every chicken, cow, goat, pig, ton of wheat, liter of milk, kilo of beef/pork, sack of potatoes, or hectare of land – all unified under **one ERP kernel (ERIK)**.

Livestock Operations Report – Design Specs

KPIs (Top Cards):

- Total headcount (by species: poultry, cattle, goats, pigs)
- Mortality %
- Feed conversion ratio (FCR)
- Average daily weight gain (cattle/pigs)
- Average egg/milk yield (layers/dairy cattle)

Report Sections:

- **Livestock Register:** Species, batch, age group, count, weight, health status.
- **Feed & Vet Consumption:** Feed issued vs consumed, medicine dosage logs.
- **Production Outputs:** Eggs collected, liters of milk, live weight gains.
- **Losses:** Mortality, disease, abnormal culls.

Visuals:

- Line chart → Weight gain trends per species.
- Bar chart → Feed vs output yield.
- Pie chart → Species distribution (stock value or headcount).
- Heatmap → Mortality distribution across batches.

Controls:

- Species filter (poultry, cattle, goats, pigs).
 - Batch/department selector.
 - Date range filter.
 - Export options (Excel, PDF, JSON).
-

Crop Operations Report – Design Specs

KPIs (Top Cards):

- Total hectares planted (wheat, soya, potatoes, other crops)
- Expected vs actual yield (tons)
- Fertilizer usage per hectare
- Irrigation hours / water usage
- Crop loss % (pests, weather, wastage)

Report Sections:

- **Crop Register:** Crop type, planted area, expected harvest, maturity stage.
- **Input Consumption:** Fertilizer, seeds, pesticides, irrigation logs.
- **Production Outputs:** Tons harvested, graded quality (A/B/C).
- **Losses & Variances:** Pests, diseases, weather damage, storage losses.

Visuals:

- Line chart → Yield trends across planting seasons.
- Bar chart → Fertilizer/pesticide input vs yield per hectare.
- Pie chart → Crop share in total output.
- Heatmap → Regional farm performance by crop.

Controls:

- Crop filter (wheat, soya, potatoes, etc.).
 - Field/region selector.
 - Season/date range filter.
 - Export (Excel, PDF, JSON).
-

Processing & Abattoir Operations Report

– Design Specs

KPIs (Top Cards):

- Live animals received (by type: poultry, cattle, pigs, goats)
- Conversion ratio (live weight → carcass weight)
- Processing output (kg of meat, liters of milk processed, packaged units)
- Yield % (usable meat vs waste/offal)
- Processing capacity utilization %

Report Sections:

- **Inputs Log:** Animals received, weight, batch IDs.
- **Processing Outputs:** Carcasses, cuts, milk packs, by-products (hides, bones, fat).
- **Quality Checks:** Passed/failed inspections, rejections, contamination logs.
- **Losses:** Abnormal losses, wastage, condemned stock.
- **Transfers:** Outputs sent to wholesale, retail butcheries, or exports.

Visuals:

- Line chart → Processing throughput over time.
- Bar chart → Live weight vs carcass yield per species.
- Pie chart → Output distribution (wholesale, retail, exports).
- Heatmap → Rejection/condemnation causes.

Controls:

- Species filter (poultry, cattle, pigs, goats).
 - Department selector (abattoir, dairy plant, processing plant).
 - Date range filter.
 - Export: Excel, PDF, JSON.
-

Wholesale & Retail Butchery Sales Report

– Design Specs

KPIs (Top Cards):

- Total sales (by value & weight)
- Gross margin %
- Units sold (kg of beef, chicken, pork, goat)
- Retail vs Wholesale split
- Daily average sales per outlet

Report Sections:

- **Sales Log:** Invoice ID, date, customer type (wholesale, retail), product, UoM, quantity, price, tax.
- **Returns Log:** Returns by product type and reason (expiry, quality, damage).
- **Inventory Flow:** Opening stock, sales, returns, closing stock.
- **Profitability Report:** Revenue, COGS, Gross margin.

Visuals:

- Line chart → Daily/weekly sales trend.
- Bar chart → Sales by product type (chicken, beef, pork, goat).
- Pie chart → Retail vs wholesale distribution.
- Heatmap → Best performing outlets/regions.

Controls:

- Date range picker.
- Filter by outlet (butchery, wholesale warehouse).
- Product category filter.
- Export: Excel, PDF, JSON.

Branding:

- ERIK + Greenomic Enterprise.
 - Compliance footer: Food safety authority, ZRA Smart Invoice, VAT reporting.
-

Land Sales & Development Report – Design Specs

KPIs (Top Cards):

- Total hectares available
- Hectares sold (YTD)
- Revenue from land sales
- Development costs incurred
- Profitability (land sales margin %)

Report Sections:

- **Land Bank Register:** Plot ID, size, location, acquisition cost, status (available, sold, under development).
- **Sales Ledger:** Customer, plot size, price, tax, payment status.
- **Development Ledger:** Roads, utilities, fencing, infrastructure costs allocated per plot.
- **Profit & Loss:** Revenue – Development costs – Land acquisition = Net margin.

Visuals:

- Bar chart → Sales by month.
- Pie chart → Available vs sold vs in development land.
- Heatmap/Map → Land locations and statuses.
- Line chart → Development costs vs sales trend.

Controls:

- Date range picker.
 - Location/region filter.
 - Status toggle (available, sold, development).
 - Export: Excel, PDF, JSON.
-

Farming Finance Dashboard – Design Specs

KPIs (Top Cards):

- Total revenue (livestock + crops + processing + retail + land)
- Operating expenses
- Net profit margin %
- Cash on hand
- Accounts receivable (wholesale/retail customers, land buyers)
- Accounts payable (suppliers, contractors)

Report Sections:

1. **Profit & Loss Statement (P&L):**
 - Revenue by sector (poultry, cattle, crops, processing, retail, land).
 - Direct costs (feed, vet, seeds, fertilizer, utilities).
 - Overheads (salaries, maintenance, marketing).
 - Net profit before/after tax.
2. **Balance Sheet:**
 - Assets (land, livestock, crops in field, equipment, receivables, cash).
 - Liabilities (loans, payables, taxes).
 - Equity.
 - ✓ Assets = Liabilities + Equity validation shown visually.
3. **Cash Flow Statement:**
 - Operating (farm income, sales, expenses).
 - Investing (land development, new equipment).
 - Financing (loans, grants, repayments).
 - Actual vs forecast toggle.

Visuals:

- Bar chart → Revenue vs expenses by sector.
- Line chart → Profit trend over time.
- Pie chart → Expense distribution.
- Heatmap → Accounts receivable aging by customer type.

Controls:

- Date range picker.
- Sector filter (livestock, crops, retail, land).
- Consolidated vs departmental toggle.

- Export (Excel, PDF, JSON).



Audit Pack & Compliance Export – Design Specs

KPIs (Top Cards):

- Number of reports generated in the audit pack
- Last export date
- Pending compliance checks
- Exceptions flagged

Report Sections:

- **Audit Pack Generator:**
 - Select report bundle (Finance, Operations, Sales, Processing, Land).
 - Select time period (monthly, quarterly, yearly).
 - Include supporting schedules (stock reconciliations, yield reports, asset registers).
 - Compliance checks (IFRS, ZRA Smart Invoice, Tax schedules).
- **Exception Report:**
 - Unreconciled stock movements.
 - Duplicate transactions.
 - Missing tax IDs (TPIN).
 - Expired or invalid compliance docs.
- **Export Options:**
 - PDF (for auditors).
 - Excel (detailed schedules).
 - JSON/XML (for regulator submission).

Visuals:

- Progress bar → Audit pack generation progress.
- Timeline → Last 5 exports with status.
- Exception heatmap → By department (livestock, crops, retail, land).

Controls:

- Time period selector.
 - Department/sector toggle.
 - Include/exclude schedules.
 - Export button with format selector.
-

- **KPIs:** Reports in pack, last export date, pending checks, exceptions flagged
- **Sections:** Audit pack generator, exception report, export options
- **Visuals:** Progress bar (generation), Timeline (last exports), Heatmap (exceptions by dept)
- **Controls:** Time period selector, sector toggle, include/exclude schedules, export formats

Consolidated Enterprise Dashboard – Design Specs

KPIs (Top Cards):

- Total enterprise revenue (all sectors combined)
- Net enterprise profit margin %
- Total assets vs liabilities (enterprise-wide)
- Livestock headcount + crop hectares overview
- Processing throughput (tons/day)
- Retail/wholesale sales value
- Land bank (available vs sold)

Report Sections:

- **Cross-Sector P&L:** Revenues and costs by major segment (livestock, crops, processing, retail, land).
- **Balance Sheet:** Enterprise-level assets, liabilities, equity.
- **Cash Flow:** Enterprise-level inflows/outflows with drill-down by sector.
- **Operational KPIs:**
 - Livestock → mortality, yield.
 - Crops → harvest yield, losses.
 - Processing → capacity utilization.
 - Retail → sales split.
 - Land → sales vs development.

Visuals:

- Enterprise-wide bar chart → Revenue vs expenses per sector.
- Line chart → Profitability trends over time.
- Pie chart → Sector contribution to revenue.
- Heatmap → Exception/variance analysis across operations.

Controls:

- Toggle → Consolidated vs sectoral view.
 - Date range picker.
 - Drill-down option (sector → department → unit).
 - Export → Excel, PDF, JSON.
-

Here's a structured write-up showing how the exact same **ERIK ERP set up and logic** can be applied to manage and account for **cemeteries and remembrance parks operations**:

□ Cemetery & Remembrance Park ERP Application (ERIK Logic Adaptation)

1. Unified Framework

Just as with hotels, hospitals, farms, and police services, ERIK's **unified chart of accounts, cost centers, operations tracking, and reporting packs** can be applied to cemeteries and remembrance parks.

- Each **operation** (burials, cremations, memorial sales, landscaping, events, land allocation) is modeled as a department, with **inputs, outputs, and transfers**.
 - Revenue and cost tracking follow the **same dual-entry, compact journal format** with account codes, references, tax type fields, and exchange rate handling.
 - Compliance modules (audit pack, statutory returns, Smart Invoice JSON) ensure regulatory adherence.
-

2. Core Operational Modules

a) Land & Plot Management

- Inventory logic is repurposed to manage **grave plots** as assets.
- Plots move from:
 - **Available stock** → **Reserved** → **Occupied** (upon burial).
- Automated valuation and recognition of sales/revenue on allocation.

b) Burial & Cremation Services

- Similar to a **production order** in manufacturing:
 - Inputs: labor, equipment (diggers, crematorium gas, etc.), materials (coffins, urns).
 - Outputs: burial service completed, memorial service completed.
- Transfer pricing logic applied between cemetery service ops and park management.

c) Memorials & Monuments

- Treated like retail/wholesale items.
- Stocking: tombstones, plaques, flowers, candles.

- Sales invoiced with Smart Invoice compliance, receipts automatically journaled.

d) Landscaping & Park Maintenance

- Managed as a recurring **cost center**.
- Inputs: water, fertilizer, staff wages.
- Outputs: maintained park (recorded as service quality KPI, not sold directly).
- Costs allocated across departments (burial ops, memorial gardens, events).

e) Events & Remembrance Ceremonies

- Like hotel conferences:
 - Bookings tracked with deposits and invoices.
 - Venue usage (chapels, gardens) recorded with time & date allocations.
 - Revenue posted on event completion.

f) Retail & Auxiliary Services

- Cafeterias, flower shops, souvenir shops treated as **retail outlets**.
- POS integration ensures sales are directly journaled.

3. Revenue Streams

1. Plot sales (once-off or long-term leases).
2. Burial services.
3. Cremation services.
4. Monuments, flowers, and accessories.
5. Chapel/event space rentals.
6. Maintenance fees (annual or one-time).
7. Retail & auxiliary (cafes, kiosks).

4. Accounting & Reporting Logic

- **Trial Balance & Ledgers:** Each operation (plots, burials, retail) linked to cost/revenue accounts.
- **Profit & Loss Pack:**
 - Revenue by service line.
 - Direct service costs (labor, consumables).
 - Overheads (landscaping, security).
- **Balance Sheet Pack:**
 - Land/plots treated as inventory until sold.
 - Infrastructure (crematorium, chapels) as fixed assets.

- **Cash Flow Pack:**
 - Installment payments for plots handled via accounts receivable.
 - Operating inflows (services) vs. capital inflows (land development).
 - **Audit Pack:**
 - Expand/collapse trees to drill down by cemetery, garden, or operation.
 - Automated compliance exports (tax, licensing, Smart Invoice JSON).
-

5. ERIK Dashboard (Cemetery/Remembrance Park Context)

- **Unified View:**
 - Available vs. sold plots.
 - Burials & cremations completed this period.
 - Event bookings pipeline.
 - Retail sales snapshot.
 - Maintenance spend vs. budget.
 - **Drilldowns:**
 - Sector → Department → Operation → Transaction.
 - Expand/collapse tree structure across audit, stock, and revenue reports.
-

✓ With this structure, the **cemetery/park operates like a multi-division enterprise:**

- Land sales = inventory management.
- Burials/cremations = service production.
- Monuments/flowers = retail sales.
- Maintenance = overhead allocation.
- Events = hospitality service.

All posting, compliance, reporting, and dashboards use the **exact same ERIK ERP logic**, just adapted to cemetery-specific terminology.

Here's the **full uncropped write-up** on how the exact same ERIK ERP setup and logic can be used to manage and account for a **Cemetery and Remembrance Park's operations**.

Cemetery & Remembrance Park ERP Setup (ERIK Logic Adaptation)

Design Goals & Core Principles

1. **Unified ERP Kernel** – Manage all cemetery/park operations (land plots, burials, cremations, events, retail) under one system.
 2. **Resource Accountability** – Every plot, service, event, and retail item is tracked like stock, services, or assets.
 3. **Multi-UoM & Compliance** – Track hectares, plots, service counts, inventory items, and monetary values, all compliant with IFRS and ZRA Smart Invoice.
 4. **Transparency & Auditability** – Automated reconciliation of land bank, service revenues, and compliance reports.
-

Organisational Model (Variable Tiers)

- **Tier 1 – Enterprise:** Remembrance Parks Company.
- **Tier 2 – Sectors:** Land & Plots, Burial & Cremation, Memorials & Monuments, Events, Retail & Auxiliary, Landscaping & Maintenance.
- **Tier 3 – Departments/Units:**
 - Land → available plots, sold plots, developed areas.
 - Services → burial, cremation, chapel services.
 - Events → remembrance ceremonies, chapel bookings.
 - Retail → flowers, souvenirs, cafes.
 - Maintenance → landscaping, security.

Each level can report individually and consolidate to enterprise totals.

Resources, Batches & Units of Measure

- **Land Plots** → Units (plots), sq. meters, hectares.
- **Burial Services** → Count per service, linked to plot IDs.
- **Cremation Services** → Count per service, UoM = cremations, gas used.

- **Memorials & Monuments** → Inventory (plaques, tombstones, urns).
 - **Events** → Bookings in hours/days, services rendered.
 - **Retail Items** → Stock in units/weight (flowers, candles).
-

Report Packs

1. **Land & Plot Reports**
 - Available vs sold vs reserved plots.
 - Revenue per plot sale/lease.
 - Development cost allocation.
 2. **Burial & Cremation Reports**
 - Services delivered per period.
 - Inputs: labor, materials (coffins, urns).
 - Variance (planned vs actual burials).
 3. **Memorials & Retail Reports**
 - Stock ledger (monuments, flowers, candles).
 - Sales & gross margin per outlet.
 4. **Event & Chapel Reports**
 - Event bookings, deposits, completion.
 - Revenue vs associated costs.
 5. **Financial Reports**
 - Profit & Loss by sector.
 - Balance Sheet (land as inventory until sold, fixed assets as chapels, crematoriums).
 - Cash Flow (plot installments, event payments).
 - Ratios → revenue per hectare, service utilization %.
 6. **Audit Pack**
 - Statutory reports (tax schedules, Smart Invoice JSON).
 - Compliance logs (land registry, funeral authority).
 - Exception logs (duplicate plots, missing TPIN, unreconciled services).
-

Consolidation Rules

- **Same-tier horizontal transfers** (e.g., plots reallocated between sections) excluded from enterprise totals.
 - **Vertical transfers** (land developed → sold plots; services delivered → revenue) included.
 - **All reconciliations** must tie to GL and Balance Sheet.
-

💰 Costing & Transfer Pricing

- **Land plots:** $\text{Cost} = \text{acquisition} + \text{development} \div \text{plots}$.
 - **Burials:** $\text{Cost} = \text{labor} + \text{coffin} + \text{consumables} \div \text{service}$.
 - **Cremations:** $\text{Cost} = \text{gas} + \text{staff} + \text{equipment depreciation}$.
 - **Events:** $\text{Cost} = \text{staff} + \text{utilities} \div \text{booking}$.
 - **Retail:** Landed cost allocation + markup.
-

📊 Example Calculations

- **Plot Sale Margin:** $\text{Sale price} - (\text{acquisition} + \text{development per plot})$.
 - **Service Utilization %:** $\text{Actual burials} \div \text{Capacity (plots per period)}$.
 - **Event Profitability:** $\text{Revenue} - \text{overhead allocation}$.
 - **Retail Margin:** $\text{Revenue} - \text{COGS}$.
-

📱 UI/UX for Users

- Unified dashboard with KPIs: available plots, services delivered, events booked, retail sales.
 - Expand/collapse trees for each park, sector, or service.
 - Date range and consolidation filters.
 - Export reports (Excel, PDF, JSON).
-

⚡ Developer Tasks Checklist

- Create **Land Bank Ledger** with status transitions (available → reserved → sold).
 - Build **Service Ledger** (burials, cremations, events).
 - Add **Retail Ledger** (stock items, sales).
 - Implement **GL integration** for all transactions.
 - Automate **audit pack exports** (Smart Invoice JSON, compliance logs).
 - Add **alerts** for duplicate plot allocations or expired compliance docs.
-

🚩 Final Notes

By applying the **same ERIK logic**:

- Land = inventory.
- Burials & cremations = services.
- Events = hospitality operations.
- Retail = POS sales.
- Maintenance = cost centers.

The cemetery/park achieves **full financial transparency, operational accountability, and compliance readiness** under a single ERP system.

- **KPIs:** Available plots, services delivered, events booked, retail sales, maintenance costs, revenue
- **Tiles/Sections:** Land & Plots, Burials & Cremations, Events, Retail, Maintenance, Finance
- **Visuals:** Bar (services by type), Line (revenue trend), Pie (land status), Heatmap (plot distribution)
- **Controls:** Date range, consolidation toggles, export options

Here's a structured **write-up** of how the **ERIK ERP setup and logic** can be adapted to manage and account for an **Accounting & Consulting Firm's operations**:

Accounting & Consulting Firm ERP Setup (ERIK Logic Adaptation)

Design Goals & Core Principles

1. **Professional Services Framework** – Track engagements (audits, tax advisory, management consulting) as projects with costs, revenues, and KPIs.
 2. **Multi-Dimensional Reporting** – Consolidate by client, industry, partner-in-charge, or service line.
 3. **Resource Efficiency** – Optimize staff utilization, billable hours, and recoveries.
 4. **Transparency & Compliance** – Full audit trail, IFRS & ISA alignment, Smart Invoice-ready for ZRA, and independence monitoring for audit clients.
-

Organisational Model (Variable Tiers)

- **Tier 1 – Firm (Enterprise):** The consulting/accounting practice.
 - **Tier 2 – Service Lines (Sectors):**
 - Audit & Assurance
 - Tax Advisory
 - Management Consulting
 - Corporate Finance / Deals
 - Risk & Compliance
 - Outsourced Accounting / Payroll
 - **Tier 3 – Engagement Teams/Projects:**
 - Client-specific assignments with allocated staff, budgets, timelines, and deliverables.
-

Resources & Units of Measure

- **Human Capital:** billable hours, charge-out rates, roles (partner, manager, associate).
- **Knowledge Assets:** templates, checklists, methodologies.
- **Engagements:** treated like work orders/projects.
- **Deliverables:** reports, audits, filings, presentations.

- **Expenses:** travel, IT tools, software licenses, subcontractors.
-

Report Packs

1. **Engagement Profitability Reports**
 - Budgeted vs actual hours.
 - Billable vs non-billable split.
 - Gross margin per engagement.
 2. **Client Ledger Reports**
 - Fees billed, collections, outstanding balances.
 - Credit notes, retainer balances.
 3. **Service Line P&L**
 - Revenue by audit, tax, advisory, etc.
 - Direct staff costs allocated.
 - Gross margin per service line.
 4. **Resource Utilization Reports**
 - Staff utilization % ($\text{billable} \div \text{total hours}$).
 - Recovery rate % ($\text{fees} \div \text{billable hours value}$).
 5. **Financial Reports**
 - Profit & Loss, Balance Sheet, Cash Flow.
 - Working capital ($\text{WIP} + \text{receivables} - \text{payables}$).
 - Ratios: debtor days, realization rates, profitability per partner.
 6. **Compliance & Audit Pack**
 - Independence checks for audit clients.
 - Tax filings (VAT, PAYE, corporate tax).
 - Smart Invoice JSON for ZRA.
 - Regulatory reports for professional bodies.
-

Consolidation Rules

- Engagements roll up into service lines.
 - Service lines roll up into firm-level consolidated reports.
 - Internal cross-charging (shared staff between projects) tracked but eliminated from consolidated P&L.
-

Costing & Transfer Pricing

- **Billable Hour Costing:** $\text{staff salaries} \div \text{available hours}$.

- **Engagement Pricing:** rate card $\times$ expected hours.
 - **Transfer Pricing:** between service lines (e.g., tax experts supporting an audit).
 - **Overhead Allocation:** IT, rent, HR, apportioned by staff hours or revenue.
-

Example Calculations

- **Utilization Rate:** Billable hours $\div$ Total available hours $\times$ 100.
 - **Recovery Rate:** Fees billed $\div$ Value of billable hours $\times$ 100.
 - **Engagement Margin:** (Fees – Direct staff costs – Allocated overheads) $\div$ Fees.
 - **Partner Profit Share:** Net profit $\div$ Profit-sharing ratio.
-

UI/UX for Users

- **Unified Dashboard:**
 - KPIs: revenue, utilization %, debtor days, realization rates.
 - Drill-down: Firm $\rightarrow$ Service Line $\rightarrow$ Engagement $\rightarrow$ Transaction.
 - **Expand/Collapse Trees:** Clients, engagements, staff allocations.
 - **Engagement Workbench:** Budget, WIP, invoices, collections.
 - **Export Options:** PDF (clients), Excel (partners), JSON (regulators).
-

Developer Tasks Checklist

- Build **Engagement Ledger** (budgets, hours, fees, status).
 - Integrate **Time Sheets** (staff hours).
 - Develop **Invoicing Module** with Smart Invoice JSON.
 - Configure **Resource Utilization Tracker**.
 - Create **Audit Pack Exporter** for compliance.
 - Automate **KPI Dashboards** (realization, utilization, margins).
-

Final Notes

By reusing the **ERIK ERP framework**:

- Engagements = projects/operations.
- Staff hours = input resources.

- Reports/deliverables = outputs.
- Fees = revenue.
- Salaries/overheads = costs.

The accounting/consulting firm gets **real-time visibility into profitability, compliance, and client relationships**, with the same **consolidation, audit, and Smart Invoice-ready logic** as in farms, hospitals, hotels, and cemeteries.

☞ Would you like me to now **generate a Unified Accounting & Consulting Firm Dashboard mock** (with KPIs like utilization %, recovery rate, revenue by service line) as we did for the other industries?

You said:

Yes

ChatGPT said:

✓ Perfect — I'll generate the **Unified Accounting & Consulting Firm Dashboard mock** with:

- **KPIs (Top Cards):**
 - Total firm revenue
 - Utilization %
 - Recovery rate %
 - Outstanding receivables (debtor days)
 - Engagement profitability margin %
 - **Sections:**
 - Service line P&L (Audit, Tax, Advisory, Outsourcing, Deals)
 - Client Ledger & Collections
 - Engagement profitability (budget vs actual hours, margins)
 - Staff utilization & realization
 - **Visuals:**
 - Bar chart → Revenue by service line
 - Line chart → Utilization trend over months
 - Pie chart → Revenue contribution by service line
 - Heatmap → Client receivables aging
 - **Controls:**
 - Date range picker
 - Service line filter
 - Client filter
 - Export options (Excel, PDF, JSON)
-

🎯 Engagement Profitability & Resource Utilization – Design Specs

KPIs (Top Cards):

- Total active engagements
- Average engagement margin %
- Budgeted hours vs actual hours variance
- Staff utilization % (firm-wide)
- Recovery rate %

Report Sections:

1. **Engagement Profitability Table:**
 - Engagement ID, Client, Service Line, Budgeted Fees, Actual Fees, Direct Costs, Gross Margin %.
2. **Time & Resource Tracker:**
 - Billable vs non-billable hours.
 - Partner/Manager/Associate breakdown.
 - Overtime utilization.
3. **WIP & Collections:**
 - Work in progress (unbilled hours).
 - Outstanding receivables linked to engagements.

Visuals:

- Line chart → Engagement profitability trend.
- Bar chart → Budget vs actual hours per engagement.
- Pie chart → Staff time split (billable vs non-billable).
- Heatmap → Profitability by service line.

Controls:

- Filters for service line, client, engagement type.
 - Date range selector.
 - Export (Excel, PDF, JSON).
-

Client & Service Line Performance – Design Specs

KPIs (Top Cards):

- Top 5 clients by revenue
- Client concentration % (top 10 vs rest)
- Average client profitability %
- Service line revenue mix (Audit, Tax, Advisory, Consulting)
- Client retention rate

Report Sections:

1. **Client Performance Table:**
 - Client ID, Client Name, Industry, Revenue, Costs, Margin %, Outstanding Receivables.
2. **Service Line Breakdown:**
 - Audit, Tax, Advisory, Consulting – revenue, costs, margin.
 - Resource allocation by service line.
3. **Cross-Selling Opportunities:**
 - Clients using multiple services.
 - Unserved services flagged as opportunities.

Visuals:

- Bar chart → Top 5 clients revenue.
- Pie chart → Service line revenue mix.
- Trend chart → Client retention over time.
- Matrix heatmap → Profitability by client × service line.

Controls:

- Filters for industry, client tier, service line.
 - Date range selector.
 - Export to Excel, PDF, JSON.
-

Audit Trail & Compliance Monitoring – Design Specs

KPIs (Top Cards):

- Total audit logs recorded
- Number of compliance breaches flagged
- Pending reconciliations
- Percentage of engagements compliant with standards
- Average resolution time for audit issues

Report Sections:

1. **Audit Trail Table:**
 - User ID, Action Taken, Module, Timestamp, Status (success/failure), Notes.
2. **Compliance Dashboard:**
 - Engagements checked vs compliant.
 - Exceptions raised.
 - Resolutions logged.
3. **Monitoring & Alerts:**
 - Real-time alerts for unusual activity.
 - Integration with internal control checklists.

Visuals:

- Line chart → Compliance rate over time.
- Bar chart → Breaches by type.
- Heatmap → User/system activity density.
- Pie chart → Status of reconciliations.

Controls:

- Filters for user, engagement, department, date range.
 - Export to Excel, PDF, JSON.
-

Financial Management & Partner Profitability – Design Specs

KPIs (Top Cards):

- Total firm revenue
- Net profit margin %
- Average debtor days
- Cash balance
- Partner profit allocation summary

Report Sections:

1. **Firm-Wide Financials:**
 - Revenue, direct costs, overheads, operating profit.
 - Variance analysis (budget vs actual).
2. **Partner Profitability Breakdown:**
 - Partner name, revenue attributable, costs allocated, profit share %.
 - Distribution summary with payout schedule.
3. **Cash Flow Monitoring:**
 - Inflows from clients.
 - Outflows (salaries, overheads, partner drawings).
 - Net cash flow trends.

Visuals:

- Bar chart → Revenue vs costs by month.
- Pie chart → Profit allocation by partner.
- Line chart → Cash flow over time.
- Heatmap → Overheads by department.

Controls:

- Filters for partner, service line, time period.
 - Export options: Excel, PDF, JSON.
-

⚙️ ERP Setup & Accounting Logic for a Law Firm

1. Design Goals & Core Principles

- **Unified framework:** Treat each practice area, client matter, and department as “operations,” similar to sectors/tiers in ERP.
- **Traceability:** Each case/matter has its own ledger, expenses, billable hours, and income flows.
- **Consolidation:** Firm-wide financials can be rolled up while ensuring no double counting of interdepartmental allocations (e.g., shared research hours).
- **Compliance:** Align with legal trust account regulations (client money, retainer handling).
- **Cross-industry extensibility:** Built on same stock, ledger, and reporting logic as farming, police, hotel, or healthcare examples.

2. Organizational Model

- **Variable tiers:**
 - Tier 1: Practice Groups (Corporate Law, Criminal Law, Family Law, Tax Advisory, Litigation, Arbitration, etc.).
 - Tier 2: Departments (Research, Paralegal, Advocacy, Advisory).
 - Tier 3: Cases / Client Matters.
- **Units of Measure (UoM):** Billable hours, fixed fee packages, retainer amounts, court filing costs, advisory notes, consultancy deliverables.

3. Items & Resources (Analogy to “Stock”)

- **Inputs:** Partner hours, associate hours, paralegal time, external counsel, expert witnesses, filing fees, court charges, research tools.
- **Outputs:** Legal opinions, contracts, filed pleadings, successful judgments, client reports.
- **Batches/Lots:** Each case file is treated like a “batch” with unique identifier (Case ID, Client Matter Code).

4. Movement Taxonomy

- **Purchases:** Office supplies, external counsel, expert services, legal databases.
 - **Transfers:**
 - *Horizontal:* Reassignment of staff hours between cases in the same department.
 - *Vertical:* Movement of cases from advisory to litigation (advisory outputs become litigation inputs).
 - **Production:** Drafting contracts, preparing pleadings, conducting hearings.
 - **Sales:** Invoicing clients (time & material, fixed fees, retainers).
 - **Closing:** Matter closure, WIP → revenue recognition.
-

5. Report Packs (adapted for Law Firms)

- **Operational Reports**
 - Case/Matter Profitability Report
 - Billable vs Non-billable Hours Report
 - Staff Utilization Report
 - Trust Fund & Retainer Balances Report
 - **Financial Reports**
 - Profit & Loss (by practice area, by client, by matter)
 - Balance Sheet (including trust liabilities)
 - Cash Flow (operating, financing, investing activities)
 - AR Aging Report (clients with outstanding invoices)
 - **Compliance Reports**
 - Client Funds Audit Pack
 - Expense Reimbursements
 - Court Fee Tracking
-

6. Consolidation Rules

- Firm-level roll-ups must include:
 - No duplication of hours when associates are reallocated across matters.
 - Interdepartmental research allocations removed on consolidation.
 - Trust accounts shown separately from operational accounts.
-

7. GL Integrations

- **Chart of Accounts** aligned to:
 - Income: Billable hours, Fixed fees, Success fees.
 - Expenses: Salaries, Filing fees, Expert fees, Subscriptions.
 - Trust Liabilities: Retainers held, unbilled disbursements.

- Assets: WIP (Work In Progress), Debtors.
-

8. Example Calculation

- Case ID: CRIM-2025-005
 - Inputs:
 - 20 Partner hrs × \$200/hr = \$4,000
 - 40 Associate hrs × \$100/hr = \$4,000
 - Court filing = \$500
 - Output Invoice: \$10,500 (includes markup on disbursements).
 - P&L Impact:
 - Revenue: \$10,500
 - Direct Costs: \$8,500
 - Profit: \$2,000
-

9. UI/UX Flow

- **Unified Dashboard:** Firm overview (active cases, fees earned, outstanding AR, trust balances).
 - **Expand/Collapse:** By practice area → department → case.
 - **Case Screen:** Shows inputs (hours, expenses), outputs (opinions, pleadings), invoices, and trust movements.
 - **Audit Pack:** Exportable for Law Association compliance audits.
-

10. Developer Tasks Checklist

- Configure **multi-tier case hierarchy**.
 - Build **time tracking integration** (hours → WIP → invoices).
 - Implement **trust account logic** (client deposits separate from operating accounts).
 - Adapt **existing stock ledger logic** into **hour ledger** for billable time.
 - Add **matter profitability engine** (like yield reports).
 - Build **retainer consumption automation** (drawdown on trust balances).
-

11. Final Notes

- The law firm setup is **not a departure** from ERP stock & transfer logic—it's a **direct adaptation**:
 - Stock = Staff time + Legal outputs.
 - Transfers = Case handovers.
 - Production = Legal services delivered.
 - Sales = Client invoices.
 - Guarantees: Transparency, compliance, and profitability analysis at both **case level** and **firm level**.
-

The **Law Firm ERP module** will focus on **Case/Matter Profitability Reports**:

- **KPIs (Top Cards):** Active cases, Avg. billable rate, Avg. utilization %, Profit margin %, Overdue invoices.
- **Tables:**
 - Case ID, Client, Practice Area, Billable Hours, Costs, Fees Billed, Profit Margin %.
 - Expand/collapse for detailed view by partner/associate.
- **Visuals:**
 - Bar chart: Profit per case.
 - Heatmap: Utilization by staff.
 - Line graph: Fees vs costs over time.
- **Controls:** Filters for client, matter type, time period.
- **Export:** Excel, PDF, JSON.

⚖️ Trust Account & Retainer Balances – Design Specs

KPIs (Top Cards):

- Total Trust Balances Held
- Active Retainers
- Average Drawdown %
- Trust-to-Fees Ratio
- Pending Reconciliations

Report Sections:

1. **Trust Ledger**
 - Client, Case ID, Deposit Date, Amount Held, Amount Utilized, Balance.
 - Expandable detail for disbursements.
2. **Retainer Management**
 - Retainer agreement details, allocated hours, hours consumed, remaining credit.
 - Alerts for depletion thresholds (e.g., below 20%).
3. **Compliance View**
 - Segregated client funds (not mixed with operating).
 - Reconciliation audit pack export.

Visuals:

- Donut chart → Distribution of trust balances by client.
- Line chart → Retainer drawdowns over time.
- Heatmap → Clients approaching depletion.

Controls:

- Filters: client, practice area, time period.
 - Export: Excel, PDF, JSON.
-

Staff Utilization & Realization – Design Specs

KPIs (Top Cards):

- Average Utilization % (billable vs available hours)
- Realization Rate % (billable vs billed)
- Productivity Index
- Overtime Hours Logged
- Top 5 fee earners

Report Sections:

1. **Staff Utilization Table**
 - Staff Name, Role (Partner, Associate, Paralegal), Available Hours, Billable Hours, Utilization %, Non-Billable Hours.
2. **Realization Table**
 - Staff Name, Billable Hours, Hours Billed, Realization %, Fees Recovered, Write-offs.
3. **Engagement View**
 - Breakdown of staff allocation by case.
 - Expand/collapse per matter.

Visuals:

- Bar chart → Utilization % by staff.
- Line chart → Realization trend over months.
- Heatmap → Over/under-utilization by department.
- Pie chart → Billable vs non-billable hours firm-wide.

Controls:

- Filters: staff role, department, date range.
 - Export options: Excel, PDF, JSON.
-

Client & Case Billing – Design Specs

KPIs (Top Cards):

- Total Billed Amount (current period)
- Total Collected Amount
- Outstanding Receivables
- Average Collection Period (days)
- Write-offs & Discounts

Report Sections:

1. **Billing Table**
 - Client, Case ID, Practice Area, Invoice No., Date, Fees Billed, Amount Collected, Balance Due.
 - Expandable by invoice line items (hours, expenses).
2. **Collections Tracker**
 - Aging buckets: 0–30, 31–60, 61–90, 90+ days.
 - Client-wise receivables summary.
3. **Write-offs & Adjustments**
 - Cases with partial recoveries.
 - Discounts approved vs actual applied.

Visuals:

- Bar chart → Fees billed vs collected by client.
- Line chart → Billing vs collections trend.
- Pie chart → Receivables aging distribution.
- Heatmap → Top overdue clients.

Controls:

- Filters for practice area, client, matter type, date range.
 - Export to Excel, PDF, JSON.
-

⚖️ Litigation & Case Progress – Design Specs

KPIs (Top Cards):

- Total Active Litigation Cases
- Average Case Duration (days)
- Win Rate %
- Upcoming Court Dates
- Settlements Achieved

Report Sections:

1. **Case Progress Table**
 - Case ID, Client, Practice Area, Stage (Filed, In Court, Judgment, Closed), Days Open, Responsible Lawyer, Next Hearing Date.
 - Expandable to show pleadings, evidence, and tasks completed.
2. **Milestones & Deadlines**
 - Timeline of hearings, filings, submissions.
 - Alerts for missed deadlines.
3. **Outcome Tracking**
 - Cases Won, Lost, Settled, Ongoing.
 - Linked to profitability per case.

Visuals:

- Gantt chart → Case timelines.
- Bar chart → Cases by stage.
- Pie chart → Outcome distribution (Won, Lost, Settled, Ongoing).
- Heatmap → Workload distribution by lawyer/department.

Controls:

- Filters for practice area, lawyer, client, court, date range.
 - Export: Excel, PDF, JSON.
-

Knowledge Management & Precedents – Design Specs

KPIs (Top Cards):

- Total Precedents Stored
- Number of Templates Used This Month
- Research Hours Logged
- Knowledge Sharing Index (%)
- Top Used Legal Templates

Report Sections:

1. **Precedent Library**
 - Case Type, Document Name, Author, Last Updated, Usage Count.
 - Expandable view to preview or download.
2. **Research Activity**
 - Hours logged by department or lawyer.
 - Linked to engagements.
3. **Knowledge Utilization**
 - Templates/documents reused in active matters.
 - Reduction in drafting time.

Visuals:

- Bar chart → Precedent usage by practice area.
- Line chart → Research hours logged over time.
- Pie chart → Templates used vs newly drafted docs.
- Heatmap → Knowledge contributions by lawyer.

Controls:

- Filters for practice area, author, document type, date range.
 - Export: Excel, PDF, JSON.
-

⚖️ Legal Compliance & Risk Monitoring – Design Specs

KPIs (Top Cards):

- Total Compliance Checks Performed
- Pending Compliance Tasks
- Regulatory Deadlines This Month
- % Compliance Achieved
- Risk Alerts Raised

Report Sections:

1. **Compliance Task Tracker**
 - Matter ID, Client, Regulation (e.g., AML, GDPR, Local Law), Due Date, Responsible Lawyer, Status (Open, In Progress, Completed, Overdue).
2. **Risk Monitoring**
 - Identified risks per case (Conflict of Interest, Filing Deadline Risk, Regulatory Breach).
 - Linked mitigation measures.
3. **Audit Trail**
 - Logs of compliance checks completed.
 - Date/time/user stamps for accountability.

Visuals:

- Calendar view → Regulatory deadlines.
- Bar chart → Compliance tasks completed vs pending.
- Pie chart → Risk categories.
- Heatmap → Department risk exposure.

Controls:

- Filters by regulation, practice area, lawyer, client, date range.
 - Export: Excel, PDF, JSON.
-

□ ERP Setup & Logic for Mining Operations

1. 🎯 Design Goals & Core Principles

- **Unified view** of mining operations from pit to processing plant to export.
- **Multi-tier organizational structure** to handle multiple sites, shafts, pits, concentrators, and smelters.
- **Traceability & accountability** across units, weights (tons), volumes (cubic meters), grades (% purity), and monetary values.
- **Cross-industry extensibility** to apply the same consolidation, costing, and reporting logic across other resource-intensive industries.
- **Regulatory compliance** with mining licenses, environmental laws, royalties, and taxation (e.g., mineral royalty, export duties).

2. 🏢 Organizational Model (Variable Tiers)

- **Tier 1 – Enterprise:** Mining holding company (e.g., “Greenomic Mining Ltd”).
- **Tier 2 – Sectors:** Divisions such as Open-Pit Mining, Underground Mining, Processing Plant, Smelter, Exploration Unit.
- **Tier 3 – Departments:** Within sectors (e.g., Drilling, Hauling, Crushing, Refining, Export Logistics).
- **Tier 4 – Units/Projects:** Individual pits, shafts, processing lines, or contracts.

Each tier can consolidate upward, with **vertical transfers allowed** (e.g., ore → concentrator → smelter → warehouse) but **horizontal transfers excluded from consolidation** (e.g., moving trucks between pits in the same tier).

3. 📦 Items, Batches & UoM

- **Items:** Ore, concentrate, refined metal, explosives, fuel, spare parts, protective gear.
- **Batch attributes:** Mine block, ore grade, shift date, lot number, assay certificate.
- **UoM:** Tonnes, kg, liters, cubic meters, ounces, and monetary units.

4. 📊 Stock Ledger & Movement Taxonomy

Movements mirror manufacturing logic:

- **Opening Stock** (ore, consumables, finished metal).
 - **Purchases** (chemicals, explosives, equipment).
 - **Vertical Transfers In/Out** (pit → plant → smelter).
 - **Production Outputs** (concentrate, anodes, cathodes, bullion).
 - **Sales/Exports** (local customers, export shipments).
 - **Closing Stock** (ore stockpiles, refined product inventory).
 - **Consignment In/Out** (third-party smelters or contractors).
 - **Returns, Expiry, Scrap** (damaged stock, unusable ore, expired chemicals).
-

5. 💰 Costing & Transfer Pricing

- **Landed Cost Allocator** → imports of equipment, chemicals, spare parts.
 - **Activity-Based Overhead Allocation** → maintenance, energy, labor distribution across shafts/plants.
 - **Scenario Costing** → simulate ore grades, recovery % changes, and price fluctuations.
 - **Transfer Pricing** → ore valued at standard cost when sent to concentrator/smelter to ensure financial transparency.
-

6. 📊 Report Packs

Mining ERP will produce:

- **Stock Reports** → ore mined, concentrate produced, metals exported.
 - **Purchases Reports** → explosives, chemicals, fuel, equipment.
 - **Production/Yield Reports** → recovery rates, grade vs output variance.
 - **Abnormal Loss Reports** → accidents, low recovery, theft, dilution.
 - **Surplus/Shortage Reports** → unexplained variances in tonnage or grade.
 - **Sales Reports** → local vs export sales, royalties due.
 - **Financial Reports** → P&L, Balance Sheet, Cash Flow, Financial Ratios (debt/equity, operating margin, recovery cost/ton).
 - **Audit Pack** → statutory compliance, royalty calculations, environmental fees.
-

7. ∞ GL & Compliance Integrations

- Automatic posting into **General Ledger**:
 - Mineral royalty tax.

- Export duty.
 - VAT on consumables.
 - Integration with **Smart Invoice & ZRA compliance**.
 - Environmental & sustainability reporting frameworks (GRI, ESG).
-

8. ✓ Validation Controls

- **Assay certificate match** between lab and production records.
 - **Tonnes mined vs. tonnes hauled** reconciliation.
 - **Dual approval** for high-value purchases (equipment, contracts).
 - **Audit trail** for production, sales, and tax filings.
-

9. 🌐 Cross-Industry Extensibility

This same schema can be adapted to:

- Agriculture (crops & livestock flows).
- Hospitality (room nights vs service revenue).
- Healthcare (patients, consumables, diagnostics).

The **mining sector** simply adapts the transfer and costing logic to ore → concentrate → refined product → sale.

10. □ Example Calculations

- **Recovery %** = $(\text{Metal content out} / \text{Metal content in}) \times 100$.
 - **Cash cost per tonne** = $(\text{Direct mining} + \text{processing costs}) \div \text{tonnes produced}$.
 - **Net smelter return** = $\text{Gross metal value} - (\text{treatment \& refining charges} + \text{royalties})$.
-

11. 📱 UI/UX (User Journey)

- **Unified Dashboard:** Shows production KPIs, recovery %, royalties due.
- **Report Selector:** Users choose stock, yield, sales, financials.
- **Drill-down Navigation:** From enterprise view → site → department → unit.
- **Export Options:** PDF, Excel, JSON for regulators & auditors.

12. ✨ Developer Tasks Checklist

- Define **schema tables** (stock ledger, costing, transfer pricing, sales, GL links).
- Build **SQL procedures** for movement tracking & reconciliations.
- Implement **FastAPI endpoints** for reports.
- Create **Streamlit/HTML dashboards** for field managers.
- Automate **tax & royalty posting** to GL.
- Integrate with **assay lab systems** for ore grade validation.

13. 📌 Final Notes

The mining ERP follows **the exact same DNA** as we've applied to farming, hotels, hospitals, and law firms:

- A **movement ledger** capturing every input/output.
 - **Tiered consolidation** with clear vertical transfer rules.
 - **Comprehensive report packs** tied back to GL.
 - **Extensible logic** that works across industries.
-